Constraint-driven architecture evolution

Evolving e-commerce platform

A Java monolith and PostgreSQL on one server, changed one measured constraint at a time.

Six phases, 51 steps

Phase	Steps	Focus
1 Single server and correctness	01–10	Invariants, query and connection limits, storage, caching, identity
2 Availability and deployment	11–19	Failure of processes and zones, recovery point, releases, spikes, observability
3 Messaging and distributed workflows	20–28	Queues, events, long-running checkout, safe retries
4 Specialized data and database scale	29–35	Replicas, write saturation, document, graph and time-series data, analytics
5 DDD boundaries and service separation	36–44	Domain boundaries, service ownership, gateways, orchestration

Phase 1

Single server and correctness

Steps in this phase

Step	Forcing constraint
01 Protect inventory invariant	Two buyers purchase the last item at once; confirmed stock cannot be negative
02 Make product queries selective	Search p95 above 300 ms; plan shows full-table scans
03 Stop exhausting DB connections	300 concurrent requests consume all DB connections
04 Separate binary storage	2 TB of images must survive server replacement
05 Reduce global static asset delay	Distant clients wait 800 ms for images
06 Avoid repeat reads	90% of calls reread unchanged catalog records
07 Preserve stock correctness behind cache	Cached availability is stale; confirmed orders must not oversell
08 Reduce client round trips	Mobile page latency violates UX target
09 Switch order updates from polling	50,000 clients require notice within 2 seconds
10 Consolidate identity	Customers, admins and employees need separate authorization and SSO

A01 · Single server and correctness

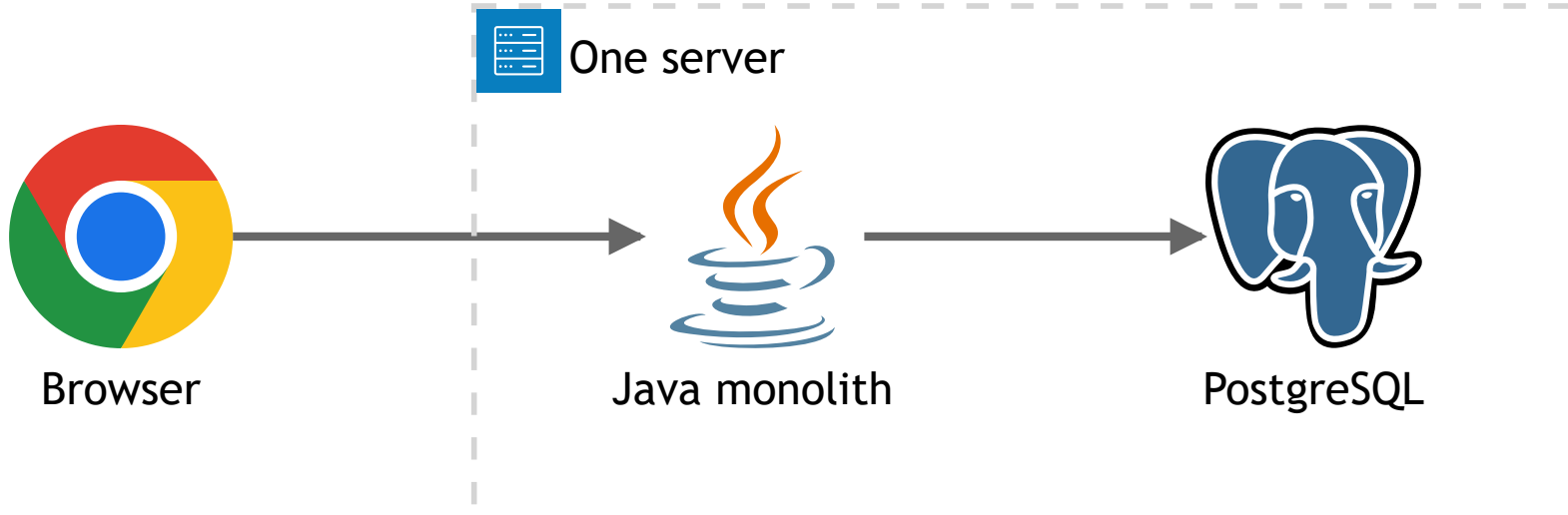
Protect inventory invariant

Two buyers purchase the last item at once. Confirmed stock cannot be negative.

Monolith + PostgreSQL on one server

Component	What it is	What it does
Browser / SPA	Customer-facing web client	Shows catalog and checkout; calls the REST API
Java monolith	One deployable application in one process	Serves REST; runs Catalog, Pricing, Inventory, Orders, Payment, Fulfillment, Notifications
Inventory module	Code that owns stock levels	Reads and decrements the available quantity
Orders module	Code that owns the order lifecycle	Confirms an order once stock and payment are accepted
PostgreSQL	Relational transactional database	Stores catalog, stock and orders durably; provides ACID transactions
One server	A single physical or virtual machine	Hosts the monolith and the database

Current architecture



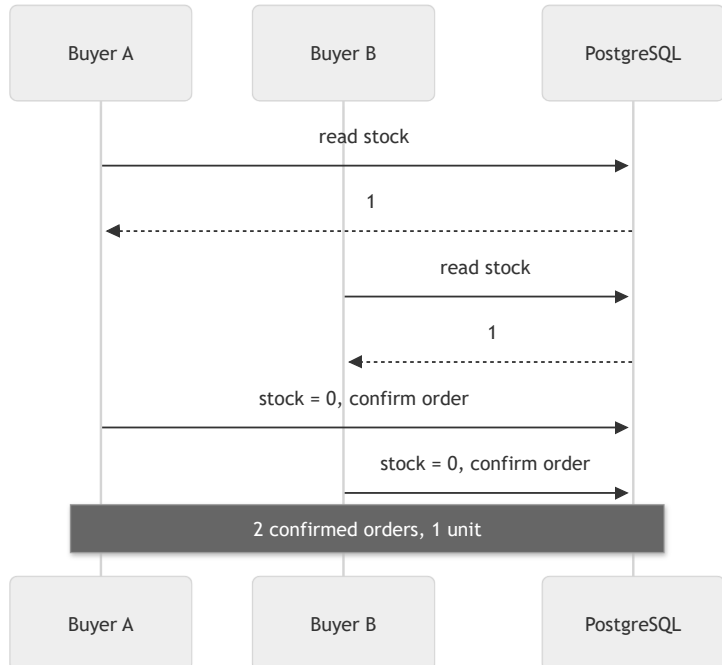
Two buyers, one last item

A limited drop has **1 unit** left. Buyer A and Buyer B press *Buy* in the same second. Both see stock = 1, both orders are confirmed, one customer receives a cancellation.

stock ≥ 0
invariant: confirmed quantity never exceeds physical stock

1000 buyers, 10 units
concurrent purchase load

0 oversold orders
target



Three ways to guard the last unit

Solution	What it is	What it does
ACID transaction + row locking	Stock decrement and order insert inside one DB transaction that locks the stock row	Concurrent buyers of the same product wait their turn; the second one sees the updated stock
Optimistic locking	A version number on the stock row, checked when writing	Takes no lock while reading; a conflicting write updates 0 rows and is retried or rejected
Reservation pattern	Stock is temporarily held for a buyer before payment completes	Hold turns into a sale on payment and is released on timeout

Each guard pays for correctness differently

Solution	Gains	Costs
ACID transaction + row locking	Correct by construction No new component	Buyers of one product queue on one row Lock wait grows with load
Optimistic locking	No locks held High concurrency while conflicts are rare	Hot product means many failed writes Retry or reject logic in the application
Reservation pattern	Stock stays protected through a long checkout	Holds must expire and be released Abandoned carts block stock More order states to model

Deciding signal: write-conflict rate on hot products and how long checkout must hold stock.

Smallest safe move

Atomic stock decrement on one route

- **Scope:** only the order-confirm route changes
- **Change:** stock decrement and order insert in one transaction
- **Safety net:** `CHECK (available ⩾ 0)` on the stock table
- **Escape route:** a feature flag keeps the old path while validation runs
- **Deferred:** reservation, until checkout must hold stock longer than one transaction can

```
UPDATE inventory
   SET available = available - :qty
 WHERE sku = :sku AND available ⩾ :qty;   -- 0 rows updated ⇒ sold out
```

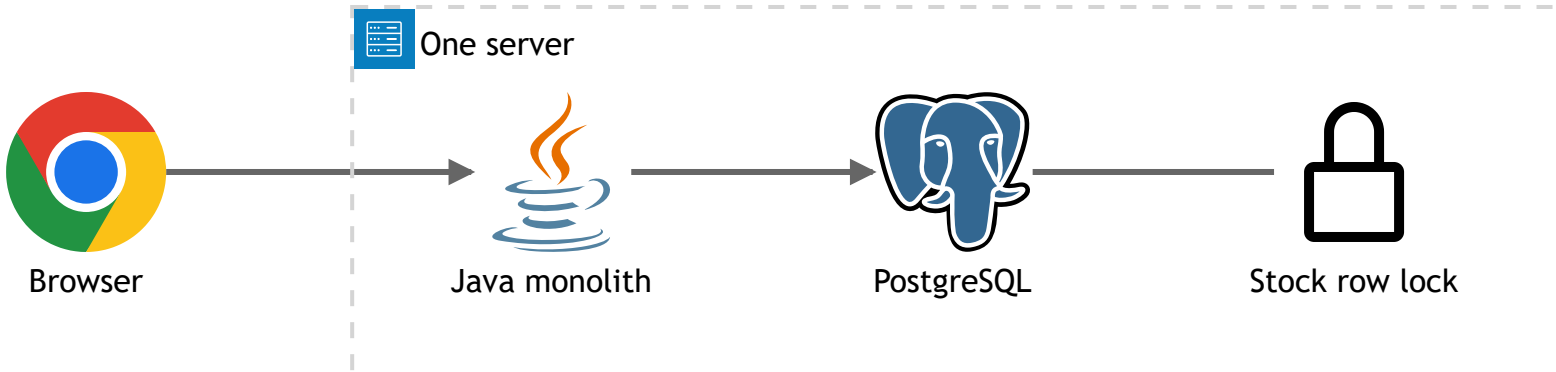
Verification: 1000 buyers, 10 units

Check	Pass condition
Confirmed orders	Exactly 10
Other requests	Answered "sold out", no server errors
Stock after the run	0, never negative in any sample
Checkout p95 and lock wait	Recorded and compared with the baseline run on the old path
Fitness function	The same test runs in CI and fails the build on any oversold order

Rollback: flag off reverts routing only. Orders confirmed in the meantime stay, so oversold ones need cancellation or refund as forward repair.

Stop: when the invariant holds and p95 is on target, no further layer is added.

Solution architecture



Same topology, no new component. Stock decrement and order insert share one transaction.

Glossary

Used in this step

Term	Definition
Invariant	Business rule that must hold at every moment
Overselling	Confirming more units than physically exist
Race condition	Outcome depends on the timing of concurrent requests
Monolith	Entire application deployed as a single unit
PostgreSQL	Open-source relational database with ACID transactions
ACID transaction	Group of changes that commit together or not at all
Row lock	Lock on one row; other writers wait for release
Optimistic locking	Write succeeds only if the row version is unchanged since read
Reservation pattern	Temporarily claim a resource until the workflow completes
Fitness function	Automated check that an architectural property still holds
p95	Latency that 95% of requests stay under

Related concepts

Term	Definition
Single writer	Only one component may modify a given piece of data
Distributed lock	Lock shared across processes through an external coordinator
Saga pattern	Chain of local transactions with compensating actions
Eventual consistency	Replicas converge over time; reads may be stale
Stale cache	Cached value older than the source of truth

A02 · Single server and correctness

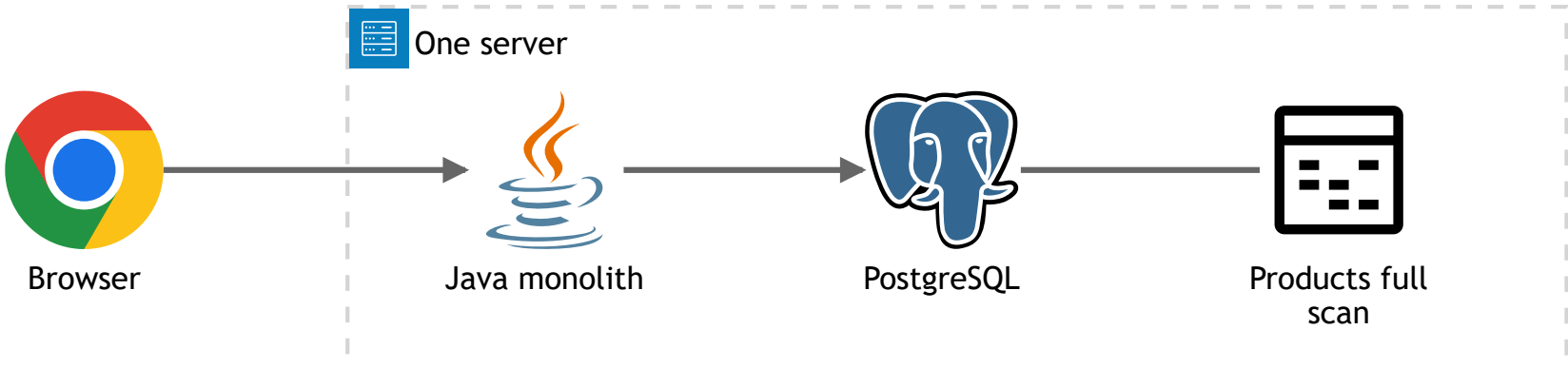
Make product queries selective

Search p95 exceeds 300 ms and the query plan shows full-table scans.

Catalog search runs as plain SQL on PostgreSQL

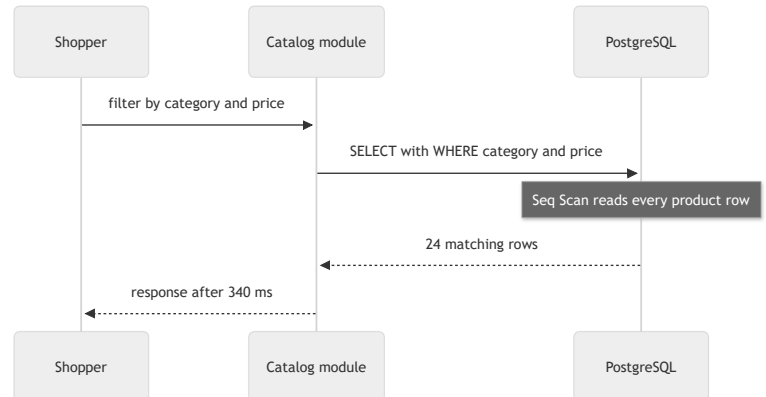
Component	What it is	What it does
Browser / SPA	Customer-facing web client	Sends search and filter requests: category, price range, text
Java monolith	One deployable application on one server	Hosts every capability, including the catalog
Catalog module	Code that owns product data	Builds the SQL for each search and returns a page of results
PostgreSQL	Relational transactional database	Stores products, categories and prices; executes every query
Query planner	The PostgreSQL part that chooses how to run a query	Picks a sequential or index scan from table statistics; `EXPLAIN` shows the choice

Current architecture



Every search reads every row

Shoppers filter by category and price. The primary database runs near 90% CPU from reads, and `EXPLAIN` shows a sequential scan over the whole products table for each search.



p95 above 300 ms
search latency today; the latency SLO asks for 200 ms

Seq Scan
plan for every filtered search

1200 QPS, 5% writes
read-heavy; prices change every 30 seconds

Three ways to make the query cheaper

Solution	What it is	What it does
Database index	Sorted lookup structure on chosen columns, such as category and price	Lets the planner jump to matching rows instead of scanning the whole table
Denormalization	Copying or reshaping data, such as category name and price, into the table that search reads	Removes joins from the hot search query
Materialized view	Query result stored as a table and refreshed on demand or on a schedule	Serves search from the stored result without re-running the query

Every read speed-up shifts cost to writes or freshness

Solution	Gains	Costs
Database index	Fast selective reads No application change; the planner uses it transparently	Every insert and update also maintains the index Extra storage; helps only queries that match its columns
Denormalization	Fewer joins Fast targeted reads	Duplicate values that must be kept in sync Update logic spreads across the code
Materialized view	Expensive joins and aggregates paid once Cheap reads	Refresh lag: a price change is invisible until refresh Refresh cost and storage

Deciding signal: read/write ratio, filter selectivity and the maximum acceptable age of a price.

Smallest safe move

One index for the slowest search

- **Scope:** the single slowest search query, found from query statistics
- **Change:** composite index on the filtered columns, equality column first, built without blocking writes
- **Safety net:** `EXPLAIN (ANALYZE)` on production-sized data confirms the planner uses the index
- **Escape route:** drop the index; no data is touched
- **Deferred:** denormalization and materialized view, until the index leaves p95 above target

```
CREATE INDEX CONCURRENTLY idx_product_category_price
    ON product (category_id, price);
```

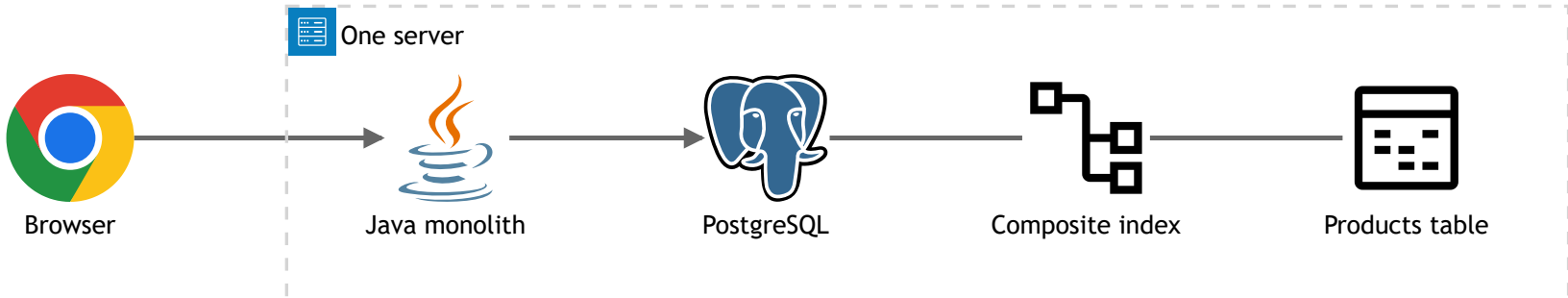
Verification: same workload, before and after

Check	Pass condition
Query plan	Seq Scan replaced by an index scan for the search query
Search p95	At or below 200 ms under replayed production traffic
Primary CPU	Lower than baseline at the same QPS
Write cost	Insert and update latency recorded before and after; regression within the agreed budget
Fitness function	Plan test on representative data volume fails the build when the search query falls back to a sequential scan

Rollback: dropping the index restores the previous behavior; there is no data to reconcile.

Stop: when p95 meets the target, no cache, replica or view is added.

Solution architecture



Same components, one new access path: the planner reads the index and fetches only the matching rows.

Glossary

Used in this step

Term	Definition
Selectivity	Share of rows a filter keeps; fewer rows, bigger index benefit
Sequential scan	Reading every row of a table to evaluate a filter
Query planner	Chooses how a query runs, based on table statistics
Database index	Data structure supporting efficient access paths
Composite index	Index on several columns; column order matters
Index maintenance	Extra work every insert and update does to keep an index current
Denormalization	Duplicate or reshape data to simplify high-frequency queries
Normalization	Reduce redundancy using disciplined relation design
Materialized view	Precomputed query result stored for efficient reads
Read/write ratio	Reads per write; decides whether slower writes are acceptable
p95	Latency that 95% of requests stay under

Related concepts

Term	Definition
Read replica	Database copy serving read operations to distribute query load
Distributed cache	Shared cache accessible by multiple application instances
Database sharding	Partitions data across multiple database instances
Query precomputation	Store results of expensive computations for later fast retrieval
Materialized query projection	Read-optimized representation built from updates or events

A03 · Single server and correctness

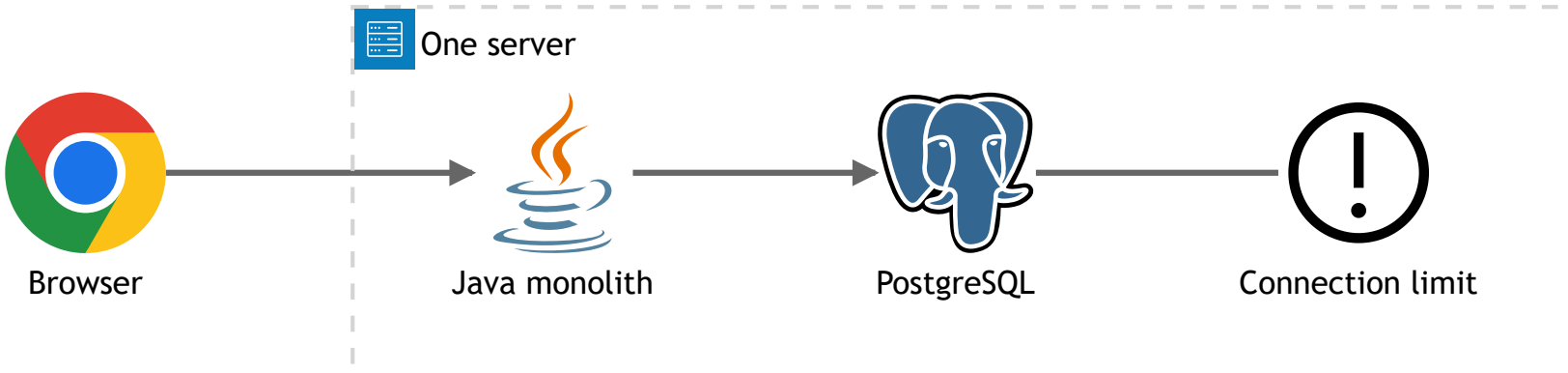
Stop exhausting DB connections

300 concurrent requests consume every database connection.

Every request opens its own database connection

Component	What it is	What it does
Browser / SPA	Customer-facing web client	Sends catalog, cart and checkout requests
Java monolith	One process with a thread per request	Opens a connection, runs its queries, closes the connection
Database connection	A session between application and database	Costs a network handshake, authentication and a server process on PostgreSQL
PostgreSQL	Relational transactional database	Serves queries; accepts at most `max_connections` sessions at once
One server	A single physical or virtual machine	Shares CPU and memory between the application and the database

Current architecture



Request 101 never gets a session

A promotion starts. 300 requests are in flight at once and each one opens a connection. PostgreSQL stops accepting sessions at its limit, so new requests fail or wait and checkout times out.

300 concurrent requests

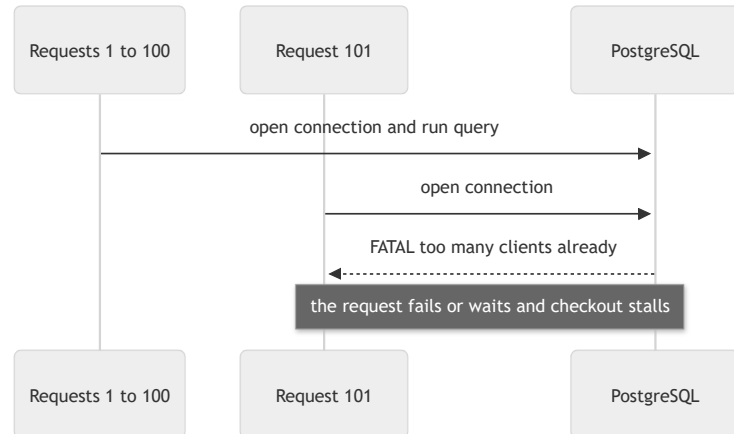
load that exhausts the connection budget

0 connection errors

target: no pool timeouts at 300 clients

p95 within SLO

latency stays inside the target while waiting for a connection



Three ways to cap connections

Solution	What it is	What it does
Connection pool	A fixed set of open connections inside the application, such as HikariCP	Request threads borrow a connection and return it; excess requests wait instead of opening new sessions
Database connection pooler	A proxy between applications and PostgreSQL, such as PgBouncer	Multiplexes many client connections onto few database sessions and caps the total at the database
Larger database	A bigger instance with a higher connection limit	Allows more concurrent sessions by adding hardware

Capping connections trades waiting for capacity

Solution	Gains	Costs
Connection pool	Removes the handshake cost per request One configuration change, no new component	Pool contention: threads wait when it is undersized Every instance has its own pool, so total = instances × pool size
Database connection pooler	One central cap across all application instances Protects the database as the application scales out	Extra network hop and one more component to run Transaction pooling limits session-level features
Larger database	Easy, no code change	Costly Each connection still costs memory, so the limit only moves

Deciding signal: connection budget against peak concurrency, number of application instances and use of session-level features.

Smallest safe move

A bounded pool inside the monolith

- **Scope:** the one data source configuration of the monolith
- **Change:** replace connection-per-request with a bounded pool and an acquire timeout
- **Sizing:** pool size is derived from what the database can run in parallel, not from the number of requests
- **Safety net:** pool size × application instances stays below `max_connections` minus admin headroom
- **Escape route:** pool size and timeout are configuration; revert the setting
- **Deferred:** external pooler, until several instances need one shared cap

```
maximumPoolSize=20
connectionTimeout=2000   # ms a request waits for a free connection
```

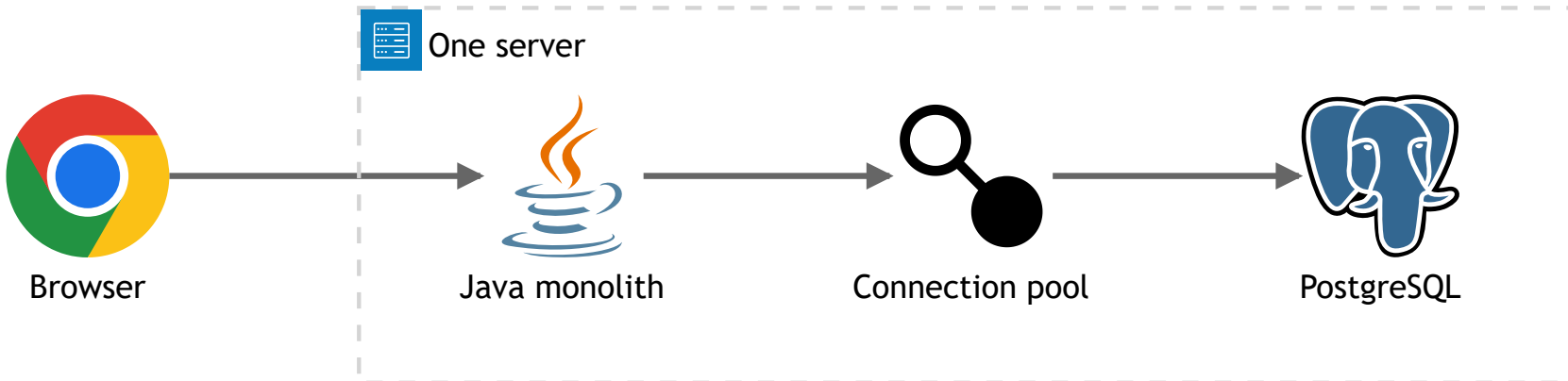
Verification: 300 concurrent clients

Check	Pass condition
Load	300 concurrent clients sustained for the full test
Pool timeouts	0
Database sessions	Never above the pool size; no "too many clients" errors
Latency	p95 within SLO; pool wait time recorded separately from query time
Fitness function	Load test fails on any connection error; alert when pool usage nears its limit

Rollback: reverting the configuration restores connection-per-request; no state is involved.

Stop: when there are no pool timeouts and p95 is on target, no external pooler or larger database is added. Adding application replicas multiplies the pool and needs a new check.

Solution architecture



Same components, one bounded set of reusable connections between the monolith and the database.

Glossary

Used in this step

Term	Definition
Database connection	Session between an application and the database; a server process in PostgreSQL
max_connections	PostgreSQL limit on concurrent sessions
Connection handshake	Network, authentication and session setup paid by every new connection
Connection pool	Reuses database or network connections to reduce overhead
Pool contention	Threads wait because every pooled connection is busy
Acquire timeout	Longest wait for a free pooled connection before the request fails
Database connection pooler	Proxy that multiplexes application connections to database sessions
Transaction pooling	Pooler mode that lends a database session only for one transaction
Vertical scaling	Add CPU or memory to one host
Concurrency	Number of requests in progress at the same moment

Related concepts

Term	Definition
Horizontal scaling	Distribute workload across more instances
Load balancer	Distributes incoming requests across multiple servers or instances
Bulkhead pattern	Isolates resources so failures do not spread across components
Timeout	Limits how long an operation may wait for completion
Read replica	Database copy serving read operations to distribute query load

A04 · Single server and correctness

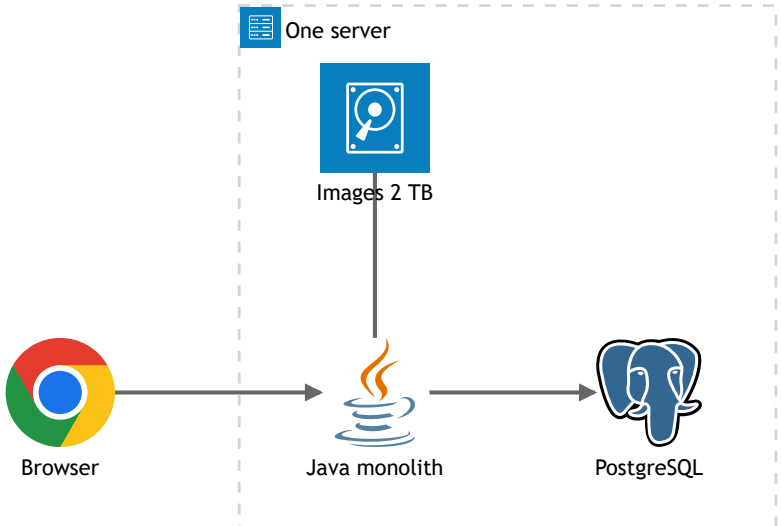
Separate binary storage

2 TB of product images must survive server replacement.

Product images live on the application server’s disk

Component	What it is	What it does
Browser / SPA	Customer-facing web client	Loads product images by URL while rendering catalog pages
Java monolith	One deployable application on one server	Serves REST and image files; writes new uploads to local disk
Application disk	The server’s local file system	Holds 2 TB of product images next to the application
PostgreSQL	Relational transactional database	Stores product data and the file path of each image, not the bytes
One server	A single physical or virtual machine	Ties the lifetime of the images to the lifetime of the machine

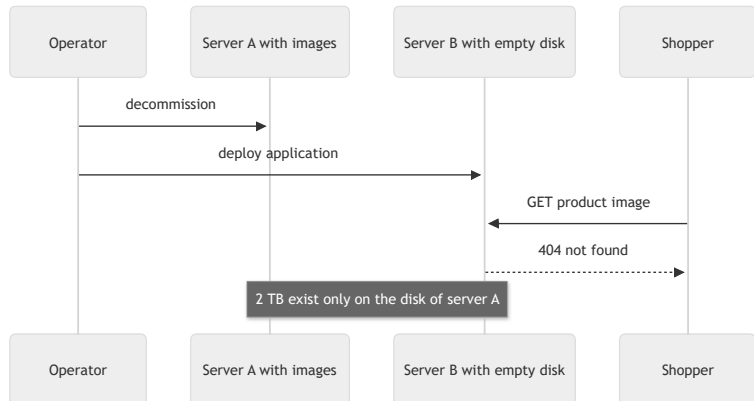
Current architecture



All 2 TB of product images sit on the same server as the application, so replacing the server removes them.

A new server starts with no images

The server must be replaced: a disk fails, the OS is upgraded or the application moves to a bigger machine. The images existed only on the old disk, so the new instance serves broken product pages.



2 TB of images
must outlive any single server

0 images lost
when the application instance is recreated

Same image URLs
pages keep working after the migration

Three places to keep image bytes

Solution	What it is	What it does
Object storage	Bucket-based service for files and binary objects, such as S3-compatible storage	Keeps each image as an object under a key; the application stores only the key
Shared filesystem	A network file system, such as NFS, mounted by the application servers	Presents one common directory tree that every instance reads and writes
Database BLOB	Image bytes stored in a PostgreSQL column	Keeps images in the same transactions and backups as the product rows

Where the bytes live decides who pays

Solution	Gains	Costs
Object storage	Scales independently of the application Application instances become disposable	Egress and request charges New dependency with its own credentials and network latency
Shared filesystem	Familiar file semantics Little or no application change	Operational burden: mounts, locking, failover of the file server The file server becomes a single point of failure unless built for HA
Database BLOB	Transactional with product data One backup path	Bloats the database: backups, restores and replication slow down Static bytes compete with queries for memory and I/O

Deciding signal: data volume and growth, read pattern, need for transactional coupling with product rows and transfer cost.

Smallest safe move

New uploads go to object storage

- **Scope:** the image read and write path only; REST contract and image URLs stay the same
- **Change:** store new uploads in a bucket and keep the object key in PostgreSQL; copy existing files in the background
- **Safety net:** reads fall back to local disk for any key missing in the bucket until the copy is verified by count and checksum
- **Escape route:** a feature flag returns the write path to local disk
- **Deferred:** shared filesystem and database BLOB; delivery speed is the next constraint

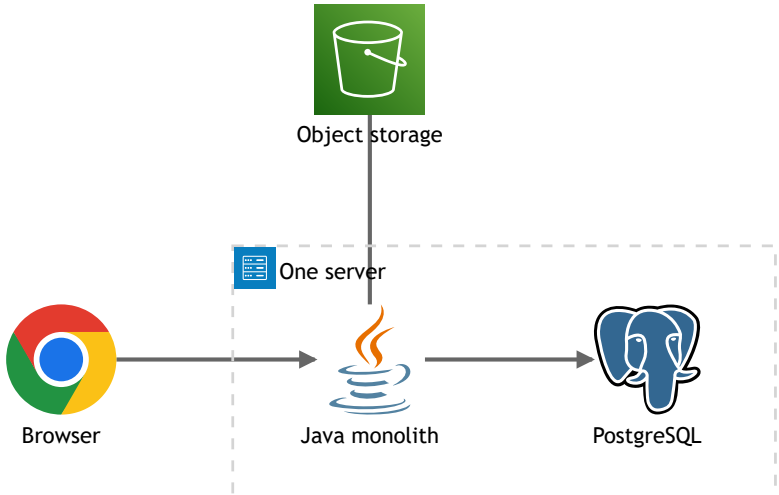
Verification: replace the application instance

Check	Pass condition
Instance recreation	A fresh instance with an empty local disk serves every sampled product image
Completeness	Object count and checksum sample match the files on the old disk
Latency	Image p95 recorded against the local-disk baseline
Cost	Monthly storage, request and egress cost recorded
Fitness function	Scheduled job starts an instance with an empty disk and fetches sample images

Rollback: the flag is a routing rollback only. Images uploaded to the bucket meanwhile are not on disk, so they are copied back or the bucket stays the read source.

Stop: when images survive instance replacement at acceptable p95 and cost, no other store is added.

Solution architecture



Same application, but the 2 TB of images now sit outside the server and outlive it.

Glossary

Used in this step

Term	Definition
Binary object	File content stored as raw bytes, such as an image or PDF
Object storage	Stores files and binary objects in scalable buckets
Bucket	Named container of objects inside object storage
Object key	Name that addresses one object within a bucket
Shared filesystem	Network file system mounted by several servers at once
BLOB column	Database column holding raw bytes
Egress	Data transferred out of a storage service, usually billed
Stateless instance	Application instance that keeps no local data and can be replaced freely
Checksum	Fingerprint of file bytes that proves two copies are identical
Hot, warm and cold storage	Tiers chosen by access frequency and acceptable retrieval time
Virtual machine	Isolated operating system running on virtualized hardware

Related concepts

Term	Definition
CDN	Distributes cached content closer to users
Backup and restore	Recovery process using historical snapshots and logs
Multi-region replication	Maintain data copies across geographically separate cloud regions
Database sharding	Partitions data across multiple database instances
Data lake	Stores large volumes of raw structured and unstructured data

A05 · Single server and correctness

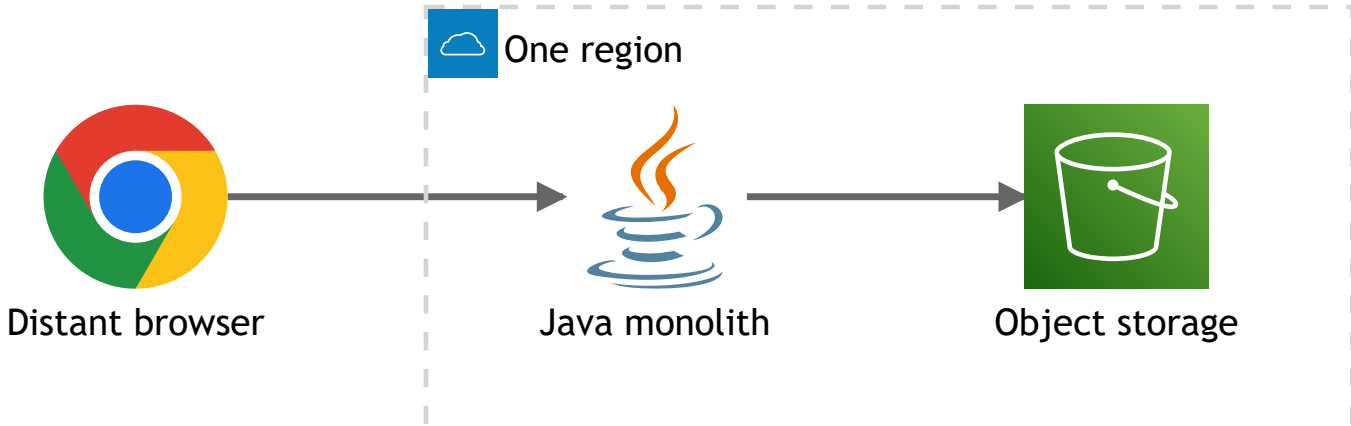
Reduce global static asset delay

Distant clients wait 800 ms for product images.

One region serves every asset

Component	What it is	What it does
Browser / SPA	Customer-facing web client, used worldwide	Loads the page, then every product image by URL
Java monolith	One deployable application	Serves REST and image requests from its single region
Object storage	Bucket holding the product images	Stores the image bytes in the same region as the application
Cloud region	Geographic area containing cloud infrastructure facilities	Hosts the whole platform in one place
DNS	Name resolution service	Resolves the shop’s domain to the one origin address

Current architecture



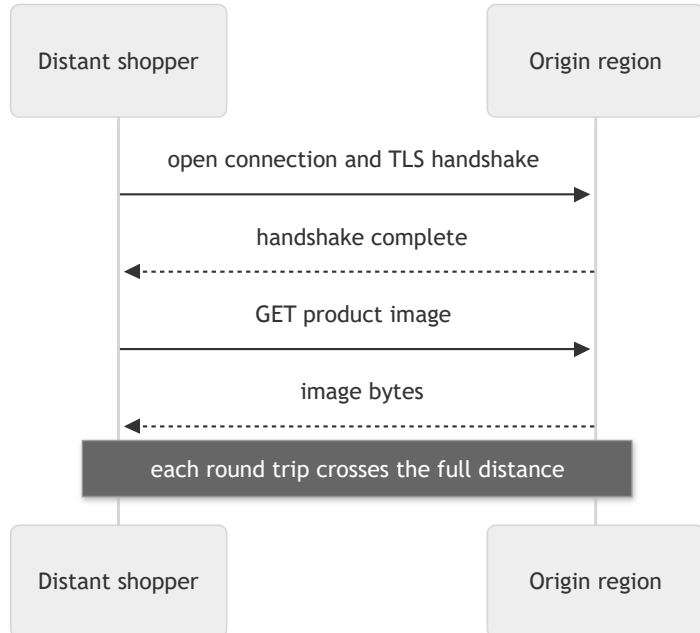
Distance is the delay

A shopper far from the origin region opens a product page. Every image request travels the full distance, and each step pays the round trip again, so one image takes 800 ms.

800 ms
wait per image for distant clients

p95 by user region
the metric to improve, reported separately from nearby users

0 stale images
an updated image must not stay wrong in any region



Three ways to move content closer

Solution	What it is	What it does
CDN	Network of edge servers that cache content close to users	Serves images from the nearest edge and contacts the origin only on a cache miss
Multi-region deployment	Full copies of the application and data in several cloud regions	Routes each user to the nearest region for static and dynamic requests
Edge computing	Application code running on nodes near users	Executes logic, such as resizing or authorization, next to the user, not only caching files

Proximity is bought with invalidation, duplication or state

Solution	Gains	Costs
CDN	Low latency for static content Offloads the origin; no application change	Cache invalidation: stale images after a change A cache miss still pays the full distance
Multi-region deployment	Dynamic requests become local too Also improves regional availability	Data duplication and cross-region synchronization Cost and operations multiply per region
Edge computing	Logic runs next to the user	Distributed state and deployment complexity Limited runtime and harder debugging

Deciding signal: geographic spread of users, static against dynamic share of the distant traffic and how often assets change.

Smallest safe move

A CDN in front of image URLs only

- **Scope:** static image requests; dynamic API calls stay on the origin
- **Change:** put a CDN in front of the origin and serve images with long cache lifetimes under versioned URLs
- **Safety net:** a changed image gets a new URL, so no invalidation is needed; purge by key covers mistakes
- **Escape route:** point the image host back at the origin, which still serves everything
- **Deferred:** multi-region and edge computing, until dynamic requests dominate remote latency

```
Cache-Control: public, max-age=31536000, immutable
```

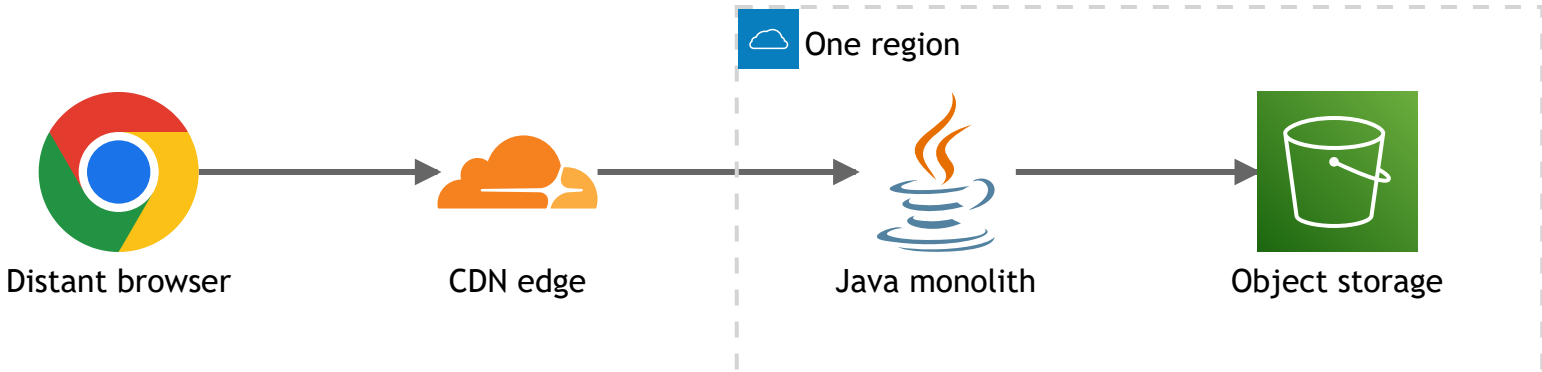
Verification: p95 per region, hits and misses

Check	Pass condition
Image p95 by region	Distant clients drop from 800 ms to the agreed target, measured from probes in several regions
Hit and miss	Cache hit ratio and hit p95 reported separately from miss p95
Stale content	After an image update the new version appears in every region within the agreed delay
Origin load and cost	Origin requests and transfer cost recorded before and after
Fitness function	Synthetic probes from distant regions alert on p95 or stale-content regression

Rollback: routing back to the origin changes no data; cached copies expire by lifetime.

Stop: when distant p95 meets the target, multi-region and edge computing are not added.

Solution architecture



Same origin region, one new node near the user: cache hits never travel the full distance.

Glossary

Used in this step

Term	Definition
CDN	Distributes cached content closer to users
Origin	Server that holds the authoritative copy of content
Edge location	CDN data center near users that caches and serves content
Cache hit and miss	Served from the cache, or fetched from the origin
TTL	How long a cached copy counts as fresh
Cache invalidation	Removing or replacing cached copies after content changes
Cache-Control	HTTP header that declares who may cache a response and for how long
Versioned URL	URL that changes when the content changes, so old copies are never reused
Round-trip time	Time for a request to reach a server and the reply to return
Cloud region	Geographic area containing multiple cloud infrastructure facilities
Edge computing	Execution close to user or source

Related concepts

Term	Definition
Multi-region replication	Maintain data copies across geographically separate cloud regions
DNS	Resolves domain names to network addresses
Reverse proxy	Intermediary that forwards client requests to backend services
Cache-aside pattern	Application explicitly reads from and populates a cache
Stale cache	Cached value older than the source of truth

A06 · Single server and correctness

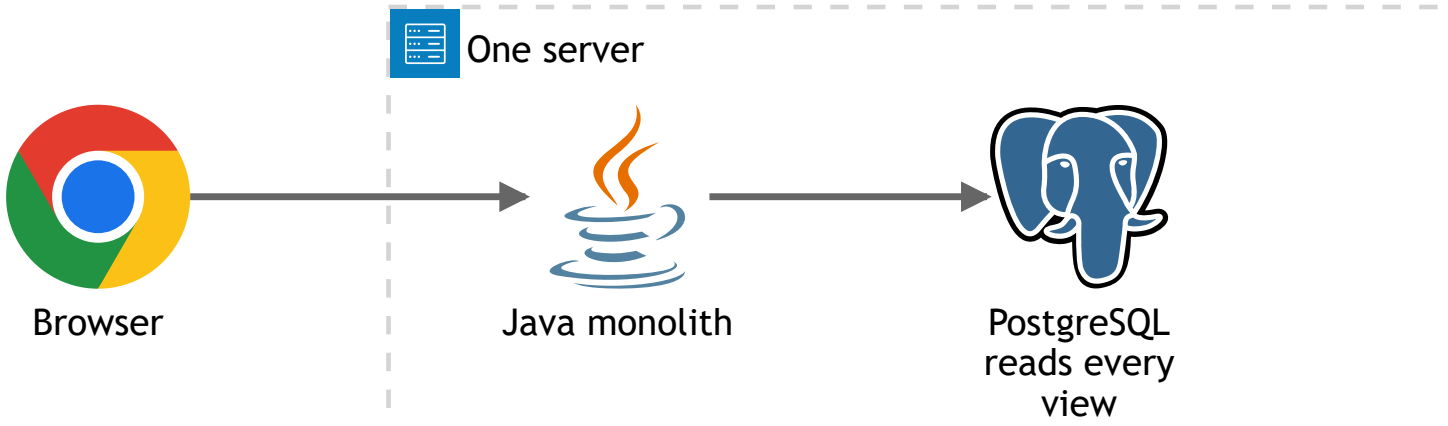
Avoid repeat reads

90% of calls reread unchanged catalog records, and each one still queries PostgreSQL.

Every product view reads PostgreSQL

Component	What it is	What it does
Browser / SPA	Customer-facing web client	Opens product pages; popular products are viewed again and again
Catalog API	Catalog module inside the Java monolith	Fetches the product record for every product view
Product record	Name, description and attributes of one product	Changes rarely; read many times between changes
PostgreSQL	Relational transactional database	Runs one query per view and is the source of truth for every value
One server	A single physical or virtual machine	Shares CPU and memory between the monolith and the database

Current architecture



The same row is read again and again

Popular products are viewed many times between edits. Each view runs the same query for a record that has not changed, so database read work grows with traffic instead of with change.

90% repeat reads

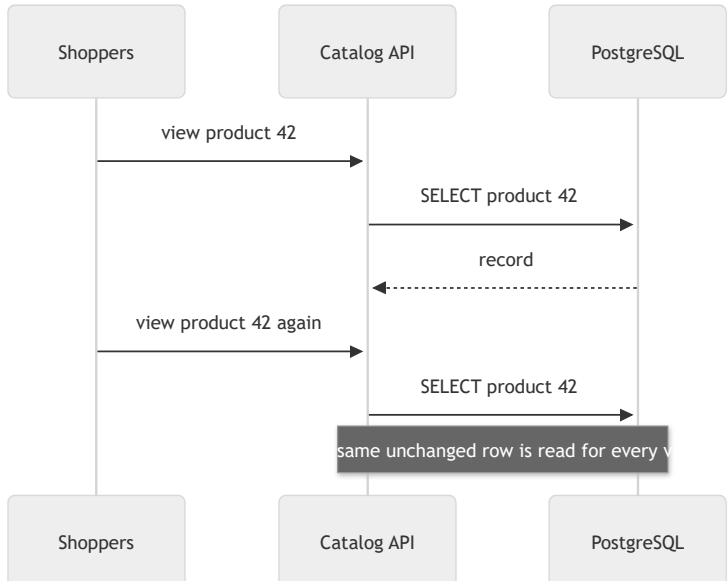
calls that reread an unchanged catalog record

1200 QPS, 5% writes

read-heavy load; reads dominate database work

90% cache hits

target: most views leave the database alone, within the agreed staleness



Three ways to serve a repeated read

Solution	What it is	What it does
Cache-Aside	Application explicitly reads from and populates a cache	Serves repeated reads from memory and sends only misses to PostgreSQL
Distributed Cache	Shared cache that several application instances can reach, such as Redis	Keeps one copy of hot records outside the application process
Direct DB read	Every read goes to PostgreSQL, the source of truth	Returns the current value and pays the full query cost each time

Fewer database reads cost freshness or a new dependency

Solution	Gains	Costs
Cache-Aside	Low latency on a hit: no database query No new component while the cache lives in the process	Invalidation: an entry can be stale until it expires or is evicted Cold start and stampede: concurrent misses for one key reach the database together
Distributed Cache	One shared copy for every application instance Survives an application restart	Extra dependency to run, secure and monitor Network hop on every read; a cache outage sends the full load to the database
Direct DB read	Always current: PostgreSQL is the source of truth No invalidation logic to get wrong	Repeated cost: unchanged rows are queried again and again Read load grows with traffic, not with change

Deciding signal: read repetition, cache hit rate, maximum acceptable staleness, cost of a data change and the number of application instances.

Smallest safe move

Cache-aside for product descriptions on one server

- **Scope:** description reads on the product-view path; price, stock and writes stay on SQL
- **Change:** check an in-process cache by product id; on a miss read SQL and store the record with a TTL
- **Safety net:** every miss reads authoritative SQL; an update evicts its entry; the cache never confirms stock at purchase
- **Escape route:** a flag bypasses the cache; entries are derived data, so nothing needs restoring
- **Deferred:** distributed cache, until a second application instance needs one shared view

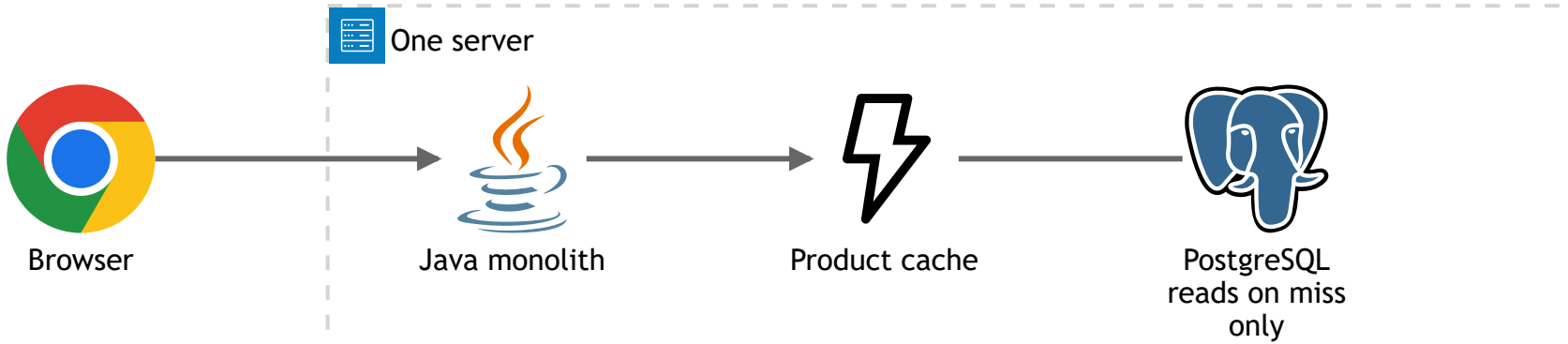
```
Product p = cache.getIfPresent(id);        // hit: no database read
if (p == null) {
    p = repository.findById(id);           // miss: authoritative SQL
    cache.put(id, p);                      // expires after the TTL
}
```

Verification: hit rate, p95 and update-to-visible delay

Check	Pass condition
Hit rate	About 90% of product views served from the cache under replayed production traffic
Database load	Product reads per second and primary CPU fall against the baseline at the same QPS
Latency	Hit p95 and miss p95 reported separately; overall p95 within the latency target
Update-to-visible delay	A changed description appears within the agreed maximum age
Stale content	No cached description older than the agreed age during the replay
Fitness function	Replay test fails the build when hit rate drops below target or a stale description exceeds the agreed age

Rollback: switching the flag off sends every read to PostgreSQL; cached entries are derived data and are dropped, so no state is reconciled.

Solution architecture



Same components, one new node: the monolith asks the cache first and reads PostgreSQL only on a miss.

Glossary

Used in this step

Term	Definition
Cache-aside pattern	Application explicitly reads from and populates a cache
In-memory cache	Stores frequently accessed data in memory for faster retrieval
Distributed cache	Shared cache accessible by multiple application instances
Direct database read	Every read goes to the database, the source of truth
Cache hit and miss	Served from the cache, or fetched from the database
Hit rate	Share of reads served from the cache
TTL	How long a cached copy counts as fresh
Cache invalidation	Removing or replacing cached copies after content changes
Stale cache	Cached value older than the source of truth
Cache stampede	Many concurrent misses for one key reach the database together
Source of truth	The one store whose value wins when copies disagree
p95	Latency that 95% of requests stay under

Related concepts

Term	Definition
Read replica	Database copy serving read operations to distribute query load
Write-through cache	Writes data to the cache and underlying storage together
Materialized view	Precomputed query result stored for efficient reads
Database index	Data structure supporting efficient access paths
Database sharding	Partitions data across multiple database instances

A07 · Single server and correctness

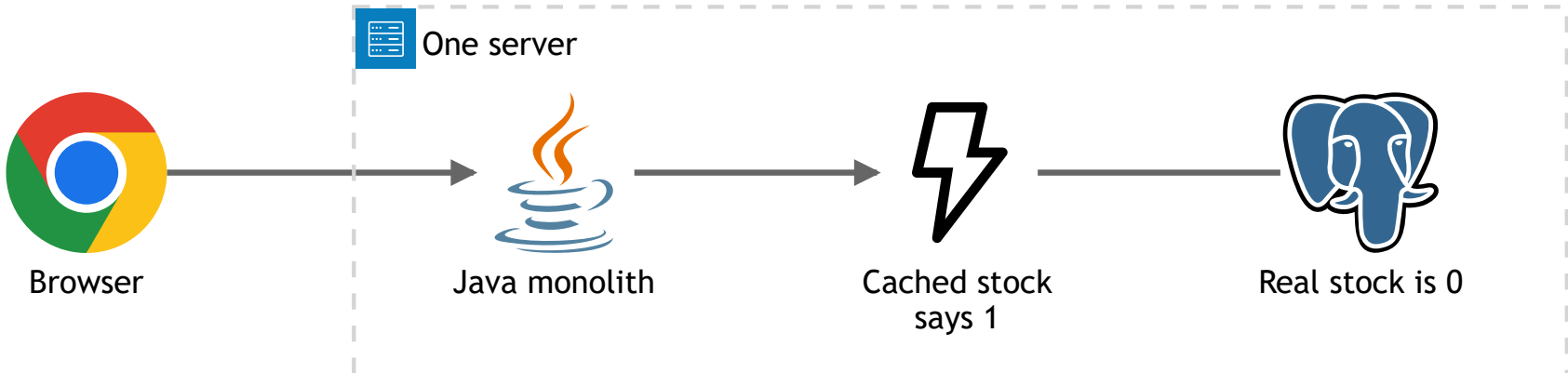
Preserve stock correctness behind cache

Cached availability is stale, and confirmed orders must never exceed physical stock.

Catalog and stock are cached together

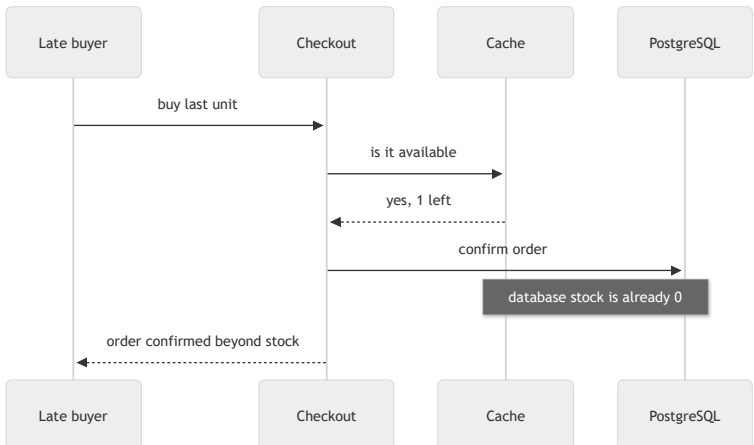
Component	What it is	What it does
Browser / SPA	Customer-facing web client	Shows product pages and sends the purchase
Checkout	Order-confirm route of the Java monolith	Reads availability from the cache and approves the order when stock looks sufficient
Product cache	Cache-aside copy of product pages, stored by product id	Holds description and availability together; entries expire by TTL
TTL	Time a cached entry counts as fresh	Bounds the age of a cached value but cannot make it current
PostgreSQL	Relational transactional database	Holds the authoritative stock and the orders

Current architecture



The cache says 1, the database says 0

The last unit sells, but the cached entry keeps reporting one left until it expires. Buyers arriving in that window are approved on stale data, and confirmed orders exceed physical stock.



Cache 1, database 0

stale availability approves a purchase

500 buyers, 10 items left

concurrent load against a stale product cache

0 oversold orders

target: the invariant violation count stays zero

Three ways to keep stock correct behind a cache

Solution	What it is	What it does
Authoritative transaction on purchase	Stock check and decrement inside one database transaction	Decides every purchase from the database; the cache never approves an order
Write-Through Cache	Writes data to the cache and underlying storage together	Updates the cached stock on every change so readers see fresh values
TTL alone	Cached entries expire after a fixed time	Limits how old a cached stock value can get, with no write-path code

Only an authoritative decision protects the last unit

Solution	Gains	Costs
Authoritative transaction on purchase	Correct by construction: the database decides The cache may lag without affecting correctness	Every purchase pays for a database transaction Buyers of one hot product queue on one row
Write-Through Cache	Readers see stock changes sooner Updates are shared by every reader of the cache	Every stock write also writes the cache The two stores can still diverge after a partial failure, so it cannot approve a purchase
TTL alone	Simple: one setting, no write-path code Fast reads	A window of stale stock remains for the whole TTL Cannot enforce a strict invariant; a shorter TTL gives back the database reads

Deciding signal: the stock age the business accepts at purchase, oversold orders under injected staleness and write contention on hot products.

Smallest safe move

The database decides, the cache only displays

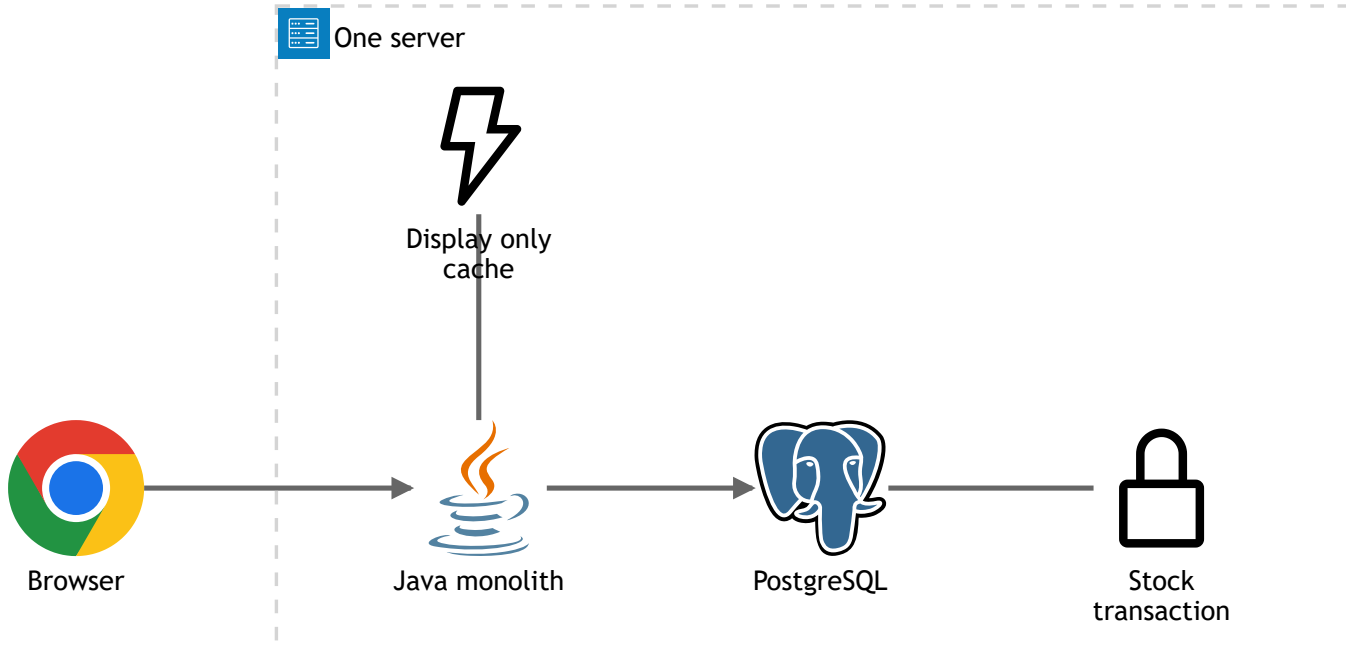
- **Scope:** the order-confirm route; product pages keep reading the cache
- **Change:** remove cache reads from checkout; approve only through the guarded stock update
- **Safety net:** `available ⩾ :qty` in the update and `CHECK (available ⩾ 0)` on the stock table
- **Escape route:** a flag restores the previous path, kept only while validation runs because it can oversell
- **Deferred:** write-through cache, until display staleness hurts conversion; a TTL never replaces the check

```
UPDATE inventory
   SET available = available - :qty
 WHERE sku = :sku AND available ⩾ :qty;   -- 0 rows updated ⇒ sold out
```

Verification: simultaneous buyers on a stale cache

Check	Pass condition
Stale-cache race	With the cache forced to report stock while the database holds 0, every purchase is answered sold out
Concurrent load	500 buyers for 10 items: exactly 10 confirmed orders, the rest sold out
Invariant	Oversold count 0 in every run; stock never negative in any sample
Cache independence	Identical outcome with the cache stale, empty or stopped
Display staleness	Age of the cached availability recorded; it may lag but never changes an outcome
Checkout latency	Checkout p95 and lock wait recorded against the baseline
Fitness function	The test runs in CI and fails on any oversold order; an architecture test fails when checkout code reads the stock cache

Solution architecture



Same components, one new guard: purchase reads stock only inside the database transaction, and the cache serves display only.

Glossary

Used in this step

Term	Definition
Authoritative transaction on purchase	Purchase decided by a database transaction on the source of truth
Database transaction	Atomic execution boundary for related data modifications
Write-through cache	Writes data to the cache and underlying storage together
TTL	How long a cached copy counts as fresh
Stale cache	Cached value older than the source of truth
Source of truth	The one store whose value wins when copies disagree
Cache-aside pattern	Application explicitly reads from and populates a cache
Cache invalidation	Removing or replacing cached copies after content changes
Invariant	Business rule that must hold at every moment
Overselling	Confirming more units than physically exist
Race condition	Outcome depends on the timing of concurrent requests

Related concepts

Term	Definition
Single writer	Only one component may modify a given piece of data
Synchronous replication	Acknowledge a commit only after required replicas persist it
Reservation pattern	Temporarily claim a resource until the workflow completes
Optimistic locking	Write succeeds only if the row version is unchanged since read
Eventual consistency	Replicas converge over time; reads may be stale

A08 · Single server and correctness

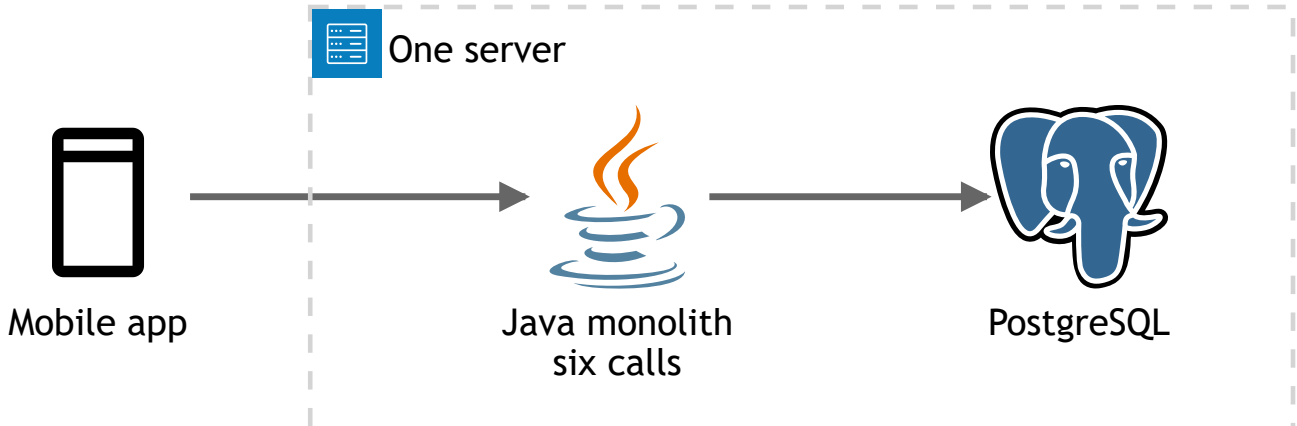
Reduce client round trips

A product page needs six REST calls, and mobile page latency violates the UX target.

A product page assembles itself from six REST calls

Component	What it is	What it does
Mobile client	Customer-facing app on a mobile network	Calls one REST endpoint per resource to build the product page
REST endpoints	Resource-oriented HTTP interface of the monolith	Return product, price, stock, reviews and other page parts separately
Java monolith	One deployable application on one server	Serves each call with its own query and its own response
PostgreSQL	Relational transactional database	Holds product, price, stock and review data
Mobile network	High-latency link between client and server	Charges a full round trip for every request

Current architecture



Three ways to fetch a page in fewer requests

Solution	What it is	What it does
Aggregated REST endpoint	One REST endpoint that returns everything a page needs	Composes product, price, stock and reviews on the server and answers in one round trip
BFF	API layer tailored to a specific frontend or client	Gives the mobile client its own API that shapes and combines data for its screens
GraphQL	Typed query interface allowing clients to request selected fields	Lets the client ask for exactly the fields of one page in a single request

Each option trades round trips for specialization, a layer or governance

Solution	Gains	Costs
Aggregated REST endpoint	One round trip per page Plain REST: existing tooling, caching and monitoring keep working	Specialized: one endpoint per page shape Payload drifts toward overfetching, or another endpoint appears when screens change
BFF	Payload and behavior tailored to one client The client team changes its API without touching the core	Extra layer to build, deploy and operate Composition logic is duplicated per client
GraphQL	The client picks fields, so one request returns no extra data One schema serves many page shapes	Query complexity: depth and cost need limits HTTP caching and resolver query counts need governance

Deciding signal: fan-out count per page, mobile network p95 and how many distinct client payloads must be served.

Smallest safe move

One aggregated read endpoint for the mobile product page

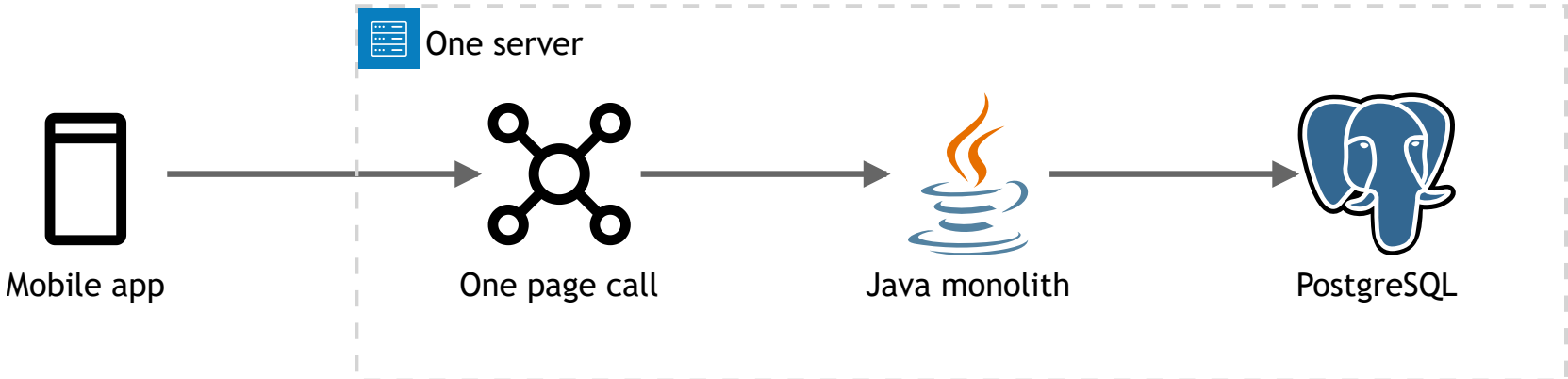
- **Scope:** one read-only endpoint for the mobile product page; the six endpoints and web clients stay
- **Change:** compose product, price, stock and reviews inside the monolith and return only the fields the page renders
- **Safety net:** a contract test compares the aggregate with the six responses; a failing part degrades gracefully; shown stock is display only
- **Escape route:** a remote flag sends the app back to the six calls
- **Deferred:** BFF, until clients need distinct payloads or release cycles; GraphQL, until page shapes multiply and query cost can be governed

```
GET /api/mobile/product-pages/42
200 OK
{ "product": {}, "price": {}, "availability": {}, "reviews": [] }
```

Verification: requests, bytes and p95 on a mobile profile

Check	Pass condition
Requests per page	1 instead of 6 for the mobile product page
Mobile page p95	Within the UX target on a throttled mobile network profile
Bytes per page	Not higher than the six calls combined; recorded before and after
Payload equality	The aggregate matches the six responses for a sample of products
Partial failure	A failing part, such as reviews, leaves the rest of the page usable
Database queries per page	Recorded; no increase over the baseline
Fitness function	A performance budget in CI fails when a page needs more than one request or misses the p95 target

Solution architecture



Same components, one new endpoint in the monolith: the client makes one call, and composition happens next to the data.

Glossary

Used in this step

Term	Definition
REST	Resource-oriented interface style using uniform HTTP semantics
Aggregated REST endpoint	One REST endpoint that composes several resources into one response
BFF	API layer tailored to a specific frontend or client
GraphQL	Typed query interface allowing clients to request selected fields
Round-trip time	Time for a request to reach a server and the reply to return
Request fan-out	Number of calls needed to build one page or response
Overfetching	Response carries more data than the client uses
Consumer-driven contract test	Consumer expectations verified against a provider
Graceful degradation	Retain core capability when optional dependencies fail
Performance budget	Agreed limit, such as requests or p95, that a build must not exceed

Related concepts

Term	Definition
API gateway	Central entry point for routing, authentication, rate limiting, and API policies
CQRS	Separates models for reading data and modifying state
Materialized view	Precomputed query result stored for efficient reads
Materialized query projection	Read-optimized representation built from updates or events
gRPC	RPC framework commonly using HTTP/2 and Protocol Buffers

A09 · Single server and correctness

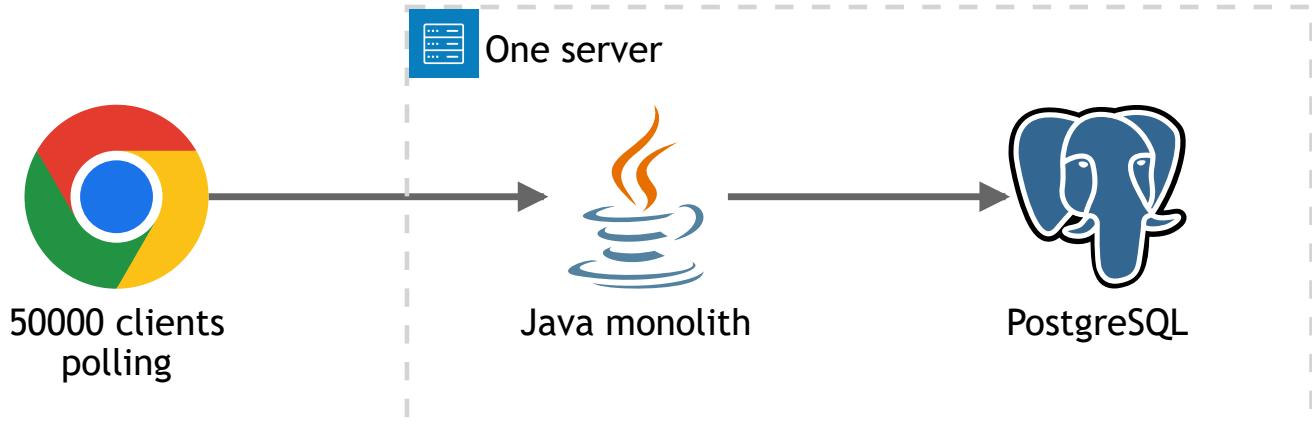
Switch order updates from polling

50,000 clients require notice of an order status change within 2 seconds.

Clients poll for order status

Component	What it is	What it does
Browser / SPA	Customer-facing web client	Repeats a status request for the order on a timer
Order status API	REST endpoint of the Java monolith	Answers every poll with the current status, changed or not
Polling	Repeated requests to determine whether data changed	Costs one request and one query per client per interval
Polling interval	Time between two requests from one client	Sets the worst-case delay before a change is noticed
PostgreSQL	Relational transactional database	Holds the orders and their status

Current architecture



Fast notice and low load pull apart

A shorter interval gives faster notice but multiplies requests, and almost every answer says nothing changed. With 50,000 clients, a 2 second interval already means 25,000 requests per second.

25,000 requests per second

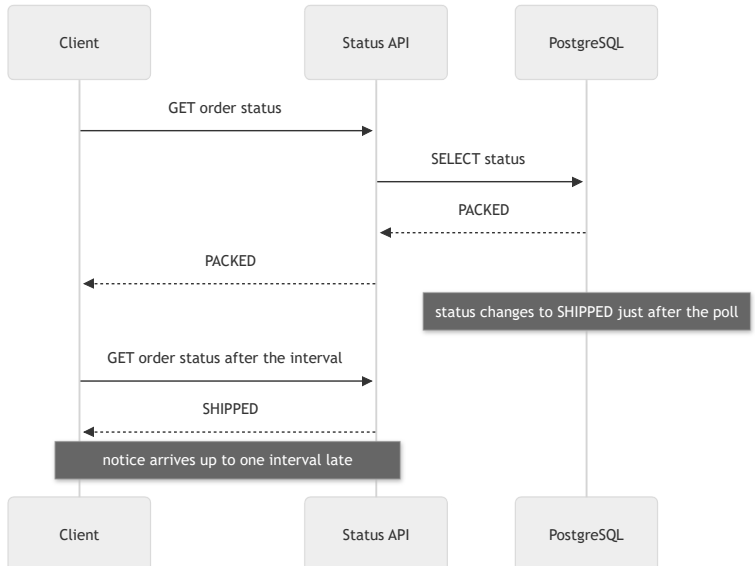
polling at a 2 s interval, derived: 50,000 clients divided by 2 s

50,000 clients

waiting for order updates at the same time

2 s

target: a status change reaches the client within 2 seconds



Three ways to learn about a status change

Solution	What it is	What it does
Adaptive polling	Polling whose interval shrinks when a change is likely and grows when it is not	Keeps the pull model and cuts requests while orders are idle
SSE	Server-to-client event stream over HTTP, short for Server-Sent Events	Keeps one connection open per client and pushes each status change
WebSocket	Persistent bidirectional client-server communication	Keeps one connection open per client for messages in both directions

Push removes empty requests and adds open connections

Solution	Gains	Costs
Adaptive polling	No new protocol or component; easy to troubleshoot Works through any proxy	Still pays requests that find nothing Notice is late by up to the current interval, so a 2 s bound needs a 2 s interval
SSE	The server pushes each change as it happens Plain HTTP; browsers reconnect automatically	50,000 long-lived connections to hold: memory and file descriptors One-way only; client messages still use normal requests
WebSocket	Messages in both directions on one connection Low latency per message	Connection lifecycle: upgrade, keep-alive, reconnect, authentication renewal Own message protocol and proxy support, more than one-way status needs

Deciding signal: subscriber count, update frequency per order, required delivery delay and whether the client must send messages back.

Smallest safe move

SSE for order status, polling kept as fallback

- **Scope:** order-status updates for customers viewing an order; other APIs unchanged
- **Change:** an event-stream endpoint in the monolith pushes each status change after its transaction commits
- **Safety net:** a connection starts with the current status; events carry an id for resume; heartbeats keep idle connections open; reconnects use jitter
- **Escape route:** a flag returns clients to polling; the status endpoint stays in place
- **Deferred:** WebSocket, until the client must send messages on the same channel; a message broker, until several instances must share events

```
GET /orders/42/events
Accept: text/event-stream

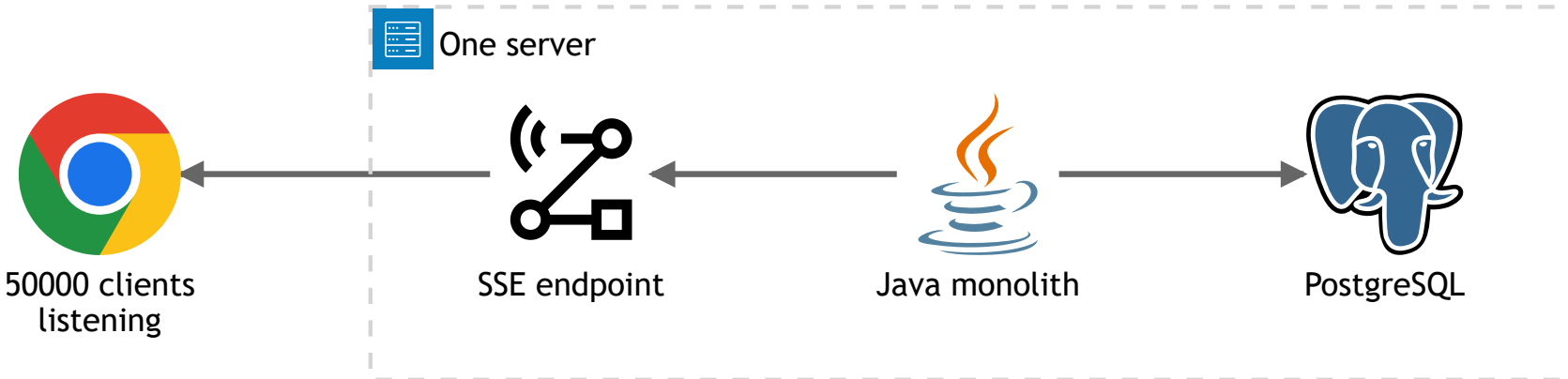
id: 7
data: {"orderId":42,"status":"SHIPPED"}
```

Verification: 50,000 connections and delivery delay

Check	Pass condition
Concurrent connections	50,000 open streams held for the whole test within memory and file-descriptor limits
Delivery delay	Status change to client notice within 2 s, reported as p99 and maximum
Poll traffic	Status requests per second fall to near zero for connected clients
Correctness	The last status each client holds equals the database status; none lost or out of order
Reconnect	After a server restart all clients reconnect with jitter and resync within the target
Fallback	With the stream blocked, clients fall back to polling and still see the status
Fitness function	A load test with simulated clients fails when delivery delay exceeds 2 s or connections drop

Rollback: the flag back to polling changes routing only; the stream holds no state, because the database stays the source of truth for status.

Solution architecture



Same components, one new endpoint: the monolith pushes status changes over open connections instead of answering polls.

Glossary

Used in this step

Term	Definition
Polling	Repeated requests to determine whether data changed
Adaptive polling	Polling whose interval changes with how likely a change is
Polling interval	Time between two requests from one client
SSE	Server-to-client event stream over HTTP
WebSocket	Persistent bidirectional client-server communication
Long-lived connection	Connection kept open for minutes or hours instead of one request
File descriptor	Operating system handle that every open connection consumes
Heartbeat	Periodic signal that shows an idle connection is still alive
Jitter	Randomized delay around retries
Delivery delay	Time from a status change to the client receiving it
p99	Latency that 99% of requests stay under

Related concepts

Term	Definition
Long polling	Keep an HTTP request open until new information or a timeout
Publish/subscribe	Distributes messages to multiple interested subscribers
Message broker	Routes and delivers messages between producers and consumers
Backpressure	Limit or slow producers when consumers cannot keep up
Load balancer	Distributes incoming requests across multiple servers or instances

A10 · Single server and correctness

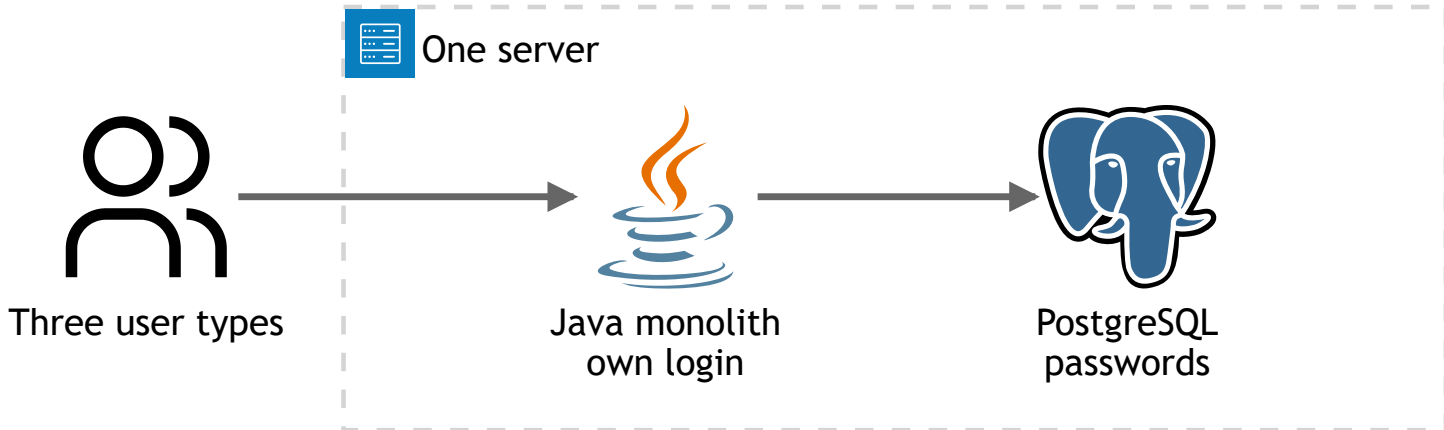
Consolidate identity

Customers, admins and employees need separate authorization and SSO.

Login and permissions live inside the application

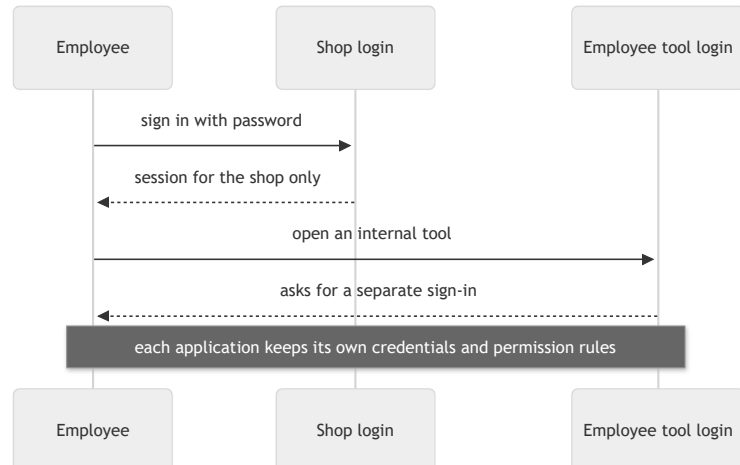
Component	What it is	What it does
Users	Customers, admins and employees	Sign in through the application’s own login
Login module	Application code that authenticates users	Checks a password against the local table and starts a session
Local password table	Credentials stored by the application in PostgreSQL	Holds password hashes for every kind of user
Authorization checks	Permission decisions made in application code	Allow or deny actions without separate rules per user type
Java monolith	One deployable application on one server	Owns authentication, sessions and every permission decision

Current architecture



One login cannot tell three audiences apart

Customers buy, admins manage the catalog and employees use internal tools. Each needs its own permissions, and employees expect to sign in once across applications. An application-specific login offers neither.



No separate authorization

rules are not split per user type, and there is no shared sign-in

3 user types

customers, admins and employees

100% of privileged actions checked

target: scoped authorization on every privileged API action, plus SSO

Three ways to separate who a user is from what the user may do

Solution	What it is	What it does
IdP + OIDC + RBAC	An identity provider, such as Keycloak or Okta, authenticates users through OpenID Connect; permissions come from predefined roles	Signs users in once for every application and grants access by role: customer, admin or employee
ABAC	Attribute-based access control: policies evaluate attributes of the user, the resource and the context	Decides each request from rules such as ownership, region or order amount, not from a fixed role list
Custom auth	Authentication and authorization built and operated inside the application	Keeps every flow under the team’s control, including passwords, sessions and recovery

Each option moves the burden: roles, policies or security

Solution	Gains	Costs
IdP + OIDC + RBAC	Standards-based sign-in and SSO across applications Credentials, MFA and recovery move to a specialized service	Role management: roles must be assigned, reviewed and kept few Central identity dependency: an identity provider outage blocks sign-in
ABAC	Fine-grained decisions from attributes Fewer roles than a role-per-case model	Policy complexity: rules are hard to write, test and audit Needs reliable attribute sources and a policy engine
Custom auth	Full control over flows and data No external dependency	Security burden: password storage, recovery and brute-force defense are owned by the team SSO and federation must be built and repeated for each application

Deciding signal: number of user types and distinct permission sets, rules that depend on resource attributes beyond role, SSO need, role explosion, and the risk of unauthorized access against user friction.

Smallest safe move

Identity provider with OIDC sign-in and three roles

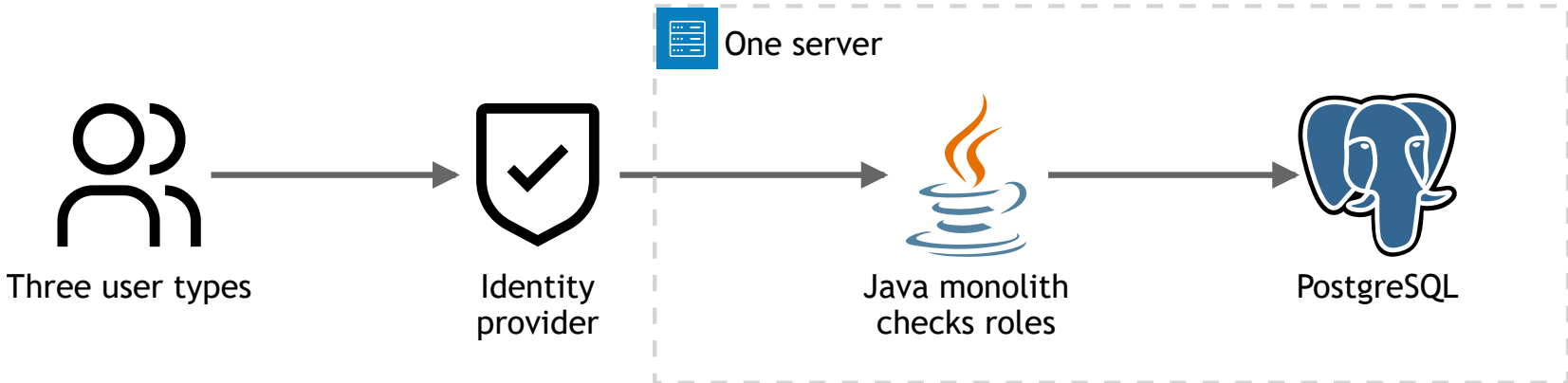
- **Scope:** sign-in and role checks for the three user types; business logic and data stay
- **Change:** redirect sign-in to an identity provider through OIDC; the monolith verifies the token and maps its role claim to route permissions
- **Safety net:** deny by default; verify signature, issuer, audience and expiry; keep an ownership check so customers reach only their own orders
- **Escape route:** a flag restores local login; the password table stays until the cutover is verified
- **Deferred:** ABAC, until rules depend on attributes or roles multiply; custom auth stays out

```
customer → /api/orders          own orders only
employee → /internal/**
admin    → /admin/**
```

Verification: SSO flows and denied privilege escalation

Check	Pass condition
SSO	One sign-in at the identity provider opens the shop and an employee tool without a second prompt
Role matrix	A routes-by-roles test matches the policy for all three roles: allowed where granted, denied elsewhere
Privilege escalation	A customer token on an admin route gets 403; a tampered role claim is rejected
Token validity	Expired, wrong-issuer, wrong-audience and unsigned tokens are rejected
Object-level access	Customer A cannot read customer B’s order
Login latency and failures	Recorded against the local login baseline; within the agreed budget
Fitness function	Authorization tests run in CI; the build fails when a privileged route has no role check

Solution architecture



Same application and database, one new node: sign-in is delegated to the identity provider, and the monolith only verifies tokens and checks roles.

Glossary

Used in this step

Term	Definition
Identity provider (IdP)	Authenticates users and issues identity credentials
OpenID Connect (OIDC)	Identity layer providing authentication on top of OAuth 2.0
SSO	Reuse sign-in for several applications
RBAC	Grants permissions through predefined roles
ABAC	Grants permissions by evaluating attributes of the user, the resource and the context
Custom auth	Authentication and authorization built and operated inside the application
Authentication	Proving who a user is
Authorization	Deciding what an authenticated user may do
Token	Signed credential issued by the identity provider; carries claims such as roles
Role explosion	Roles multiply as permissions need finer distinctions
Privilege escalation	Gaining permissions beyond those granted
Object-level authorization	Check that a user may access one specific record

Related concepts

Term	Definition
OAuth 2.0	Framework for delegated authorization
MFA	Require more than one authentication factor
Least privilege	Provide only necessary access for an actor or workload
Zero trust	Require verification for access rather than assuming network trust
API gateway	Central entry point for routing, authentication, rate limiting, and API policies
Audit log	Recorded business or security action with actor, time and outcome

Phase 2

Availability and deployment

Steps in this phase

Step	Forcing constraint
11 Survive application process loss	Maintenance causes 45 minutes downtime per month; 99.95% availability target
12 Remove instance-affine session state	Login sessions disappear when requests switch nodes
13 Survive availability-zone outage	Zone loss cannot stop checkout longer than 5 minutes
14 Improve transaction recovery point	RPO must be at most 5 minutes
15 Deploy without planned downtime	Weekly release outage exhausts error budget
16 Scale for 10x campaign spikes	20-minute promotion generates 10x load; idle baseline most month
17 Reduce abusive API traffic	Bots consume 40% of processing capacity
18 Contain slow payment dependency	Provider blocks 20 seconds, exhausting workers
19 Observe cross-service failures	Operators cannot locate fault source

A11 · Availability and deployment

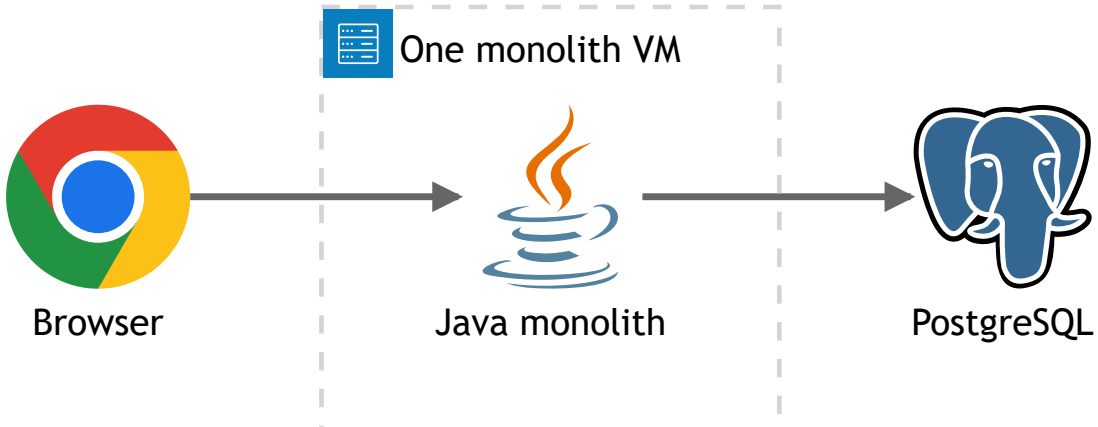
Survive application process loss

Maintenance causes 45 minutes of downtime per month against a 99.95% availability target.

One monolith VM serves every request

Component	What it is	What it does
Browser / SPA	Customer-facing web client	Sends every request to the one VM
Monolith VM	One virtual machine hosting the application	Is the only place where a request can be served
Java monolith	One deployable application, one process	Serves catalog, cart and checkout; a crash or restart ends every in-flight request
Maintenance window	Planned stop for patches, restarts and deploys	Takes the whole shop offline; accounts for 45 minutes of downtime per month
PostgreSQL	Relational transactional database	Stores orders and stock; reached over the network from the VM

Current architecture



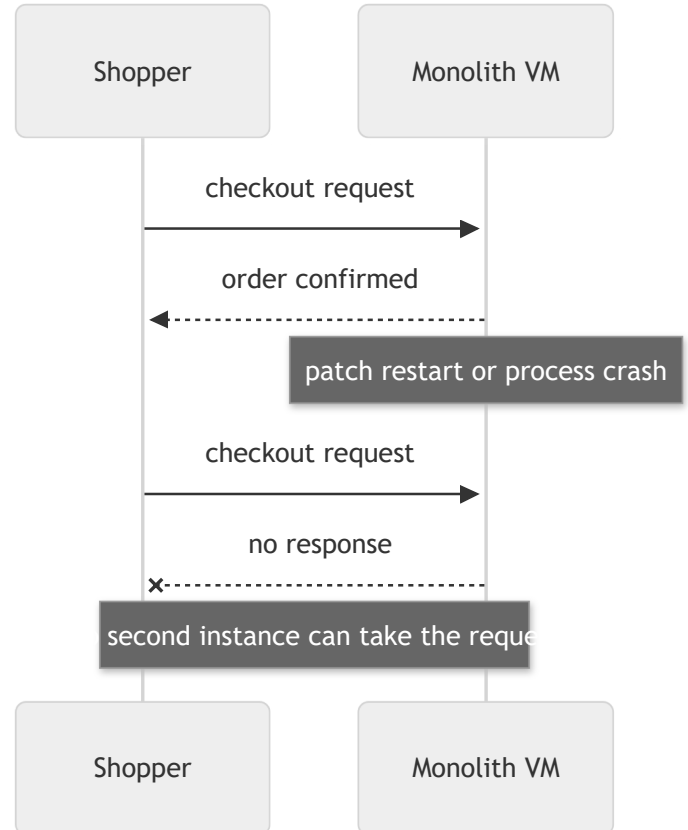
Every restart is an outage

Patches, restarts and crashes stop the only application process, so every shopper gets errors until it returns. In downtime terms, 99.95% over 30 days allows 21.6 minutes (43,200 minutes × 0.05%); 45 minutes is about 99.90%.

45 min per month
measured downtime from maintenance

1 VM, 1 process
every restart or failure is a full outage

99.95%
target availability, about 21.6 minutes of downtime per 30 days



Three ways to stop one instance from taking the shop down

Solution	What it is	What it does
Load Balancer + replicas	Several identical application instances behind a load balancer	Spreads requests across instances and routes around a failed or restarting one
Active-Passive	One deployment processes traffic while another waits	Promotes the waiting standby when the active deployment fails
Larger VM	A bigger single virtual machine	Adds CPU and memory to the same one instance

Surviving instance loss costs duplicate capacity or switchover time

Solution	Gains	Costs
Load Balancer + replicas	Survives an instance fault without a switchover step Instances can be restarted one at a time	Duplicate cost: every replica needs its own compute A new component to run; instance-local state breaks when requests move
Active-Passive	Simpler authority: one deployment serves at a time Easy to reason about during an incident	Standby expense: capacity that serves nothing until needed Switchover delay: detection and promotion before traffic resumes
Larger VM	Minimal architecture change More capacity for the one instance	Same single point of failure: a crash or restart is still a full outage A larger VM costs more and has a size ceiling

Smallest safe move

Two instances behind a load balancer with health checks

- **Scope:** the application tier; both instances run the same build against the same PostgreSQL
- **Change:** put a load balancer in front of a second identical instance and drop an instance from rotation when its health check fails
- **Safety net:** the second instance takes traffic only after it passes its health check, and one instance alone can carry peak load
- **Escape route:** send all traffic back to the original VM, which stays untouched during validation
- **Deferred:** active-passive, until a measured switchover time fits the budget; a larger VM keeps the single point of failure

```
# HAProxy example
backend shop
    option httpchk GET /health/ready
    server a app-a:8080 check inter 5s fall 3 rise 2
    server b app-b:8080 check inter 5s fall 3 rise 2
```

Verification: terminate one instance at peak

Check	Pass condition
Instance kill at peak	One instance is terminated under peak load; the successful request ratio stays at or above 99.95%
Detection	The balancer drops the failed instance within the health-check window, for example about 15 s with 3 probes at 5 s
Capacity	The surviving instance carries peak traffic with p95 inside the SLO
Recovery	A restarted instance passes its health check and rejoins without manual action
Maintenance	Restarting instances one at a time adds 0 minutes of downtime
Fitness function	A recurring kill test under load fails the pipeline when the success ratio drops below the target

Rollback: sending traffic back to the single VM changes no data, because both instances share one database; users whose in-memory sessions sat on a removed instance sign in again.

Solution architecture

Same browser, application and database, plus a load balancer and a second instance: losing one instance no longer stops requests. PostgreSQL is still one instance.

Glossary

Used in this step

Term	Definition
Availability	Share of valid requests that succeed during a rolling window, such as 30 days
SLO	Target for an SLI over a defined time window
Error budget	Maximum permitted unreliability under an SLO
Single point of failure	Component whose failure stops the whole service
Monolith	Entire application deployed as a single unit
Virtual machine	Isolated operating system running on virtualized hardware
Load balancer	Distributes incoming requests across multiple servers or instances
Replica	Identical running copy of the application behind a load balancer
Health check	Detects whether an application instance is functioning correctly
Active-Passive	One deployment processes traffic while another waits
Failover	Switches operations to standby resources after failure
Vertical scaling	Add CPU or memory to one host

Related concepts

Term	Definition
Active-Active	More than one deployment actively processes traffic
Horizontal scaling	Distribute workload across more instances
Readiness probe	Signals whether an instance may receive new requests
Automated failover	Switch service or data authority on detected failure through orchestration
Multi-AZ deployment	Run workload across separate availability zones
Chaos engineering	Controlled experiments under injected failures

A12 · Availability and deployment

Remove instance-affine session state

Login sessions disappear when requests switch nodes; the target is 0 forced re-logins.

Login state lives in the memory of one instance

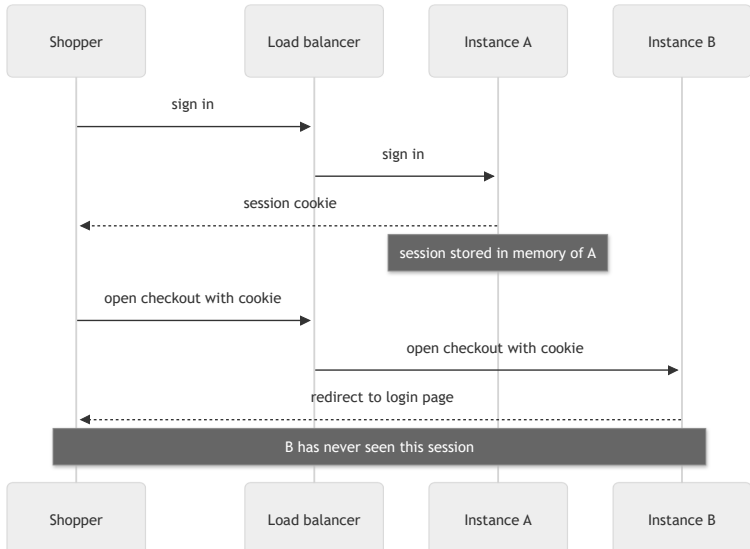
Component	What it is	What it does
Browser / SPA	Customer-facing web client	Sends a session cookie with every request after sign-in
Load balancer	Entry point in front of two identical instances	Routes each request to either instance, without knowing who holds the session
Monolith instance A	One of two identical Java processes	Handles the sign-in and keeps the session in its own memory
Monolith instance B	The second identical Java process	Has no record of a session created on instance A
HTTP session	Server-side record of a signed-in user, found by a session id	Exists only in the memory of the instance that created it
Identity provider	Authenticates users through OpenID Connect	Confirms the sign-in; the application then starts its own session

Current architecture

Each instance keeps its login sessions in its own memory, so a request routed to the other instance finds no session.

The next request meets an instance without the session

A shopper signs in on instance A. The load balancer sends the next request to instance B, which has no session, so the shopper lands on the login page again. A restart of instance A ends every session it held.



Session lost on node switch

symptom: a signed-in request is redirected to the login page

2 instances, even routing

about half of the requests after sign-in reach the instance without the session

0 forced re-logins

target: a signed-in user survives the loss of any one instance

Three ways to let any instance recognize a signed-in user

Solution	What it is	What it does
Stateless tokens	Signed, short-lived credential carried by the client, such as a JWT	Lets each instance verify the signature and read the user from the claims, with no stored session
Shared session store	External store, such as Redis, holding the session data for all instances	Lets every instance read and write the same session, so any node can serve any request
Sticky sessions	Load balancer affinity that pins a client to one instance	Sends a client’s requests to the instance that holds its session

Each option pays in token lifecycle, a dependency or uneven routing

Solution	Gains	Costs
Stateless tokens	Independent replicas: any instance verifies any request No lookup per request and no new runtime component	Token lifecycle: expiry, refresh and revocation must be designed Revocation before expiry needs extra state; claims travel with every request
Shared session store	Compatible with existing server-side session code Revocation is immediate: delete the entry	New dependency: a store outage logs users out or blocks requests Extra network hop per request; the store needs its own redundancy
Sticky sessions	Minimal change: one balancer setting No application change and no new component	Uneven routing: load follows the pinned clients, not the capacity The session is still lost when its instance fails or restarts

Smallest safe move

Signed short-lived tokens from the existing identity provider

- **Scope:** authentication state only; the identity provider, roles and business data stay as they are
- **Change:** each instance verifies the signed token on every request and reads user and roles from its claims
- **Safety net:** the application accepts the old session cookie and the token until the instance-kill test passes
- **Escape route:** a flag restores cookie sessions; balancer affinity can bridge the gap, and no data needs repair
- **Deferred:** shared session store, until revocation must be immediate or sessions hold server-side data; a larger single server stays out, as it keeps the single point of failure

```
accept the token only if
  the signature matches a key published by the issuer
  iss is the expected issuer, aud is this shop, exp is in the future
then read sub and roles from the claims
```

Verification: kill the instance that served the login

Check	Pass condition
Kill session-host instance	Sign in, terminate the instance that served the login, repeat the request: success without a new sign-in
Even routing	For example 100 consecutive requests alternate between both instances: all authorized, 0 redirects to login
Expiry and refresh	An expired token is rejected, and a refresh issues a new one without a new sign-in
Revocation delay	A user disabled at the identity provider loses access within the token lifetime, for example 15 minutes
Tampering	A token with a changed payload, wrong issuer or wrong audience is rejected
Latency	p95 stays at the previous level, because verification makes no network call per request
Fitness function	A recurring instance-kill test with a signed-in user fails the pipeline on any forced re-login

Solution architecture

Same four nodes; the identity moves out of instance memory into a signed token that the browser holds and either instance verifies.

Glossary

Used in this step

Term	Definition
Instance-affine state	Data that only the instance holding it can use; requests routed elsewhere do not find it
Session	Server-side record of a signed-in user, found through an identifier the browser sends
Stateless tokens	Signed credential, such as a JWT, that carries identity and expiry so no server stores a session
Shared session store	External store that every instance reads and writes for session data
Sticky sessions	Load balancer affinity that sends a client to the same instance every time
Token claims	Statements inside a token, such as subject, roles, issuer, audience and expiry
Token lifecycle	Issue, expiry, refresh and revocation of a token
Load balancer	Distributes incoming requests across multiple servers or instances
Identity provider (IdP)	Authenticates users and issues identity credentials
OpenID Connect (OIDC)	Identity layer providing authentication on top of OAuth 2.0

Related concepts

Term	Definition
Monolith	Entire application deployed as a single unit
Horizontal scaling	Distribute workload across more instances
Key-value store	Retrieves values using unique keys with minimal query overhead
Distributed cache	Shared cache accessible by multiple application instances
OAuth 2.0	Framework for delegated authorization
SSO	Reuse sign-in for several applications

A13 · Availability and deployment

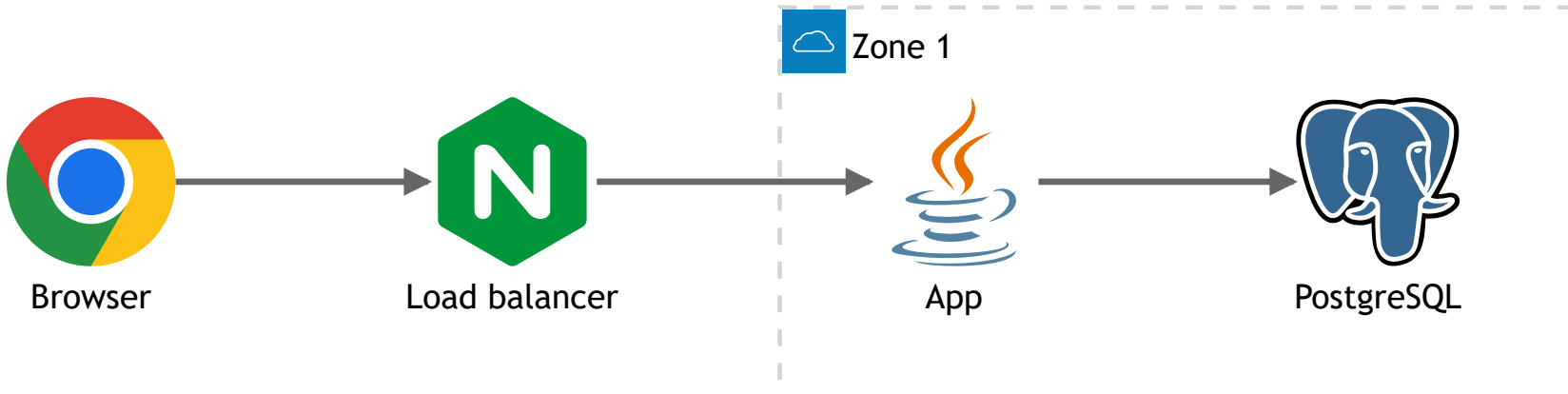
Survive availability-zone outage

Zone loss cannot stop checkout longer than 5 minutes.

Application and database share one availability zone

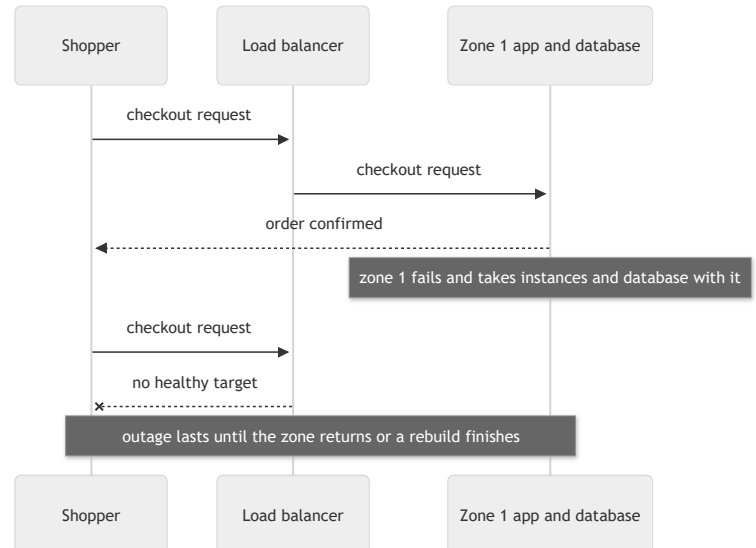
Component	What it is	What it does
Browser / SPA	Customer-facing web client	Sends every request to the load balancer
Load balancer	Managed entry point that spans zones	Spreads requests across the application instances
Application instances	Two identical stateless Java monolith instances	Serve catalog, cart and checkout from Zone 1
PostgreSQL	One relational transactional database	Stores orders and stock, in the same zone as the instances
Availability zone	Isolated infrastructure location within a cloud region	Is the shared fault domain: power, cooling and network are separate from other zones

Current architecture



One zone fault removes everything

A power, cooling or network fault in the zone stops both instances and the database at once. Instance redundancy does not help, because all instances share the fault, and checkout stays down until the zone returns.



Zone loss is a full outage

instances and database share one fault domain

1 zone, 2 instances, 1 database

no healthy target remains for the load balancer

5 minutes

target: the longest checkout stoppage when a zone is lost

Three ways to keep checkout running when a zone fails

Solution	What it is	What it does
Multi-AZ	Instances and a database standby placed in separate availability zones of one region	Keeps serving from the surviving zone and promotes the standby database when the primary zone fails
Active-Active	More than one deployment actively processes traffic	Serves requests from several zones at once, so no zone has to take over
Single-AZ backups	All workload in one zone, with periodic backups stored outside it	Rebuilds the platform in another zone from the backups after a zone loss

Zone survival is paid with duplicate capacity, conflict handling or recovery time

Solution	Gains	Costs
Multi-AZ	Survives a zone outage One primary accepts writes, so authority stays simple	Network and infrastructure cost: duplicate capacity and cross-zone traffic Failover takes detection and promotion time; false failover and split brain are risks
Active-Active	Rapid continuation: no takeover step for the zone that survives Both zones carry production load	Shared-state and conflict complexity: two writers can both sell the last item Needs conflict resolution or data ownership per zone
Single-AZ backups	Cheapest: no standing capacity in a second zone Also recovers from deletion and corruption	Slow: provisioning and restore time grow with data size and exceed the target Writes since the last backup are lost

Smallest safe move

A second zone with a promotable standby database

- **Scope:** instances, PostgreSQL and the entry point of checkout; the deployment stays in one region
- **Change:** run the same instance build in a second zone, add a standby database replicated from the primary, enable automated failover
- **Safety net:** each zone alone carries peak load; promotion fences the old primary, and a witness outside both zones breaks ties
- **Escape route:** drain zone 2 from the load balancer and disable promotion; the Zone 1 stack is untouched
- **Deferred:** active-active until the drill misses 5 minutes; backups stay for deletion and corruption, not for zone loss

```
failover policy, example values
  primary unreachable for 30 s and witness agrees → promote the standby in zone 2
  fence the old primary before the new primary accepts writes
```

Verification: disable one zone under peak load

Check	Pass condition
Zone drill	With zone 1 disabled in a recovery drill at peak, checkout resumes within 5 minutes
Application tier	Health checks drop zone 1 targets and zone 2 serves traffic, for example within 1 minute
Database failover	The standby is promoted and accepts writes; measured promotion time is recorded against the budget
Capacity	Zone 2 alone carries peak traffic with p95 inside the SLO
Order integrity	No duplicated order and no negative stock after promotion; acknowledged orders missing from the standby are counted, not assumed zero
Fencing	A returning old primary accepts no writes until it is rebuilt as a standby
Fitness function	A scheduled zone-isolation drill fails the pipeline when checkout takes longer than 5 minutes to resume

Solution architecture

Same browser, balancer and Zone 1 stack, plus Zone 2 with a second instance and a standby database replicated from the primary.

Glossary

Used in this step

Term	Definition
Availability zone	Isolated infrastructure location within a cloud region
Fault domain	Set of components that fail together when one shared dependency fails
Multi-AZ deployment	Run workload across separate availability zones
Active-Active	More than one deployment actively processes traffic
Active-Passive	One deployment processes traffic while another waits
Backup and restore	Recovery process using historical snapshots and logs
RTO	Maximum targeted recovery duration after disruption
Standby database	Copy of the primary database kept ready to take over writes after a failure
Automated failover	Switch service or data authority on detected failure through orchestration
Split brain	Two nodes both act as primary and accept conflicting writes
Write fencing	Prevent stale writers from modifying state after ownership changes

Related concepts

Term	Definition
Load balancer	Distributes incoming requests across multiple servers or instances
Health check	Detects whether an application instance is functioning correctly
Cloud region	Geographic area containing multiple cloud infrastructure facilities
Synchronous replication	Acknowledge a commit only after required replicas persist it
Asynchronous replication	Acknowledge commit before all replicas confirm persistence
RPO	Maximum targeted period of data loss

A14 · Availability and deployment

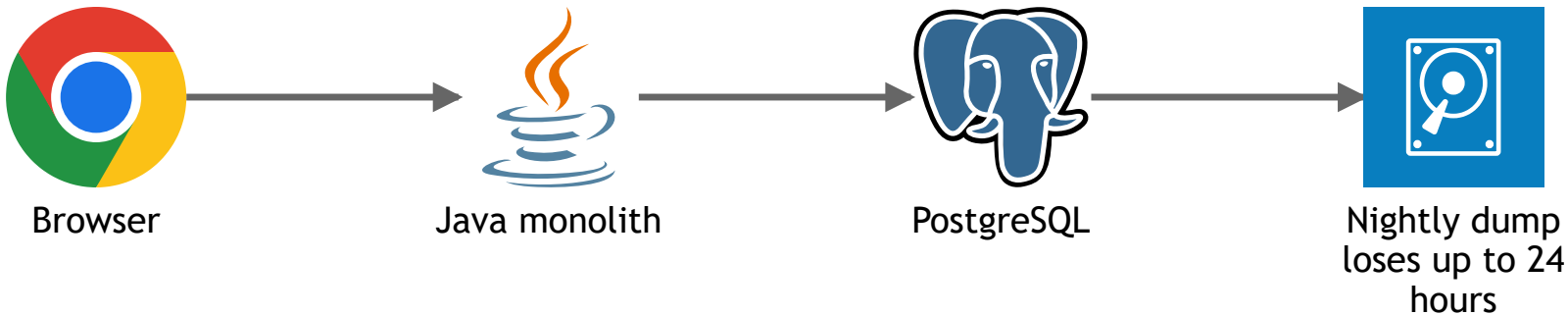
Improve transaction recovery point

RPO must be at most 5 minutes, while nightly SQL backups can lose up to 24 hours.

Recovery depends on one nightly SQL dump

Component	What it is	What it does
PostgreSQL	Relational transactional database	Holds orders and payments; acknowledges a commit once the change is in its own write-ahead log
Write-ahead log	Sequential log of changes written before the data pages are modified	Serves crash recovery, then is recycled; no copy is kept elsewhere
Nightly SQL dump	Logical backup file produced once a day	Captures the data as of the moment the dump ran
Backup storage	Disk or bucket holding the dump files	Provides the file that every restore starts from
Restore procedure	Replays the newest dump into a new database	Returns the data as of the last dump; everything committed after it is gone

Current architecture



A failure loses a day of orders

The only recovery source is last night's dump. If the database is lost or damaged in the afternoon, every order and payment since the dump is gone, and the data cannot be rewound to the moment before a faulty change.

Up to 24 hours lost

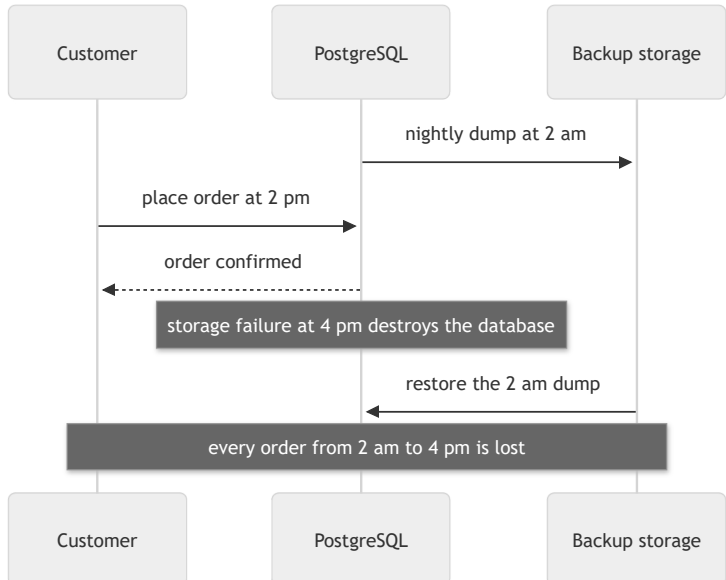
worst-case loss of committed transactions with a nightly dump

1 dump per day

a restore always returns to dump time, never to the moment of failure

RPO of 5 minutes or less

target: the maximum acceptable loss of committed transactions



Three ways to shrink the window of lost transactions

Solution	What it is	What it does
WAL + Point-in-Time Recovery	A base backup plus a continuous archive of the write-ahead log	Restores the database to a chosen moment by replaying archived log records on top of the base backup
Async replication	A standby that receives the log stream without delaying commits	Keeps a near-current copy; commits are acknowledged before the standby confirms them
Sync replication	A standby that must confirm persistence of each commit	Acknowledges a commit only after the required replica has persisted it

Each option trades recovery time, possible loss or write latency

Solution	Gains	Costs
WAL + Point-in-Time Recovery	Fine-grained restore to any moment inside the archive Also undoes a bad change, and commits are not slowed	RTO: base restore and log replay take time and grow with data volume Log retention storage; archive failures must be monitored
Async replication	Low lag: a near-current copy exists Takeover is quick and commits are not slowed	Possible data loss: commits the standby has not received are lost on failover Lag can grow under load, and a bad change replicates at once
Sync replication	No acknowledged-loss for the covered failure, such as loss of the primary node The standby is current at takeover	Higher latency: every commit waits for a round trip and a remote flush Writes can stall when the standby is unreachable, so availability drops

Smallest safe move

Continuous WAL archiving beside the nightly dump

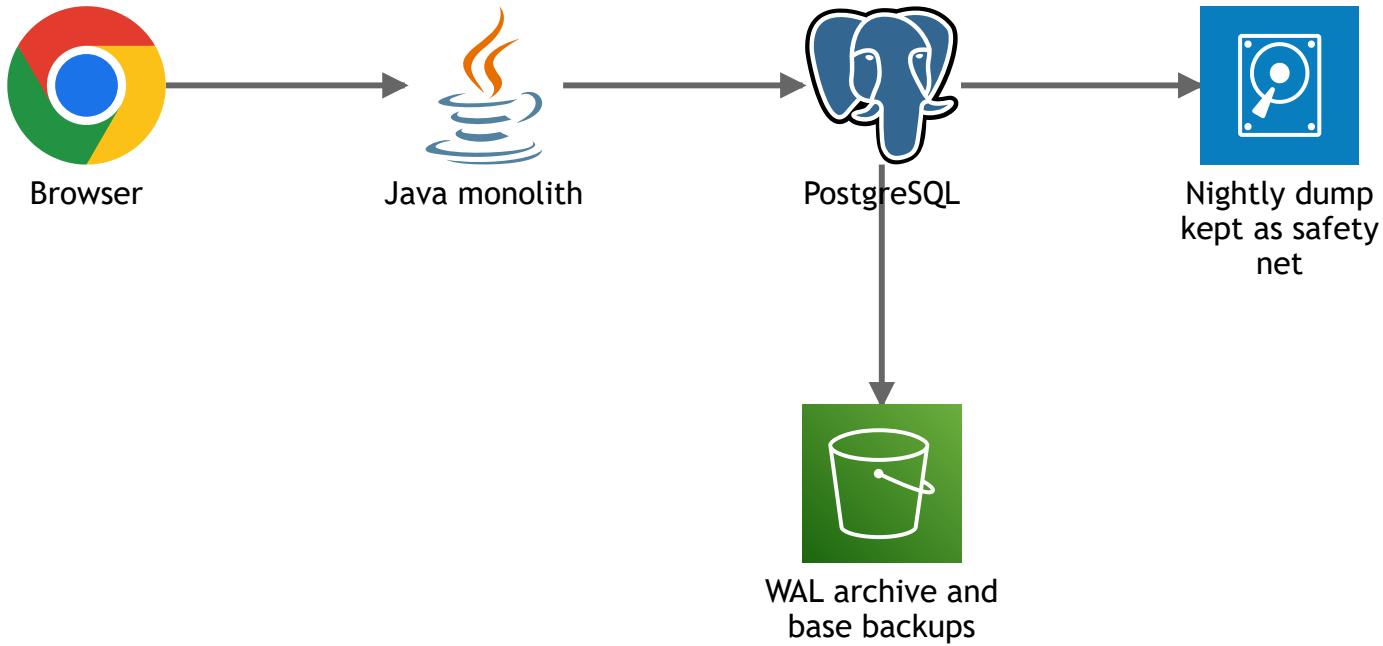
- **Scope:** the one PostgreSQL primary; application, schema and request path stay unchanged
- **Change:** take a physical base backup and archive every WAL segment outside the database host and zone, forcing a switch every 60 seconds, for example
- **Safety net:** nightly dumps keep running; a restore into a scratch instance proves each backup; archive failures alert before the disk fills
- **Escape route:** stop archiving and continue with the nightly dumps; no application data is touched
- **Deferred:** replication until restore time exceeds the RTO budget; synchronous replication until RPO must be zero, because stretching it across regions can sacrifice write availability

```
wal_level = replica
archive_mode = on
archive_timeout = 60
archive_command = 'pgbackrest --stanza=shop archive-push %p'
```

Verification: destroy the primary, then restore

Check	Pass condition
Measured RPO	After the primary is destroyed under write load, the newest transaction missing from the restored database is at most 5 minutes old
Measured RTO	Base restore and log replay finish within the agreed recovery time; the duration is recorded, not assumed
Point-in-time accuracy	A restore to the second before an injected bad bulk update returns the data as it was before it
Business invariants	The restored database has no negative stock and every paid order has its payment
Archive health	An alert fires when the archive fails or its newest segment is older than the budget, for example 2 minutes
Write latency	Commit p95 stays at baseline, because archiving runs outside the commit path

Solution architecture



Same application and database, plus a continuous WAL archive and base backups outside the database host; the nightly dump stays as a safety net.

Glossary

Used in this step

Term	Definition
RPO	Maximum targeted period of data loss
RTO	Maximum targeted recovery duration after disruption
SQL dump	Logical backup that stores the data as of the moment it ran
WAL	Append transaction log records before database pages are modified
WAL archive	Continuous copy of completed log segments to storage outside the database host
Base backup	Physical copy of the database files from which log replay starts
Point-in-Time Recovery	Restore a database to a selected moment using backups and logs
Asynchronous replication	Acknowledge commit before all replicas confirm persistence
Synchronous replication	Acknowledge a commit only after required replicas persist it
Replication lag	Delay between a commit on the primary and its arrival on the replica

Related concepts

Term	Definition
Backup and restore	Recovery process using historical snapshots and logs
Database transactions	Atomic execution boundary for related data modifications
Failover	Switches operations to standby resources after failure
Read replica	Database copy serving read operations to distribute query load
Multi-region replication	Maintain data copies across geographically separate cloud regions

A15 · Availability and deployment

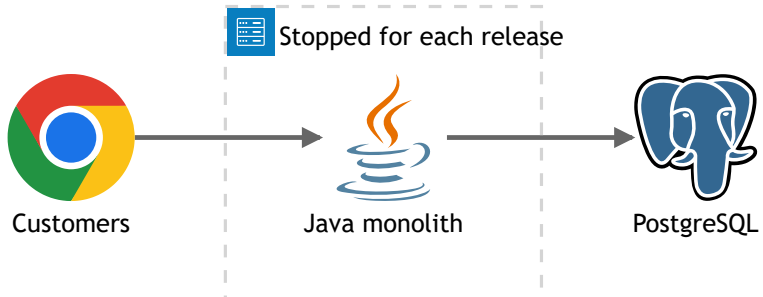
Deploy without planned downtime

The weekly release stops the application, and that outage exhausts the error budget.

A release stops the application, replaces it and starts it again

Component	What it is	What it does
Customers	Shoppers using the web client	Get errors or a maintenance page while the application is down
Java monolith	One deployable application	Serves every request; unavailable from stop to ready after the start
Release procedure	The weekly deployment run	Stops the process, replaces the artifact and starts the new version
PostgreSQL	Relational transactional database	Holds the data; schema changes are applied together with the release
Error budget	Maximum permitted unreliability under an SLO	Shrinks with every minute of release downtime

Current architecture



A release stops the single monolith, replaces it and starts it again, so every release is an outage.

Every release is an outage

The release stops the monolith for the whole deployment window, so browsing and checkout fail. Weekly releases keep spending the error budget on planned work.

Weekly outage

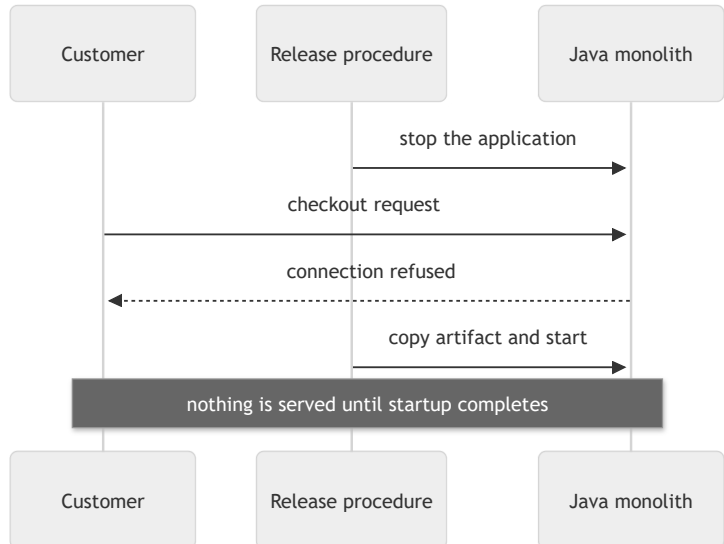
every release stops the application

99.95% SLO

about 21.6 minutes of error budget per 30 days (computed)

0 failed requests

target: none caused by a release



Three ways to replace a running version

Solution	What it is	What it does
CI/CD + rolling	An automated pipeline that replaces instances incrementally behind a load balancer	Drains one instance, updates it, waits for its health check, then moves to the next
Blue-Green	Two release environments, one live and one idle	Deploys to the idle one, verifies it, then switches all traffic to it
Canary	A small controlled segment of live traffic sent to the new version	Watches metrics on that segment, then widens it or aborts

Safer releases cost capacity, compatibility or monitoring

Solution	Gains	Costs
CI/CD + rolling	Minimal downtime Automated and repeatable; no second environment	Version compatibility: old and new code serve traffic together Capacity dips while an instance is out; rollback is another rollout
Blue-Green	Quick switch to the new version Switching back is the rollback	Cost: double temporary capacity Both environments share one database, so schema compatibility still applies
Canary	Gradual exposure: a bad release reaches few users Real traffic judges the new version	Needs good metrics and traffic routing to judge it Two versions run for the whole observation window

Deciding signal: need for exposure control, environment cost, rollback timing and schema compatibility.

Smallest safe move

Rolling release with expand/contract schema changes

- **Scope:** the release procedure only; code, routes and the database stay as they are
- **Change:** run two or more instances behind the load balancer; drain one, update it, wait for a passing health check, take the next
- **Safety net:** schema changes ship as expand then contract, so old and new code both fit the schema
- **Escape route:** halt the rollout and redeploy the previous artifact; untouched instances keep serving
- **Deferred:** blue-green until rollback must be one switch and spare capacity exists; canary until trusted metrics can judge a small traffic share

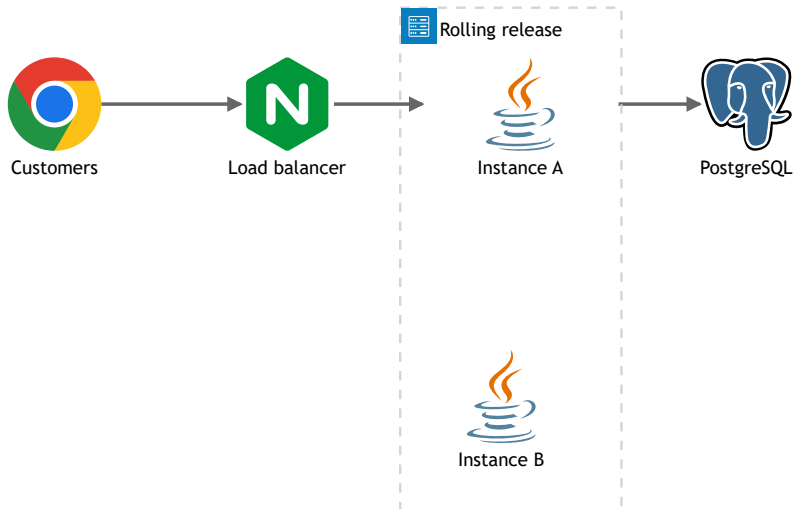
```
-- release N, expand: old and new code both run against this schema
ALTER TABLE customer_order ADD COLUMN shipping_note text;
-- release N+1, contract: runs only after no instance reads the old column
ALTER TABLE customer_order DROP COLUMN note;
```

Verification: release under live traffic, then revert

Check	Pass condition
Release under live traffic	Continuous browse and checkout requests through the full rollout: 0 failed requests caused by the release
Capacity	At least one healthy instance serves at every moment; p95 stays within SLO during the rollout
Version compatibility	Old and new versions pass the same request suite against the expanded schema
Revert	Redeploying the previous artifact mid-rollout and after completion restores service with 0 failed requests
Error budget	Release-caused unavailability recorded per release; releases no longer draw on the budget
Fitness function	Pipeline rejects a migration that drops or renames a column the running version still uses

Rollback: redeploying the previous artifact is a routing and configuration change with no data involved. A contract step that already dropped a column is state: a redeploy cannot restore it, so contract runs only after the new version has proven itself,

Solution architecture



Same application and database; a load balancer and a second instance let a release replace one instance while the other keeps serving.

Glossary

Used in this step

Term	Definition
Error budget	Maximum permitted unreliability under an SLO
SLO	Target for an SLI over a defined time window
CI/CD pipeline	Automation that builds, tests and deploys every change
Rolling deployment	Replace application instances incrementally
Blue-Green deployment	Maintain two release environments and switch traffic
Canary routing	Shift a small controlled segment of live traffic to a new implementation
Load balancer	Distributes incoming requests across multiple servers or instances
Health check	Detects whether an application instance is functioning correctly
Connection draining	Lets an instance finish in-flight requests before it leaves rotation
Expand-Contract	Add compatible representation, migrate consumers, then remove old representation
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Feature flag	Runtime control enabling behavior without deploying new code
Shadow traffic	Send production requests to a candidate implementation without using candidate responses
Active-Passive	One deployment processes traffic while another waits
Consumer-driven contract test	Consumer expectations verified against a provider
Schema compatibility view	Database view presenting old shape over new data

A16 · Availability and deployment

Scale for 10x campaign spikes

A 20-minute promotion generates 10x load, while the baseline stays idle for most of the month.

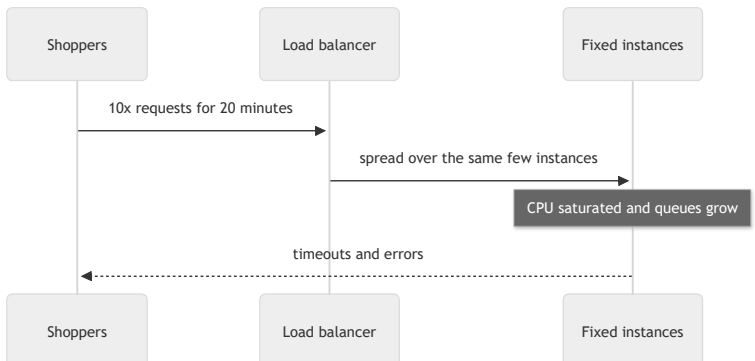
A fixed fleet is sized once for every day of the month

Component	What it is	What it does
Browser / SPA	Customer-facing web client	Sends browse and checkout requests, far more of them during a promotion
Load balancer	Distributes incoming requests across multiple servers or instances	Spreads requests over the fixed set of instances
Java monolith instances	A fixed number of identical, stateless application instances	Each serves requests up to its capacity; excess load queues or fails
PostgreSQL	Relational transactional database	Serves every instance; connections total instances × pool size, capped by `max_connections`
Promotion	A campaign with a start time chosen by the business	Drives 10x load for 20 minutes; the other days stay near baseline

Current architecture

Twenty minutes decide the sizing

A 20-minute promotion drives ten times the usual load. The fixed fleet saturates, latency climbs and requests fail. Sizing it for the peak would leave most of it idle the rest of the month.



10x load

for 20 minutes during the promotion

Idle baseline

most of the month, so peak sizing wastes capacity

p95 and errors within SLO

target: from the first minute of the spike

Four ways to add capacity for a spike

Solution	What it is	What it does
Horizontal Autoscaling	A group of identical instances whose count follows a load metric or a schedule	Adds instances when load rises and removes them when it falls; the load balancer spreads requests over the current set
Vertical Scaling	A bigger instance with more CPU and memory	Raises the capacity of each instance without changing the architecture
FaaS	Short-lived functions run by the platform for each request or event	Starts as many function instances as requests need and bills per call
Reserved Capacity	Committed use of resources or spend for discounted pricing	Keeps instances running at a lower unit price for a committed term

Every way to absorb a spike pays in lag, ceiling, rewrite or commitment

Solution	Gains	Costs
Horizontal Autoscaling	Elastic: capacity is paid for only while the spike lasts No single-machine ceiling	Startup lag: new instances need time to boot, warm up and pass health checks Needs stateless instances; the database can become the next limit
Vertical Scaling	Simple: minimal architecture change No new moving parts	Capped by the largest machine Paid for all month unless resized around each campaign
FaaS	Pay per call; no idle cost Scales out without a capacity plan	Cold starts add latency to the first calls Checkout code must be rewritten as functions; concurrency quotas apply
Reserved Capacity	Cheap, predictable baseline price Capacity is already running when the spike starts	Long-term commitment Adds no elasticity: sizing for the peak pays for idle machines

Smallest safe move

Scheduled and metric-driven scale-out of the same instances

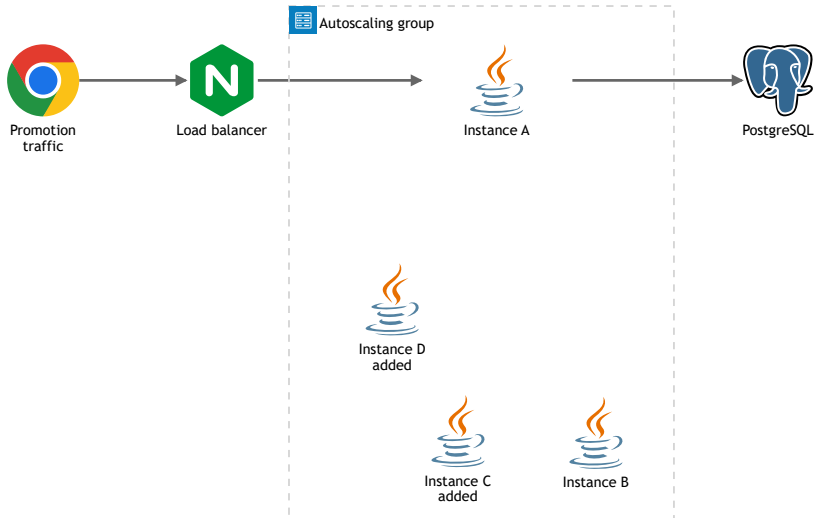
- **Scope:** the stateless monolith instances; code, routing and database stay as they are
- **Change:** autoscaling group with a baseline minimum, a scheduled scale-out before the promotion and a load-metric rule
- **Safety net:** maximum size keeps instances × pool size below `max_connections`; health checks keep unready instances out
- **Escape route:** restore the old fixed size; configuration only
- **Deferred:** message queue if backlog drain breaks latency; reserved capacity for the baseline; vertical scaling and FaaS

```
# illustrative values
minInstances: 2
maxInstances: 12            # bounded by the database connection budget
scaleOutAt: promotion start minus 10 minutes
targetCpuPercent: 60
```

Verification: replay the 10x load profile

Check	Pass condition
Autoscale lag	Time from load rise to new instances in rotation is charted against p95; p95 stays within SLO throughout
Success rate	Error rate stays within the SLO from the first minute of the 10x profile
Database	Connections stay below `max_connections` minus headroom; the database does not become the new limit
Scale-in	After the spike instances drain first; 0 in-flight requests fail
Cost	Instance-hours for the month are lower than for a fleet fixed at peak size
Fitness function	A scheduled load test with the 10x profile fails when autoscale lag or p95 exceeds its target

Solution architecture



Same load balancer, instances and database; the fleet now grows ahead of and during the spike, and shrinks afterwards.

Glossary

Used in this step

Term	Definition
Peak-to-average ratio	Peak demand divided by average demand; a high ratio makes fixed sizing wasteful
Autoscaling group	Automatically adjusts compute capacity based on demand
Horizontal scaling	Distribute workload across more instances
Scheduled scaling	Changes capacity at planned times, ahead of a known event
Startup lag	Time a new instance needs to boot, warm up and pass health checks
Vertical scaling	Add CPU/memory to one host
Serverless function (FaaS)	Executes short-lived functions triggered by events or requests
Cold start	Extra delay when a function or instance starts from nothing
Reserved capacity	Committed use of resources or spend for discounted pricing
Load balancer	Distributes incoming requests across multiple servers or instances

Related concepts

Term	Definition
Message queue	Buffers messages for asynchronous processing by consumers
Connection pool	Reuses database or network connections to reduce overhead
Spot instance	Discounted compute capacity that can be interrupted
Read replica	Database copy serving read operations to distribute query load
Multi-AZ deployment	Run workload across separate availability zones

A17 · Availability and deployment

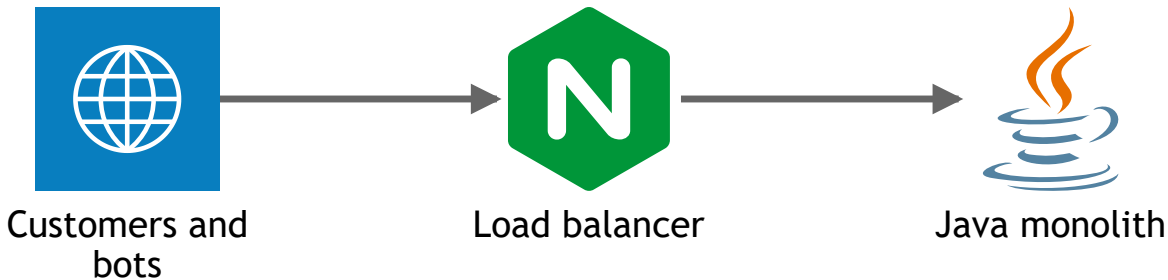
Reduce abusive API traffic

Bots consume 40% of the application’s processing capacity.

The load balancer passes every request straight to the application

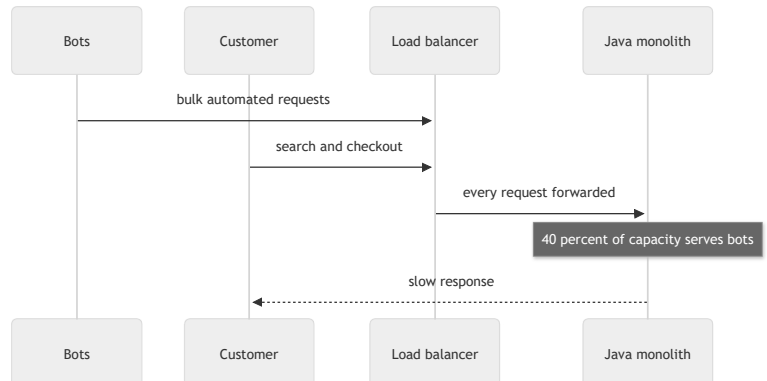
Component	What it is	What it does
Customers and bots	Every client that calls the public web and API endpoints	Send shopper requests and automated requests through the same entry point
Load balancer	Distributes incoming requests across multiple servers or instances	Forwards each request to the application without judging who sent it
Java monolith	One deployable application	Spends the same worker, CPU and database time on a bot request as on a customer request
Processing capacity	The CPU, workers and connections of the application tier	40% of it serves bots, and customers get the rest
Access logs	Records of each incoming request	Show client address, path and request rate, the evidence for telling bots from customers

Current architecture



Four requests in ten are not customers

Automated clients call the same endpoints as shoppers, and the application serves them all alike. Bot traffic takes 40% of capacity, so customers wait and capacity grows only to serve unwanted load.



40% of capacity

spent on bot requests

Shared endpoints

bots and customers call the same paths

Bot share far below 40%

target: customers still served and not rejected

Three ways to stop unwanted load before it costs capacity

Solution	What it is	What it does
Rate limiting + reverse proxy	A reverse proxy in front of the application that counts requests per client in a time window	Forwards requests within the limit and answers the excess itself, so they never reach the application
WAF	A web application firewall that filters malicious HTTP traffic	Matches requests against rules and blocks those that look like attacks or known bad clients
Scaling	More application instances behind the load balancer	Adds capacity so every request is served, bot requests included

Filtering risks wrong decisions, scaling pays for the unwanted load

Solution	Gains	Costs
Rate limiting + reverse proxy	Protects the application at its edge One explainable rule for every endpoint: N requests per client per window	False rejections: customers sharing one address can hit a limit together Limits need tuning from real traffic; bots can spread across many addresses
WAF	Filters attack patterns and known bad clients Maintained rule sets save effort	False positives block legitimate requests that look unusual Rule tuning; well-formed high-volume requests can still pass
Scaling	No customer is rejected No new component	Pays for unwanted load: bot requests are still processed Treats the symptom, so bots can grow with the capacity

Smallest safe move

Per-client rate limits in a reverse proxy, observed before enforced

- **Scope:** the endpoints bots hit most, found in access logs; payment provider callbacks and partners are allowlisted
- **Change:** a reverse proxy between load balancer and application counts requests per client and answers the excess with 429 and `Retry-After`
- **Safety net:** log-only mode first, checked against replayed customer traffic; limits sit above the legitimate peak; the real client address comes from a trusted header
- **Escape route:** point the load balancer at the application again, or return the limit to log-only
- **Deferred:** WAF for payload-based attacks; API gateway for many client-facing policies; scaling is no fix

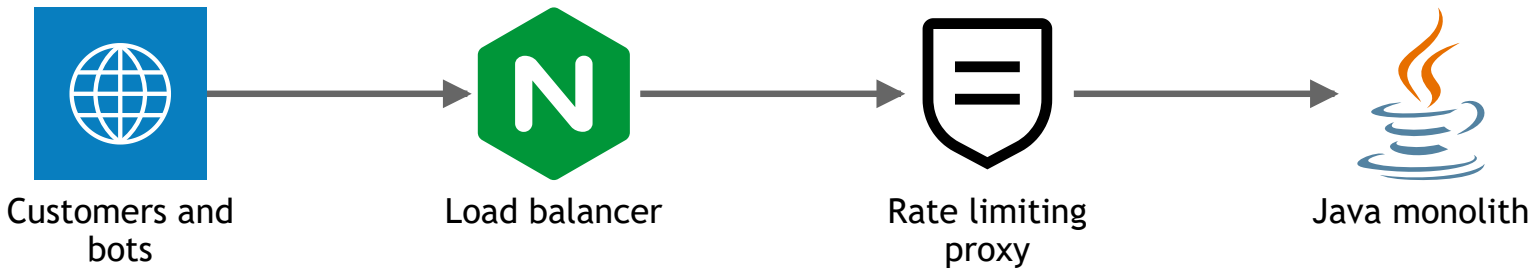
```
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
location /api/ {
    limit_req zone=api burst=20 nodelay;   # illustrative limit
    limit_req_dry_run on;                  # log only until validated
    limit_req_status 429;
}
```

Verification: replay legitimate and abusive traffic

Check	Pass condition
Bot share	Falls from 40% to the agreed target, for example below 5%, under replayed abusive traffic
Legitimate rejections	Customer requests answered with 429 stay under the agreed rate, for example 0.1%
Critical flows	0 rejections for checkout and payment provider callbacks
Application load	CPU and worker use fall at the same customer volume; customer p95 stays within SLO
Log-only result	Requests the limit would reject are abusive clients, not shoppers
Fitness function	A replay of a customer and abuse mix runs in the pipeline and fails when rejections or bot share pass their thresholds

Rollback: routing the load balancer back to the application removes the proxy and its limits. No business state is involved, and the counters are transient.

Solution architecture



Same application and load balancer; a rate limiting reverse proxy now answers excess requests before they cost application capacity.

Glossary

Used in this step

Term	Definition
Rate limiter	Controls the number of requests allowed within a time window
Reverse proxy	Intermediary that forwards client requests to backend services
WAF	Web application firewall; filters malicious HTTP traffic
Horizontal scaling	Distribute workload across more instances
Load balancer	Distributes incoming requests across multiple servers or instances
HTTP 429 and Retry-After	Status code for too many requests, and the header telling the client when to retry
False rejection	A legitimate request refused by a limit or a filter; also called a false positive
Log-only mode	A rule records what it would reject without rejecting it
Allowlist	Clients or sources exempt from a limit, such as payment provider callbacks
Client address	Address of the original caller; behind a load balancer it comes from a trusted forwarded header

Related concepts

Term	Definition
API gateway	Central entry point for routing, authentication, rate limiting, and API policies
Ingress controller	Routes external traffic into Kubernetes services
Service mesh	Manages service-to-service traffic, security, and observability
Autoscaling group	Automatically adjusts compute capacity based on demand
CDN	Distributes cached content closer to users

A18 · Availability and deployment

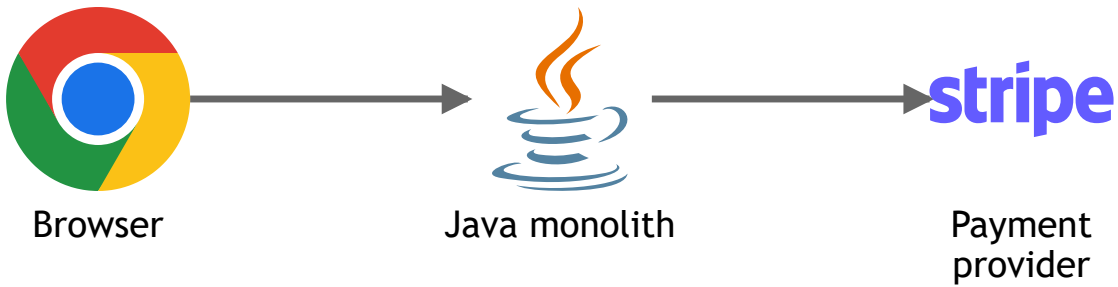
Contain slow payment dependency

The payment provider blocks for 20 seconds, and the hung calls exhaust the request workers.

Checkout waits for the payment provider inside a request worker

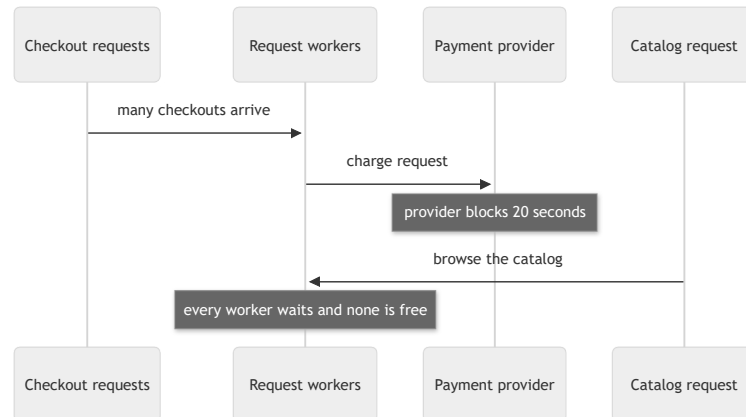
Component	What it is	What it does
Browser / SPA	Customer-facing web client	Submits checkout and keeps browsing the catalog and cart
Java monolith	One application with a bounded pool of request workers	Serves every endpoint, checkout and catalog included, with the same workers
Checkout module	Code that owns order placement	Calls the payment provider synchronously and holds its worker until the call returns
Payment provider	External vendor API, such as a card processor	Authorizes and captures the charge; its response time is outside the platform’s control
Request worker	A thread that serves one request at a time	Stays blocked for the whole provider call and is unavailable to every other request

Current architecture



One slow vendor stops the whole shop

Provider calls start to hang for 20 seconds. Each checkout holds a worker the whole time, the pool fills, and catalog and cart requests that never touch payments wait for a free worker too.



20 seconds

how long the provider blocks one call

All workers busy

the pool is exhausted and unrelated endpoints stall

Unrelated endpoints on SLO

target: catalog and cart hold p95, and no order is charged twice

Four ways to keep a slow dependency from taking the workers

Solution	What it is	What it does
Timeout + backoff	A deadline on every call, and retries with increasing delays	Frees the worker when the deadline passes and retries transient failures after growing pauses
Circuit Breaker	A switch that stops requests to a failing dependency temporarily	Opens after repeated failures, rejects calls at once, then probes the provider to close again
Bulkhead	Separate resources for each dependency so failures do not spread	Gives payment calls their own bounded pool, so only checkout can run out of workers
Graceful Degradation	Retaining core capability when an optional dependency fails	Keeps browsing and cart working and answers checkout with a reduced but honest result

Each guard protects the workers by giving up some completeness

Solution	Gains	Costs
Timeout + backoff	Transient failures recover without an operator Worker time per call is bounded: busy workers ≈ arrival rate × timeout	Amplification: retries add load to a provider that is already struggling A timed-out charge has an unknown outcome and can be charged twice
Circuit Breaker	Fail-fast: no worker waits once the breaker is open The provider gets room to recover	Rejects requests, including some that would have succeeded Thresholds and probes need tuning
Bulkhead	Isolation: payment trouble cannot take the workers of other endpoints Predictable blast radius	Reserved capacity: slots set aside for payment sit idle while it is healthy Pool sizes need tuning per dependency
Graceful Degradation	Partial service: browsing and cart keep working Preserves partial business value	Reduced functionality: checkout cannot confirm payment at once

Smallest safe move

A hard timeout on the payment call, without blind retries

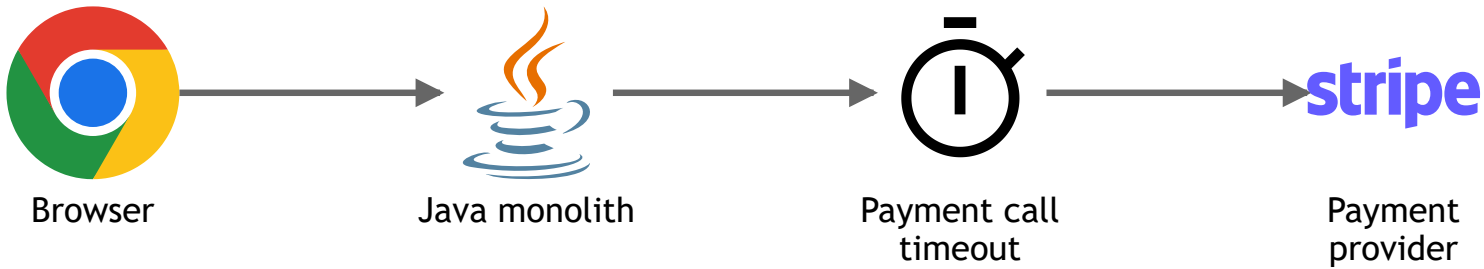
- **Scope:** only the outbound payment call in checkout
- **Change:** a deadline far below the 20-second hang, for example 3 seconds, and a retryable error for the customer when it passes
- **Safety net:** an idempotency key per order attempt, so a retry cannot double charge; automatic retries stay in a small budget with jittered backoff because retries amplify load
- **Escape route:** the timeout is configuration; raise or remove it
- **Deferred:** bulkhead if payment still holds too many workers; circuit breaker if every call still pays the timeout; degradation if the business accepts payment pending

```
HttpRequest request = HttpRequest.newBuilder(uri)
    .timeout(Duration.ofSeconds(3))                // illustrative deadline
    .header("Idempotency-Key", orderAttemptId)
    .POST(body).build();
```

Verification: inject a long provider hang

Check	Pass condition
Injected fault	A provider stub blocks 20 seconds on every payment call while normal checkout load runs
Unrelated endpoints	Catalog and cart p95 stay within SLO during the hang
Worker occupancy	Workers held by payment calls stay below the share planned for them: arrival rate × timeout
Timed-out checkouts	Fail within the timeout with a retryable response; none waits 20 seconds
Double charge	0 orders charged twice after timeouts and retries with the same idempotency key
Fitness function	A fault-injection test in the pipeline fails when unrelated endpoints breach their SLO during a provider hang

Solution architecture



Same checkout path; a deadline on the payment call now frees the worker long before the provider answers.

Glossary

Used in this step

Term	Definition
Timeout	Limits how long an operation may wait for completion
Retry with backoff	Repeats failed operations with increasing delays
Jitter	Random variation added to retry delays so clients do not retry in step
Retry budget	Cap on retries relative to normal calls, so retries cannot multiply the load
Circuit breaker	Stops requests to failing dependencies temporarily
Bulkhead pattern	Isolates resources so failures do not spread across components
Graceful degradation	Retain core capability when optional dependencies fail
Idempotency key	Stable identifier recognizing multiple attempts at the same logical command
Fail-fast	Rejecting a call at once instead of waiting on a dependency that is likely failing
Request worker	Thread that serves one request at a time; a blocked worker serves nobody else

Related concepts

Term	Definition
Reserved capacity	Committed use of resources or spend for discounted pricing
Message queue	Buffers messages for asynchronous processing by consumers
Health check	Detects whether an application instance is functioning correctly
SLO	Target for an SLI over a defined time window
p99 latency	Duration not exceeded by 99 percent of observations

A19 · Availability and deployment

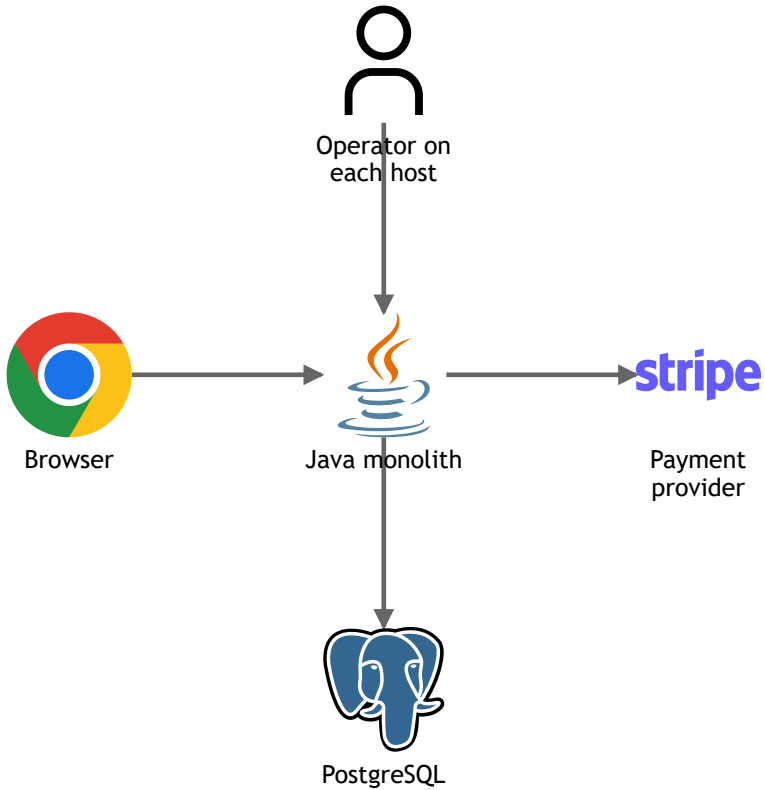
Observe cross-service failures

Operators cannot locate the fault source when a request crosses several components.

Each instance keeps its own plain text log

Component	What it is	What it does
Java monolith instances	Two or more identical application instances	Each writes its own log lines, with no identifier shared across instances
Local log files	Plain text files on each host	Hold free-form lines; reading them means logging in to each host and searching
PostgreSQL	Relational transactional database	Records slow queries on its own host, in its own format
Payment provider	External vendor API	Is visible only through the monolith's own log line about the call, if one exists
Operator	The person investigating a failure	Compares timestamps across hosts by hand to guess which component failed

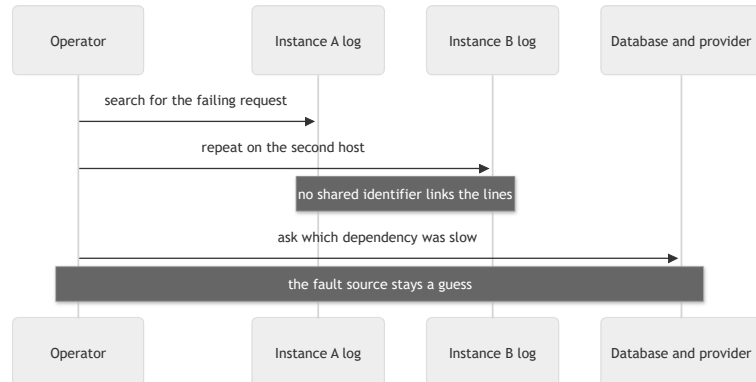
Current architecture



A request crosses the monolith, the database and the payment provider, while operators read logs host by host.

A slow checkout and no way to say why

Checkout is slow for some customers. The evidence for one request sits in separate logs on each instance, in the database and at the provider, with no common key to join them.



Unlocated fault

nobody can name the component that slowed or failed a request

One request, many logs

its evidence is spread over instances and dependencies

One search

target: show every step of a request and the time each took

Three ways to see inside a request path

Solution	What it is	What it does
Centralized Logging	Aggregation of application logs from multiple services	Ships structured log lines from every instance to one store, searched and joined by a correlation id
Metrics + Alerts	Numerical measurements of system behavior, with notifications when thresholds are exceeded	Shows rates, errors and latency per component and alerts when an SLO is at risk, not which request failed
Distributed Tracing	Tracking of a request across services and dependencies	Records a timed span for each hop under one trace id and shows the request as a timeline

Each view costs storage, blind spots or overhead

Solution	Gains	Costs
Centralized Logging	Searchable: one query across every instance Cheap to start; reuses existing logging calls	Storage and retention cost grow with log volume Without a correlation id the lines of one request still cannot be joined
Metrics + Alerts	SLO evidence: shows when and how much service degrades Alerts replace waiting for customer complaints	Incomplete cause: shows that something is slow, not which request or why Per-customer or per-order labels make metrics expensive
Distributed Tracing	Dependency diagnosis: shows which hop used the time Makes the call path visible without guessing	Overhead: instrumentation and context propagation in every component Sampling drops some requests, and traces need storage and tooling

Deciding signal: time an operator needs to name the failing component, hops per request, and the budget for storing and processing telemetry.

Smallest safe move

Structured logs with a correlation id in one searchable store

- **Scope:** log output of the monolith, including its database and payment calls
- **Change:** create a correlation id at the edge, add it to every log line and outgoing call, log each call's duration, ship lines to one store
- **Safety net:** asynchronous shipping fails open, so a down log store never blocks requests; logs carry no card or personal data
- **Escape route:** local files stay on each host; switch the shipper off
- **Deferred:** metrics with alerts when detection is the pain; tracing when requests cross more services than log joins can follow

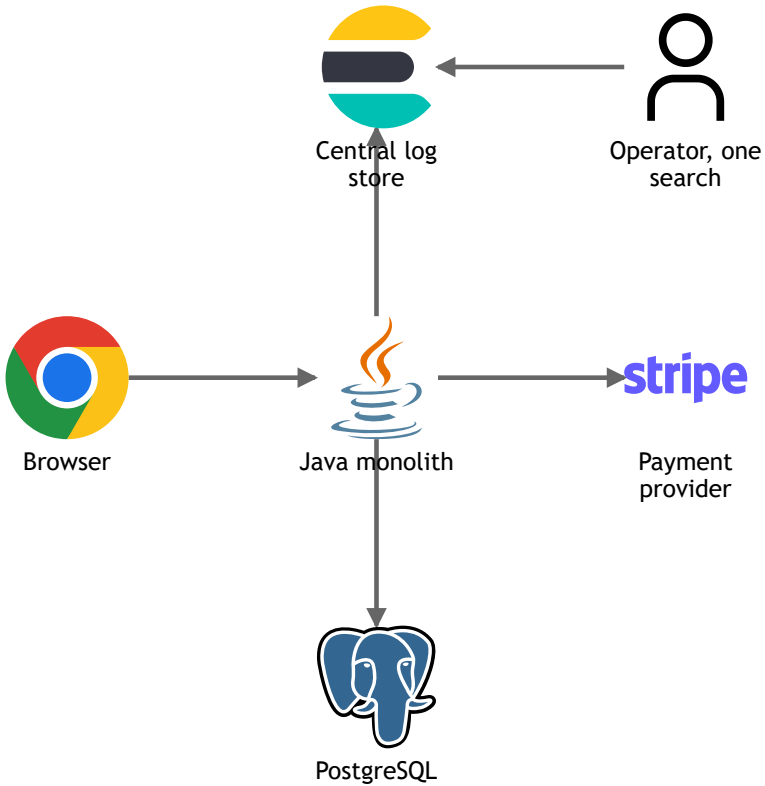
```
{"correlationId":"c-4f7a","component":"postgres","event":"query.done","durationMs":12}
{"correlationId":"c-4f7a","component":"payment","event":"call.done","durationMs":20012}
```

Verification: find the bottleneck of one request

Check	Pass condition
Fault drill	A slow payment call and a slow query are injected in turn; an operator names the faulty component from the logs alone
One search	One query on a correlation id returns every log line of the request from every instance
Time per step	Each database and payment call logs its duration, so the slowest hop is visible without comparing clocks
Propagation	Share of log lines carrying a correlation id, for example above 99%
Ingestion	Lines reach the store within the agreed delay; request latency is unchanged while the store is down
Fitness function	A test fails the build when a log line lacks the correlation id or an outgoing call lacks a duration

Rollback: switching the shipper off returns to local files; the request path was never changed. Stored log lines are evidence and stay, and no business data is touched.

Solution architecture



Same request path; every instance now ships correlated log lines to one store, where the operator searches once.

Glossary

Used in this step

Term	Definition
Centralized logging	Aggregates application logs from multiple services
Structured log	Log line written as named fields, such as JSON, so it can be filtered and joined
Correlation ID	Identifier carried across related operations
Distributed tracing	Tracks requests across multiple services and dependencies
Trace span	Timed unit of work within distributed trace
Metrics collection	Captures numerical measurements of system performance and behavior
Alerting system	Notifies operators when monitored conditions exceed thresholds
SLO	Target for an SLI over a defined time window
Sampling	Recording only a fraction of traces to limit overhead
Log retention	How long stored log lines are kept before deletion
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
SLI	Measured numerical indicator of service behavior
MTTR	Average recovery duration across incidents
p95 latency	Duration not exceeded by 95 percent of observations
p99 latency	Duration not exceeded by 99 percent of observations
Error budget	Maximum permitted unreliability under an SLO

Phase 3

Messaging and distributed workflows

Steps in this phase

Step	Forcing constraint
20 Move email off checkout critical path	Confirmation email adds 1.5 s; checkout p95 target 300 ms
21 Prevent queue backlog runaway	1000 msgs/s arrive; only 500 msgs/s processed; backlog grows 500/s
22 Isolate poison messages	One malformed event retries indefinitely and blocks progress
23 Decouple order consumers	New consumers should not modify order release
24 Prevent lost events between DB and broker	Crash between DB commit and publish leaves fulfillment unaware
25 Replay historical business events	Fraud analysis must replay prior 30 days
26 Coordinate long-running checkout	Payment success may precede later inventory failure
27 Eliminate query-time service fanout	p95 exceeds 500 ms from remote calls
28 Make webhook retries safe	One business charge and fulfillment effect per payment

A20 · Messaging and distributed workflows

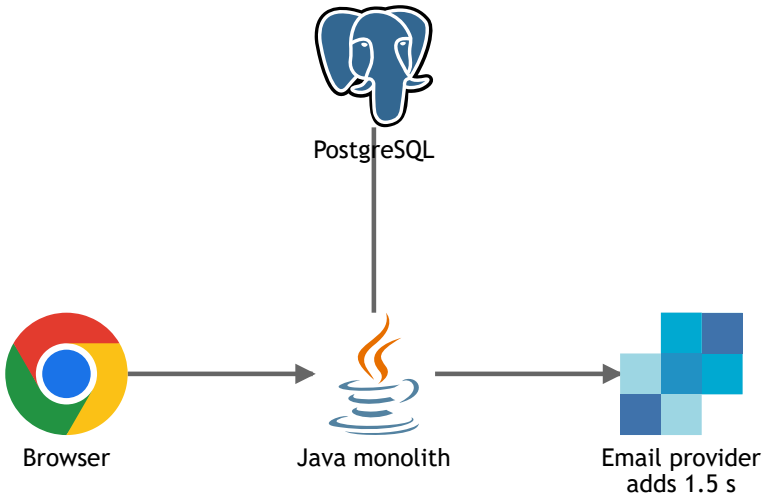
Move email off checkout critical path

The confirmation email adds 1.5 s to checkout, whose p95 target is 300 ms.

Checkout sends the confirmation email before it answers

Component	What it is	What it does
Browser / SPA	Customer-facing web client	Submits the order and waits for the confirmation response
Java monolith	One deployable application with a pool of request workers	Runs checkout and holds a worker until the response is sent
Checkout module	Code that places an order	Commits the order, then sends the confirmation email inside the same request
PostgreSQL	Relational transactional database	Stores the committed order
Email provider	External email service reached by SMTP or HTTP, such as SendGrid	Accepts each confirmation; adds about 1.5 s per call

Current architecture



The email provider call runs inside the checkout request and adds 1.5 s to every order.

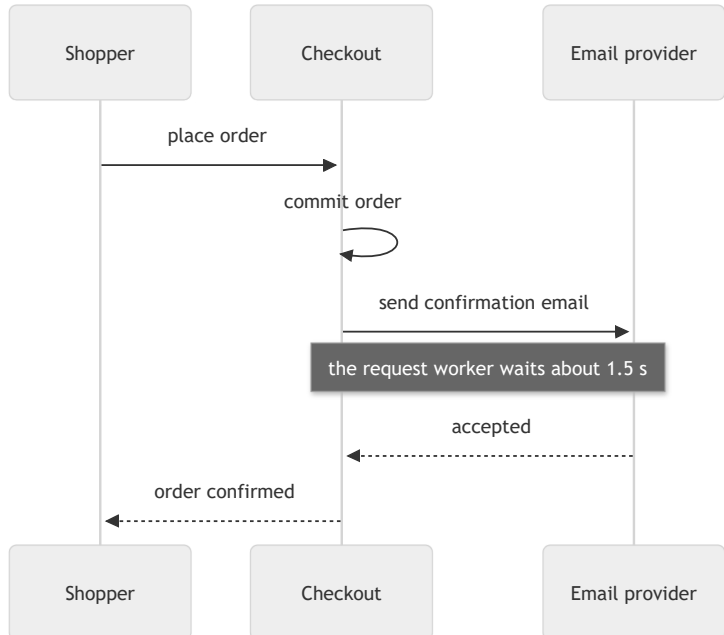
Every order waits for the mail server

Checkout calls the email provider and holds a request worker until it answers. Shoppers wait for work they do not need to see, and a slow or failing provider slows or fails checkout itself.

+1.5 s
added to each checkout by the confirmation email

Synchronous call
a request worker is held for the full provider round trip

p95 300 ms
checkout target, with the order still committed when email is down



Three ways to take the email off the request path

Solution	What it is	What it does
Message queue + worker	A broker queue plus a separate consumer process, such as RabbitMQ and an email worker	Checkout publishes a message and returns; the worker sends the email later
FaaS	Short-lived functions run by a cloud platform when an event arrives	Sends the email on demand, scaled and billed per invocation
In-process fire-and-forget	A background thread or async call inside the monolith	Starts the send and returns at once; nothing is stored

Each option removes the wait and adds a different risk

Solution	Gains	Costs
Message queue + worker	Checkout no longer waits for the provider A durable queue keeps messages through a provider outage while the worker retries	Delayed email: it arrives seconds or minutes after the order A broker and a worker to run; at-least-once delivery can send duplicates
FaaS	Scales with demand and down to zero when idle No worker servers to patch	Quotas on concurrency and execution time Cold starts; buffering and retries depend on the trigger and the platform
In-process fire-and-forget	Easiest change: no new infrastructure Checkout returns at once	Lost on crash: a restart drops every unsent email No retry and no record of what is pending

Deciding signal: whether the shopper needs the email result now or only the order acknowledgement, how long a delay is acceptable, and whether a lost email is acceptable.

Smallest safe move

Publish a message after commit and send from a worker

- **Scope:** the confirmation email only; the order transaction and the response to the shopper stay as they are
- **Change:** after the order commits, checkout publishes an order-confirmed message carrying the order id; a worker reads it and calls the provider
- **Safety net:** the worker sends one email per order id, because a message can be delivered twice
- **Escape route:** a feature flag switches checkout back to the synchronous send
- **Deferred:** FaaS, until worker operations or bursty volume justify it; a crash between commit and publish can drop one email, which an outbox would close once that loss is unacceptable

```
orderRepository.save(order);                        // commit first
if (flags.asyncEmail()) publisher.publish("order.confirmed", order.id());
else emailClient.sendConfirmation(order);           // previous synchronous path
```

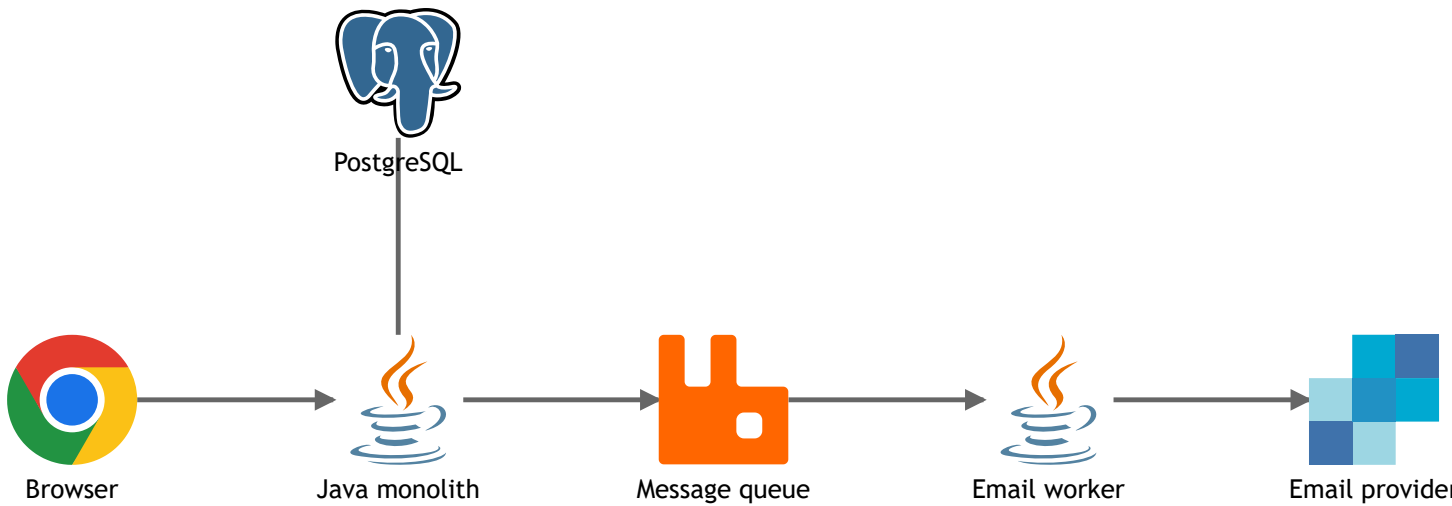
Verification: provider disabled, checkout under target

Check	Pass condition
Checkout latency	p95 at or below 300 ms under replayed production traffic, with the provider still taking 1.5 s
Email disabled	Orders still commit; none is lost or rejected because of email
Email delivery	After the provider returns, every committed order has one confirmation email
Queue age	The oldest waiting message stays within the agreed email delay
Redelivery	A message delivered twice sends one email
Fitness function	A test with the provider disabled fails the build when checkout fails or p95 exceeds the target

Rollback: the flag restores the synchronous send and changes no data. State rollback is separate: messages already queued must be drained by the worker or deliberately purged.

Stop: when p95 meets 300 ms and no email is lost during a provider outage, no FaaS or further layer is added.

Solution architecture



Same browser, monolith, database and provider; a queue and a worker now sit between checkout and the provider.

Glossary

Used in this step

Term	Definition
Critical path	Steps a request must finish before its response is sent
Synchronous call	The caller waits for the result before it continues
Asynchronous processing	Work is accepted now and completed later, after the response
Message queue	Buffers messages for asynchronous processing by consumers
Message broker	Routes and delivers messages between producers and consumers
Worker	Separate process that takes messages from a queue and does the work
Serverless function (FaaS)	Executes short-lived functions triggered by events or requests
Fire-and-forget	Starts work and returns without waiting for or recording its outcome
At-least-once delivery	Delivery semantics allowing duplicate deliveries in exchange for retries
Idempotency key	Stable identifier recognizing multiple attempts at the same logical command

Related concepts

Term	Definition
Transactional outbox	Atomically records business changes and outgoing events in the same database transaction
Dead letter queue (DLQ)	Stores messages that could not be processed successfully
Retry with backoff	Repeats failed operations with increasing delays
Circuit breaker	Stops requests to failing dependencies temporarily
Timeout	Limits how long an operation may wait for completion

A21 · Messaging and distributed workflows

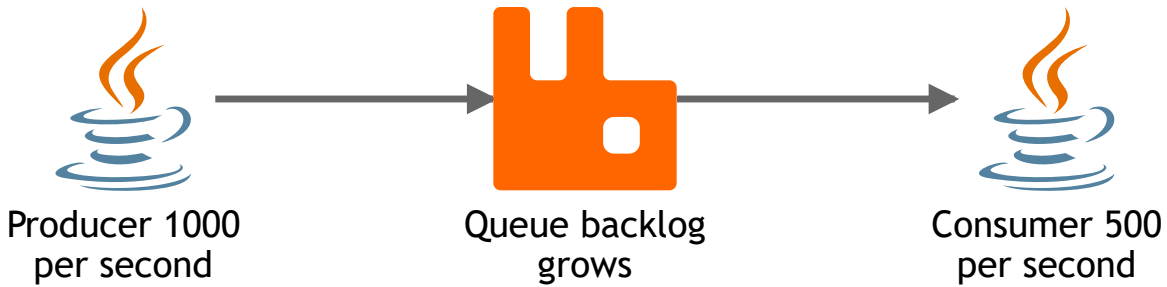
Prevent queue backlog runaway

1000 msgs/s arrive but only 500 msgs/s are processed, so the backlog grows 500 msgs/s.

One queue feeds one consumer

Component	What it is	What it does
Producer	Application code that publishes a message for each business event	Publishes 1000 msgs/s during the peak in this scenario
Message queue	Broker queue that buffers messages	Holds each message until a consumer takes it
Consumer	One worker process reading the queue	Processes 500 msgs/s and acknowledges each finished message
Acknowledgement	The consumer’s signal that a message is done	Lets the broker delete it; an unacknowledged message is redelivered
Backlog	Messages waiting in the queue	Grows by the gap between arrival rate and processing rate

Current architecture



A queue buffers the deficit, it does not remove it

Messages arrive faster than the single consumer can finish them. Every second 500 more messages wait than leave, so each new message waits longer until deadlines are missed or the broker runs out of space.

+500 msgs/s

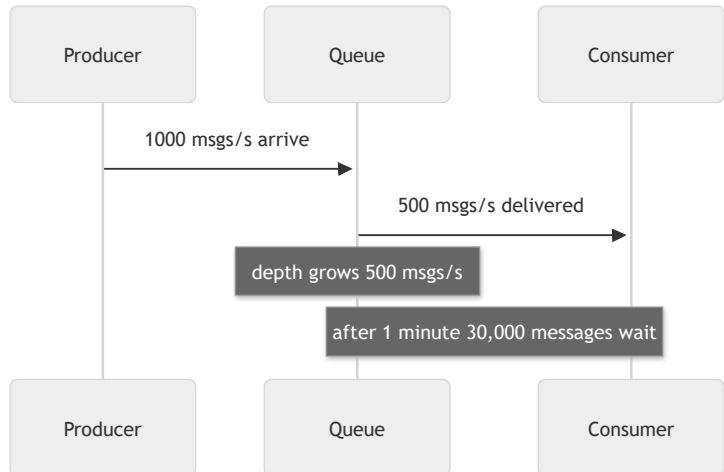
backlog growth: 30,000 per minute and 300,000 after 10 minutes

1000 in, 500 out

arrival rate against processing rate

Drain within SLA

target: processing rate above arrival rate, with the drain time agreed



Four ways to match processing to arrival

Solution	What it is	What it does
More workers	Several consumers reading the same queue, known as competing consumers	Each message goes to one of them, so throughput is the sum of their rates
Batch processing	Process groups of work together	Takes several messages per call, so per-message overhead is paid once per batch
Backpressure	Limit or slow producers when consumers cannot keep up	Delays, throttles or rejects new messages so the backlog stays bounded
Partitioning	Splitting the stream into partitions chosen by a routing key	Each partition has its own consumer, so work runs in parallel and per-key order is kept

Every option buys throughput with ordering, delay or slower producers

Solution	Gains	Costs
More workers	Throughput grows with the number of consumers No change to producers or messages	Ordering across messages is lost, because workers finish out of order Shared downstream resources see the higher rate
Batch processing	Fewer round trips and commits per message More throughput from the same consumer	Delay: each message waits for its batch A failed batch is harder to retry cleanly
Backpressure	The backlog is bounded, so memory and disk stay safe Overload becomes visible to producers	Slower upstream work or rejected requests Producers must handle slowdown and rejection
Partitioning	Parallel consumers with per-key order Capacity scales with the partition count	Needs a routing key and an even spread, and a hot key stays slow Consumer count is capped by the partition count

Smallest safe move

Competing consumers sized above the arrival rate

- **Scope:** the one queue and its consumer; producers and message format stay unchanged
- **Change:** run enough consumers on the same queue to exceed 1000 msgs/s with headroom, for example 3 consumers (illustrative)
- **Safety net:** messages are independent or handled idempotently; alerts on queue depth and oldest message age; the downstream limit is checked before scaling
- **Escape route:** scale back to one consumer; queued messages stay queued
- **Deferred:** backpressure when arrival can exceed total capacity; batching when per-message overhead dominates; partitioning when per-key order is required

```
backlog growth = 1000 - 500 = 500 msgs/s = 30,000 per minute
10 minutes     = 300,000 messages waiting
3 consumers    = 1500 msgs/s, the surplus of 500 msgs/s drains 30,000 in 60 s
```

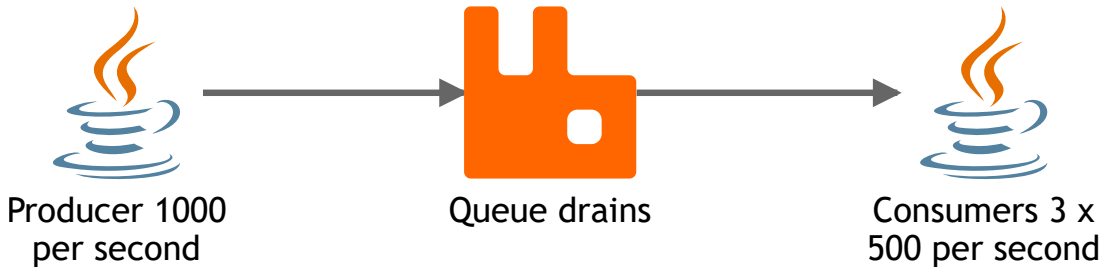
Verification: one-hour burst and time to drain

Check	Pass condition
Burst simulation	One hour at 1000 msgs/s; queue depth stays flat or shrinks instead of adding 500 msgs/s
Time to drain	A backlog built during a test spike returns to zero within the agreed drain SLA
Message age	The oldest message stays under the business deadline
Ordering and duplicates	No ordering rule is violated; a redelivered message has no second effect
Downstream load	The database or provider stays within its limits at the higher rate
Fitness function	A sustained load test fails when queue depth trends upward; alert on depth and message age

Rollback: scaling consumers back removes capacity but loses no message. State rollback is separate: effects already produced by extra consumers are not undone, so reprocessing needs idempotent handlers.

Stop: when capacity stays above arrival and the drain time meets the SLA, no batching, partitioning or backpressure is added.

Solution architecture



Same producer and queue; the single consumer becomes a group of competing consumers whose total rate exceeds arrival.

Glossary

Used in this step

Term	Definition
Message queue	Buffers messages for asynchronous processing by consumers
Consumer	Process that reads messages from a queue and does the work
Acknowledgement	Consumer signal that a message is finished and can be deleted
Backlog	Messages waiting in a queue, also called queue depth
Arrival rate and service rate	Messages entering the queue per second against messages finished per second
Drain time	Backlog divided by the surplus of service rate over arrival rate
Competing consumers	Several consumers on one queue, each message going to one of them
Batch processing	Process groups of work together
Backpressure	Limit or slow producers when consumers cannot keep up
Partitioning	Splits a stream into partitions by a key, each read by one consumer

Related concepts

Term	Definition
Consumer group	Cooperating consumers divide partitions of a stream
Horizontal scaling	Distribute workload across more instances
Autoscaling group	Automatically adjusts compute capacity based on demand
Dead letter queue (DLQ)	Stores messages that could not be processed successfully
Event streaming platform	Stores ordered event streams for independent consumers
At-least-once delivery	Delivery semantics allowing duplicate deliveries in exchange for retries

A22 · Messaging and distributed workflows

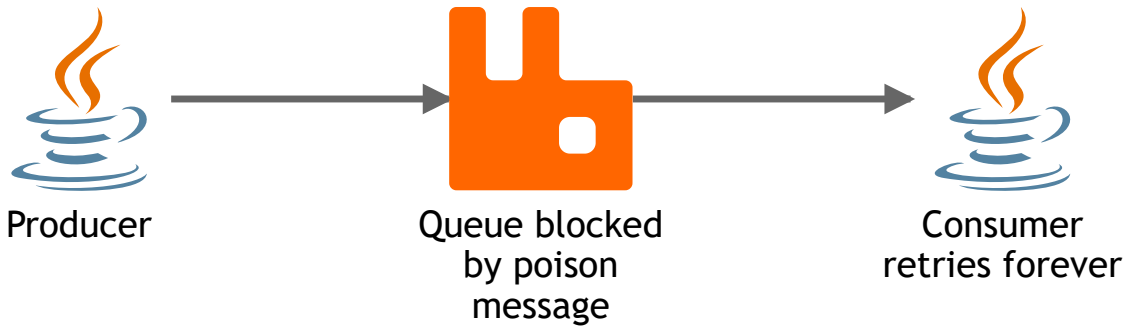
Isolate poison messages

One malformed event retries indefinitely and blocks every healthy event behind it.

A failed message is retried without limit

Component	What it is	What it does
Producer	Application code that publishes events	Sends each event to the queue; it cannot know whether an event is valid for the consumer
Message queue	Broker queue holding events for one consumer	Redelivers a message until the consumer acknowledges it
Consumer	Process that handles one message at a time	Parses and applies each event; on failure it returns the message to the queue
Retry policy	Rule for what happens after a failure	Unbounded: immediate redelivery, no attempt limit and no delay
Malformed event	A message the consumer can never process, for example a missing required field	Fails on every attempt, so it is never acknowledged

Current architecture



One bad message stops the line

A producer publishes an event the consumer cannot parse. The consumer fails and the broker redelivers the same message at once, so every attempt fails and healthy events stay queued behind it while retries burn CPU and logs.

1 malformed event

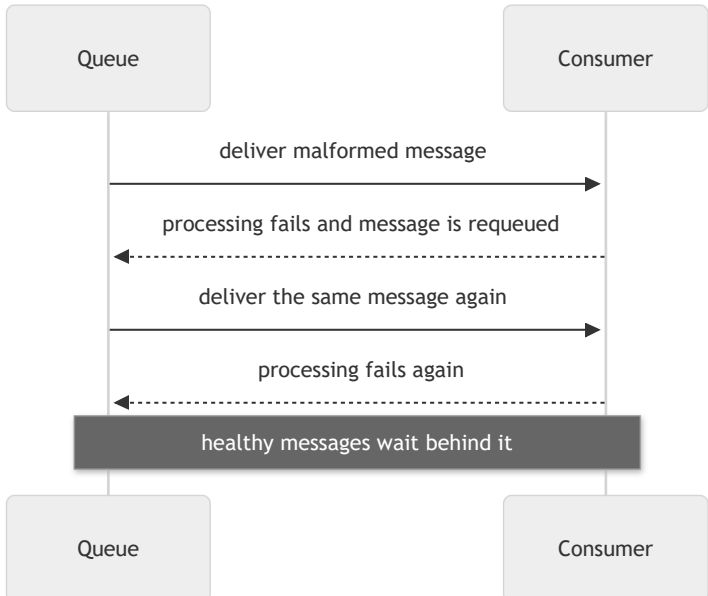
blocks every healthy event behind it

Unlimited retries

no attempt limit and no delay: a retry storm

0 healthy events blocked

target: healthy events keep flowing while the bad one is isolated



Three ways to stop one message from blocking the rest

Solution	What it is	What it does
DLQ	A separate queue that stores messages that could not be processed successfully	Receives a message when its attempts are exhausted, so the main queue keeps flowing
Bounded retries + backoff	A fixed attempt limit with growing delays between attempts	Absorbs transient failures and stops hammering when the failure is permanent
At-most-once	Delivery without retries or redelivery	Acknowledges the message before processing, so a failure drops it

Isolation costs replay tooling, delay or lost messages

Solution	Gains	Costs
DLQ	The main queue keeps flowing The failed message is kept with its error for diagnosis	Replay tooling: someone must inspect, fix and redrive the messages Needs depth monitoring and an owner, or it becomes a silent graveyard
Bounded retries + backoff	Recovers from transient failures such as a short database outage Ends the retry storm	Delay: a message is processed later, after waits that grow After the last attempt the message still needs a place to go
At-most-once	Simplest: no retries, no duplicates, nothing blocks No extra queue to run	Message loss is possible, including valid messages on a transient failure No record of what failed unless it is logged

Deciding signal: whether losing the message is acceptable, the share of failures that are transient or permanent, and how much the retries multiply load on the downstream system.

Smallest safe move

Retry a few times, then park the message in a DLQ

- **Scope:** the one queue and its consumer; producers and message format stay unchanged
- **Change:** a limited number of attempts with growing delays, for example 5 (illustrative), then the message moves to a dead letter queue with its failure reason
- **Safety net:** the handler is idempotent, because retries redeliver; non-retryable errors skip the retries; depth alert and a named owner for the DLQ
- **Escape route:** change the attempt limit or the routing in configuration; parked messages stay in the DLQ
- **Deferred:** at-most-once, only for messages whose loss costs nothing; replay tooling beyond a manual redrive, until DLQ volume justifies it

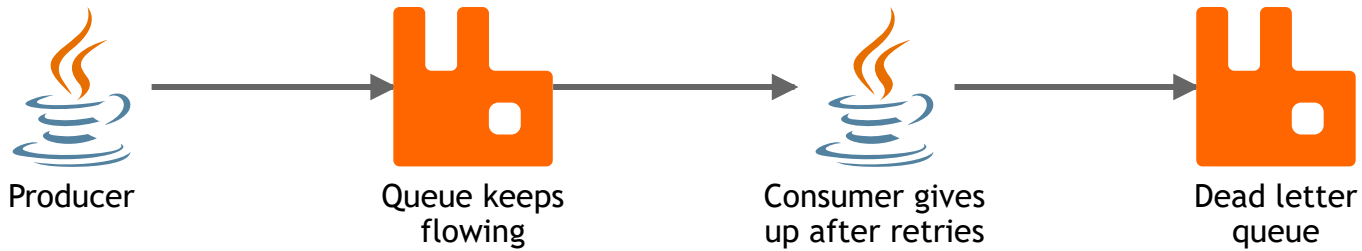
```
delay before attempt n = 1 s x 2^(n-1) : 1 s, 2 s, 4 s, 8 s    (illustrative)
after the last attempt or a non-retryable error → dead letter queue
the parked message keeps its payload, failure reason and attempt count
```

Verification: a poison message among healthy events

Check	Pass condition
Poison insertion	One malformed message is published in the middle of a stream of healthy events
Healthy events	All healthy events are processed within the normal delay and none is blocked
Attempt limit	The malformed message is attempted no more than the limit, then moves to the DLQ
DLQ content	The message arrives with its payload, failure reason and attempt count
Retry load	Retries per second against the downstream stay within its limit; no retry storm
Fitness function	A test publishes a malformed message and fails the build when healthy events wait or the DLQ stays empty

Rollback: reverting the retry and routing configuration restores the old behavior. State rollback is separate: never purge the DLQ; redrive its messages after the fix, with idempotent handlers.

Solution architecture



Same producer, queue and consumer; a bounded number of attempts ends in a dead letter queue instead of an endless loop.

Glossary

Used in this step

Term	Definition
Poison message	Malformed or invalid message repeatedly causing consumer failures
Head-of-line blocking	A message at the front that cannot finish holds back every message behind it
Retry storm	Repeated retries that multiply load on a failing component
Transient and permanent failure	A failure that may pass on retry, against one that fails every time
Bounded retries + backoff	A fixed attempt limit with growing delays between attempts
Exponential backoff	Increase time between consecutive retry attempts
Dead letter queue (DLQ)	Stores messages that could not be processed successfully
Redrive	Moving parked messages back to the main queue after the cause is fixed
At-most-once delivery	Delivery semantics not retrying possibly lost messages
At-least-once delivery	Delivery semantics allowing duplicate deliveries in exchange for retries

Related concepts

Term	Definition
Message queue	Buffers messages for asynchronous processing by consumers
Idempotency key	Stable identifier recognizing multiple attempts at the same logical command
Event replay	Reconsume historical records to rebuild a projection
Circuit breaker	Stops requests to failing dependencies temporarily
Timeout	Limits how long an operation may wait for completion
Bulkhead pattern	Isolates resources so failures do not spread across components

A23 · Messaging and distributed workflows

Decouple order consumers

New consumers should not require changes to order release.

Order release calls every consumer itself

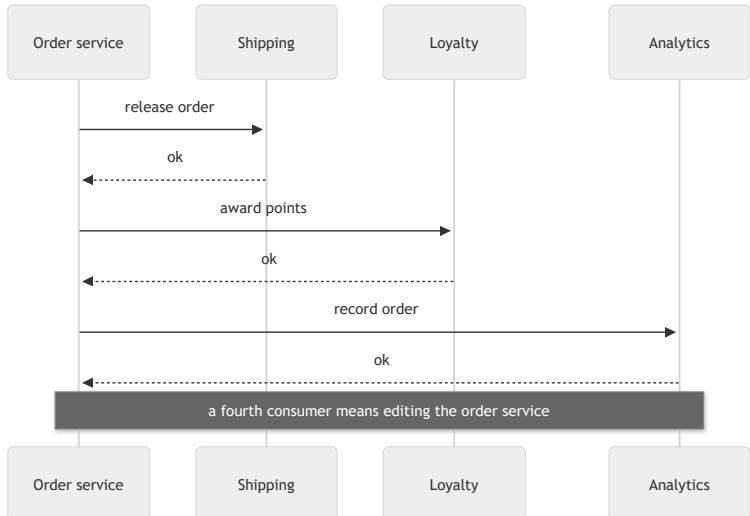
Component	What it is	What it does
Order service	Code that releases a paid order	Calls shipping, loyalty and analytics one after another
Shipping	Capability that prepares the shipment	Receives a direct call with the order data and answers it
Loyalty	Capability that awards points	Receives the same direct call and answers it
Analytics	Capability that records order metrics	Receives the same direct call and answers it
Direct call	Synchronous request from one capability to another	Makes the order service know every consumer and wait for each answer

Current architecture

The order service calls each consumer directly, so every new consumer changes the order release.

A new consumer is a new order release

Every new reaction to an order needs another call inside order release. The order service is edited, tested and deployed each time, and a slow or failing consumer delays or fails the release itself.



3 direct calls

made by order release today

1 change per consumer

the order service is edited for each consumer added

0 publisher changes

target: add a consumer without touching order release

Three ways to let consumers react to an order

Solution	What it is	What it does
Pub/sub	The publisher sends an event to a topic and each subscriber receives its own copy	The order service publishes one event and does not know who consumes it
Direct APIs	Synchronous calls from the order service to each consumer	Every consumer gets the order at once and answers the call
Event streaming	A platform that stores ordered event streams for independent consumers	Keeps events as a log, so consumers read at their own pace and can replay

Decoupling trades immediate answers for eventual consistency and operations

Solution	Gains	Costs
Pub/sub	New consumers join without a publisher change A slow or failed consumer no longer delays the order	Eventual consistency: consumers see the order a moment later Delivery is at least once, so consumers must tolerate duplicates
Direct APIs	Immediate: the order service knows each outcome at once Simple to follow and debug	Coupled: every new consumer changes the order service Temporal coupling: a slow consumer adds to release time
Event streaming	Events are kept, so consumers can replay history and join late Each consumer reads at its own position	Operations: a cluster, partitions and retention to run More than needed when no consumer must replay

Smallest safe move

Publish one order event beside the direct calls

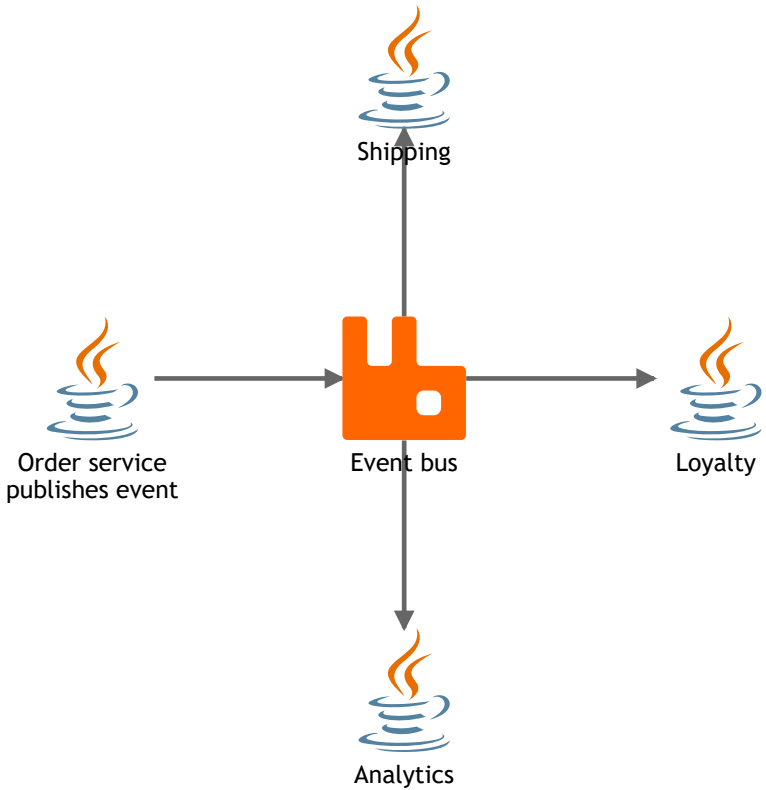
- **Scope:** one `OrderReleased` event added next to the existing direct calls; the release logic stays unchanged
- **Change:** the order service publishes the event to a topic; each new consumer subscribes with its own queue; analytics moves first, loyalty next
- **Safety net:** a feature flag keeps the direct call until the subscriber matches it; consumers ignore duplicates, since delivery is at least once
- **Escape route:** the flag restores the direct call for that consumer; the subscription can stay
- **Deferred:** shipping keeps its direct call until publishing after commit is reliable; event streaming, until a consumer must replay history

```
{ "type": "OrderReleased", "version": 1, "eventId": "evt-1001",
  "orderId": 18423, "occurredAt": "2026-11-02T10:15:00Z" }
```

Verification: a new consumer joins without a publisher change

Check	Pass condition
New consumer	A fourth subscriber is added with no change to the order service code, configuration or deployment
Delivery	Every released order reaches each subscriber, and a duplicate event changes nothing
Release latency	Order release p95 is no higher than with direct calls
Consumer outage	With one consumer stopped, orders still release and the consumer catches up on restart
Overlap comparison	For a migrating consumer, the event path and the direct call give the same result
Fitness function	A test adds a consumer without touching publisher code; alert when the oldest unprocessed event exceeds the agreed window

Solution architecture



Same order service and consumers; an event bus sits between them, so adding a consumer means adding a subscription only.

Glossary

Used in this step

Term	Definition
Publish/subscribe (Pub/Sub)	Distributes messages to multiple interested subscribers
Topic	Named channel a publisher sends to; every subscriber receives a copy
Subscription	A subscriber's own queue bound to a topic, read at its own pace
Event bus	Infrastructure for distributing events between components
Domain event	A fact about the business that has already happened, such as an order release
Direct API call	Synchronous request and response from one capability to another
Temporal coupling	Caller and callee must both be available at the same time
Eventual consistency	Consumers reflect a change after a delay, not in the same transaction
Event streaming platform	Stores ordered event streams for independent consumers

Related concepts

Term	Definition
Message queue	Buffers messages for asynchronous processing by consumers
Message broker	Routes and delivers messages between producers and consumers
Consumer group	Cooperating consumers divide partitions of a stream
Event replay	Reconsume historical records to rebuild a projection
Transactional outbox	Atomically records business changes and outgoing events in the same database transaction
Saga pattern	Coordinates distributed business transactions using local transactions and compensations

A24 · Messaging and distributed workflows

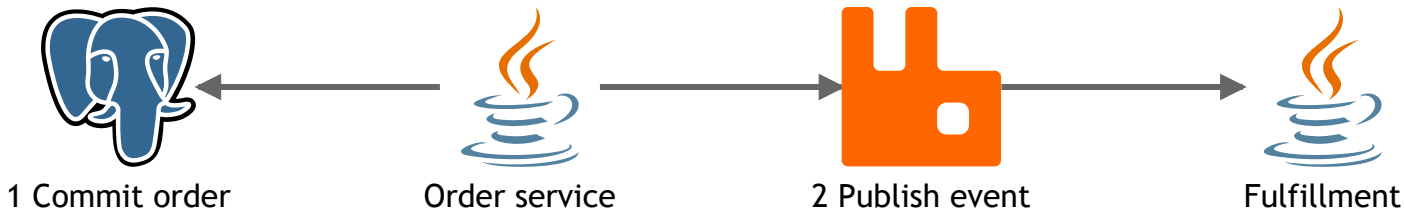
Prevent lost events between DB and broker

A crash between the DB commit and the publish leaves fulfillment unaware of a committed order.

The order commits first and the event is sent afterwards

Component	What it is	What it does
Order service	Java code that places and releases orders	Commits the order to PostgreSQL, then publishes an order event in a second step
PostgreSQL	Relational transactional database	Makes the order durable on commit; knows nothing about the broker
Message broker	Broker that already carries events between capabilities	Accepts the published event and delivers it to subscribers
Fulfillment	Consumer that prepares shipments	Acts only on events it receives, never on database rows
Order event	Message announcing that an order was committed	Is sent after the commit, so the two writes are independent

Current architecture



A crash between two writes loses the event

The order is committed, then the process is killed, redeployed or loses the broker before the publish. The order exists, fulfillment never hears of it, and nothing retries because no record says an event is owed.

Committed, never announced

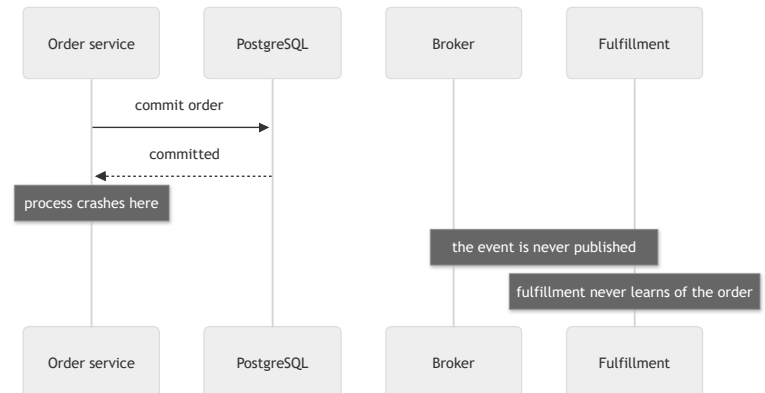
symptom: an order row exists but its event was never published

Any crash point

process kill, deploy or broker outage between commit and publish

0 lost events

target: every committed order eventually produces its event



Three ways to close the commit-publish gap

Solution	What it is	What it does
Transactional outbox	An outbox table written in the same database transaction as the order	A relay reads unsent rows, publishes them to the broker and marks them sent
Change data capture (CDC)	A mechanism that detects committed changes, typically from the database transaction log	Streams row changes to the broker without the application publishing anything
Two-phase commit (2PC)	A protocol in which a coordinator has the database and the broker prepare, then commit together	Aims at one atomic commit across both resources

Every option trades atomicity for operations, meaning or blocking

Solution	Gains	Costs
Transactional outbox	The event commits or rolls back with the order The event is a deliberate business message, not a table row	A relay to run, monitor and keep running Delivery is at least once, so consumers must tolerate duplicates
Change data capture (CDC)	Captures every committed write without changing the order code Also works for tables the code does not own	Events mirror table changes, not business meaning A connector and log retention to operate; schema changes reach consumers
Two-phase commit (2PC)	One coordinated commit across database and broker	Blocking: a stuck participant holds locks until the coordinator recovers Both sides must support the protocol, which many brokers do not

Smallest safe move

One outbox table beside the order insert

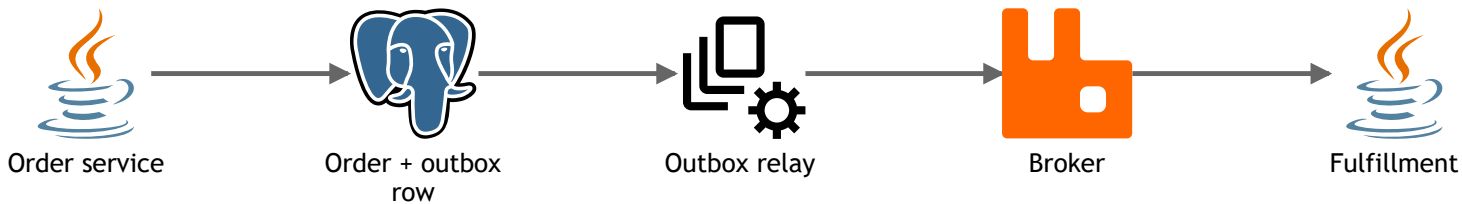
- **Scope:** the order-placed event only; other events publish as today
- **Change:** write the event to an `outbox` table in the order's transaction; a relay publishes unsent rows and marks them sent
- **Safety net:** publish and mark-sent are still two writes, so a crash means a duplicate, never a loss; a stable `eventId` lets consumers deduplicate
- **Escape route:** a feature flag restores publishing after commit; the outbox table can stay
- **Deferred:** CDC, until consumers need raw table changes; two-phase commit is not planned, because it blocks

```
BEGIN;
INSERT INTO orders (id, status) VALUES (:id, 'PLACED');
INSERT INTO outbox (event_id, type, payload) VALUES (:eid, 'OrderPlaced', :json);
COMMIT;
```

Verification: kill the process at every step

Check	Pass condition
Crash points	Process killed before commit, between commit and publish, and after publish: every committed order ends with its event delivered
Lost events	0 committed orders without a delivered event after recovery
Duplicates	A duplicate delivery changes nothing in fulfillment
Publish delay	Event delay p95 stays inside the agreed budget; the relay’s polling interval counts toward it
Outbox backlog	Unsent rows stay near zero; alert on the age of the oldest unsent row
Fitness function	A crash-injection test on the order flow runs in CI and fails on any committed order without an event

Solution architecture



Same service, database, broker and consumer; the direct publish is gone, and a relay now publishes what the order transaction recorded.

Glossary

Used in this step

Term	Definition
Dual write	Attempt to update two independent stores from one business operation
Transactional outbox	Atomically records business changes and outgoing events in the same database transaction
Outbox table	Table in the business database that holds events waiting to be published
Outbox relay	Process that reads unsent outbox rows, publishes them and marks them sent
Change data capture (CDC)	Detects and propagates changes made to database records
Database transactions	Atomic execution boundary for related data modifications
Two-phase commit (2PC)	Protocol in which a coordinator asks all participants to prepare, then to commit together
At-least-once delivery	Delivery semantics allowing duplicate deliveries in exchange for retries
Idempotent consumer	Consumer that gives the same result when it receives the same event twice

Related concepts

Term	Definition
WAL	Append transaction log records before database pages are modified
Idempotency key	Stable identifier recognizing multiple attempts at the same logical command
Event streaming platform	Stores ordered event streams for independent consumers
Exactly-once effect	Application-visible effect occurs once despite repeated attempts under defined guarantees
Dead letter queue (DLQ)	Stores messages that could not be processed successfully
Data reconciliation	Compare systems using keys, checksums and domain invariants to find divergence

A25 · Messaging and distributed workflows

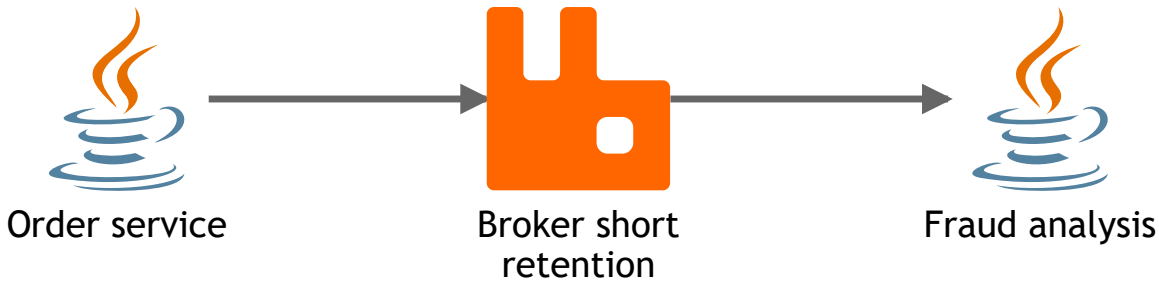
Replay historical business events

Fraud analysis must replay the prior 30 days of events, but the broker keeps them only briefly.

The broker forgets events soon after delivery

Component	What it is	What it does
Order service	Java code that publishes business events	Publishes order and payment events to the broker as they happen
Message broker	Broker configured with short retention	Delivers each event to its consumers, then discards it
Fraud analysis	Consumer that scores customer behavior	Sees only events published while it is subscribed
Short retention	Policy that deletes delivered or expired messages	Leaves no history to read back
Order database	PostgreSQL holding current order state	Stores the latest state, not the sequence of events that produced it

Current architecture



The broker cannot go back in time

Analysts want a new fraud rule applied to the previous 30 days of orders and payments. The broker discarded those events after delivery, so only events from now on are visible.

30 days

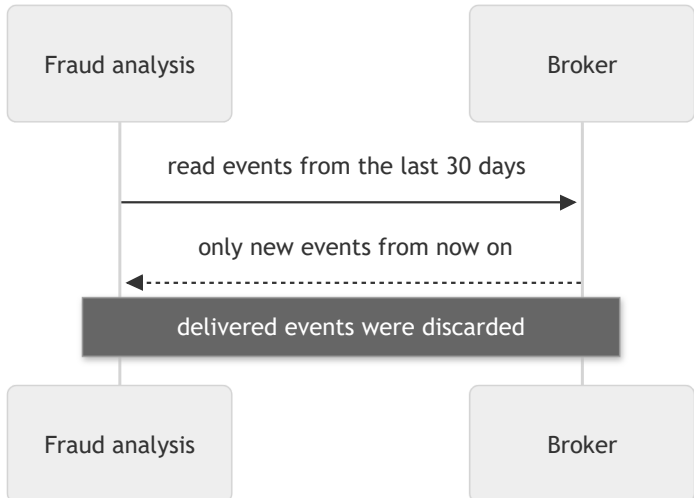
of business events the fraud analysis must replay

Short retention

delivered events are gone from the broker

Rebuild to a checkpoint

target: a projection rebuilt from history matches the live one



Three ways to keep history readable

Solution	What it is	What it does
Event streaming	A platform that keeps ordered event streams as a retained log	Lets consumers read from any retained position, so a replay is a read from an earlier offset
Event sourcing	Persisting state changes as an append-only sequence of events	Makes the event history the record of state, so any past state can be rebuilt
Object storage archive	Events copied into files in a bucket	Keeps history cheaply for batch reads outside the live flow

History costs storage, schema discipline or retrieval time

Solution	Gains	Costs
Event streaming	Durable, ordered log with replay from any retained offset New consumers can join late and read history	Storage grows with event volume times retention A cluster, partitions and retention to operate
Event sourcing	Complete state history and a natural audit trail Any projection can be rebuilt from the events	Event schema evolution: old events must stay readable for years The write model of a whole domain changes
Object storage archive	Low cost per GB and practically unlimited capacity Does not touch the live broker	Slow retrieval: a replay is a batch load A reader is needed to turn files back into a stream

Deciding signal: how often replays run, how quickly history must return, events per day multiplied by the retention window, and whether state itself must derive from events.

Smallest safe move

A 30-day log fed by one more subscriber

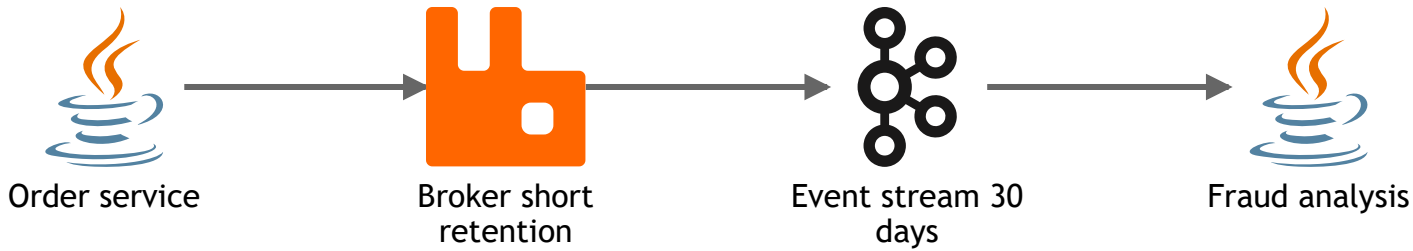
- **Scope:** order and payment events only; existing queues and their consumers stay unchanged
- **Change:** one more subscriber appends each event to a log-based stream with 30-day retention; fraud analysis reads it from a stored offset
- **Safety net:** retention covers the replay window with a margin; events carry a stable id and a schema version, so replays tolerate duplicates and older formats
- **Escape route:** fraud analysis returns to the live broker; the stream is derived data and can be deleted
- **Deferred:** an object storage archive, until history beyond 30 days is needed cheaply; event sourcing, until state itself must derive from events

```
retention.ms=2592000000
# 30 days in milliseconds: the replay window, to be widened by a margin
```

Verification: rebuild a projection from 30 days of history

Check	Pass condition
Completeness	Per-day event counts in the stream match the order and payment records for the full 30 days
Rebuild	A projection rebuilt from the oldest retained offset equals the live projection at a known checkpoint
Replay duration	A full 30-day replay finishes inside the agreed time without slowing live consumers
Live consumers	Latency and error rate of existing queue consumers stay at baseline
Old formats	Events of the oldest schema version in the window still replay correctly
Storage cost	Stream storage for the window is recorded against the agreed budget
Fitness function	A scheduled replay of a sample window fails on count or checksum divergence; alert when the oldest retained event is younger than the window

Solution architecture



Same producer and broker; one new subscriber feeds a retained log, and fraud analysis replays from it instead of from the live broker.

Glossary

Used in this step

Term	Definition
Event streaming platform	Stores ordered event streams for independent consumers
Log retention	How long events stay readable in the log before they are deleted
Offset	Position of an event in the log; a consumer stores its own position
Event replay	Reconsume historical records to rebuild a projection
Materialized query projection	Read-optimized representation built from updates or events
Checkpoint	Known position or state used to compare a rebuilt projection with the live one
Event sourcing	Persists state changes as an append-only sequence of events
Object storage	Stores files and binary objects in scalable buckets
Cold storage	Cheap storage for rarely read data, with slower retrieval
Message broker	Routes and delivers messages between producers and consumers

Related concepts

Term	Definition
Consumer group	Cooperating consumers divide partitions of a stream
Transactional outbox	Atomically records business changes and outgoing events in the same database transaction
CQRS	Separates models for reading data and modifying state
Audit log	Recorded business/security action with actor, time and outcome
Immutable storage	Data retention mechanisms resisting unauthorized modifications

A26 · Messaging and distributed workflows

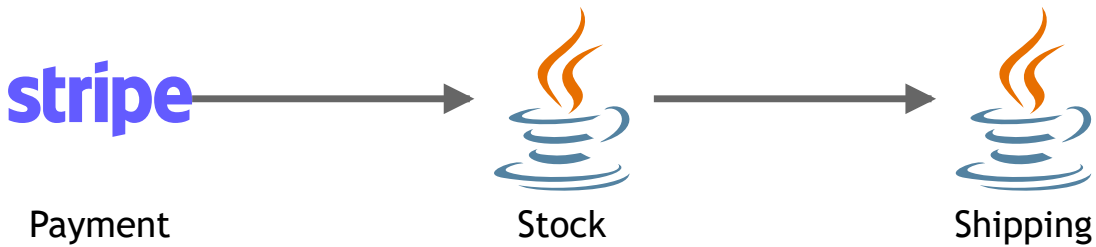
Coordinate long-running checkout

Payment success may precede a later inventory failure, and no single transaction spans the systems.

Checkout steps run in separate systems

Component	What it is	What it does
Payment	System that charges the customer’s card	Commits the charge in its own transaction
Stock	Inventory capability with its own data	Allocates the item in its own transaction
Shipping	System that creates the shipment	Starts the shipment once stock is allocated
Checkout flow	The sequence payment, stock, shipping for one order	Moves an order through the three systems one step at a time
Local transaction	Atomic unit inside one system	Covers a single system only; none spans all three

Current architecture



The customer pays, then the item is gone

Payment succeeds first. The stock step then fails because the last unit was taken. No transaction spans both systems, so the charge stays and no step ever refunds it.

Charged, then rejected

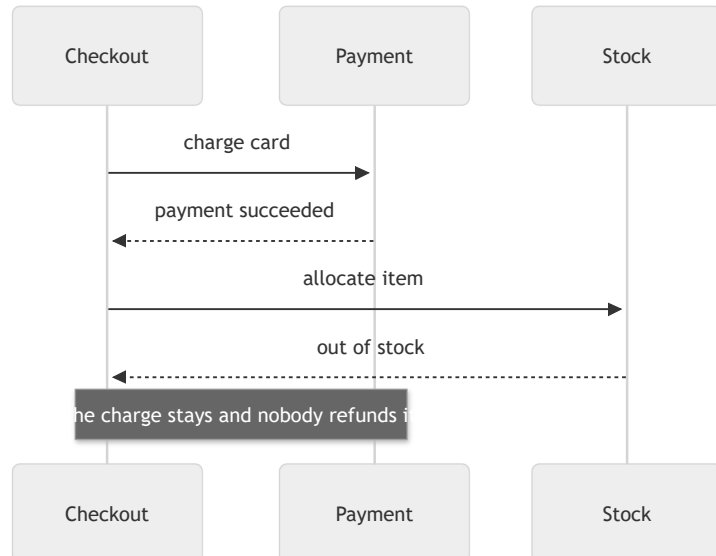
symptom: payment succeeded and the stock step failed afterwards

3 separate systems

payment, stock and shipping each commit on their own

0 paid orders left unresolved

target: every failed checkout ends fulfilled, or cancelled and refunded



Four ways to keep a multi-system checkout consistent

Solution	What it is	What it does
Saga orchestration	A central coordinator that runs the checkout steps and tracks their state	Calls each step in order and triggers compensations when a step fails
Saga choreography	Participants that react to each other's events, with no central coordinator	Each service does its local step, then emits the next event or a compensating event
Single transaction	One ACID transaction around all steps	Commits or rolls back payment, stock and shipping together, if they share one database
Reservations	Temporary claims on a resource, with an expiry	Hold stock while payment completes and release it on failure or timeout

Every option trades visibility, autonomy, atomicity or expiry

Solution	Gains	Costs
Saga orchestration	The whole flow is visible in one place: state, timeouts, audit Compensation logic lives in one component	A coordinator to build, persist and keep available The coordinator knows every participant
Saga choreography	Autonomous services and no central component A new reacting service joins without changing the others	The flow is implicit and found only by reading every service Hard to answer where an order is stuck
Single transaction	Real atomic rollback and the simplest reasoning	Possible only when all steps share one database, not with an external payment provider Locks held for the whole flow

Smallest safe move

One orchestrated saga for checkout

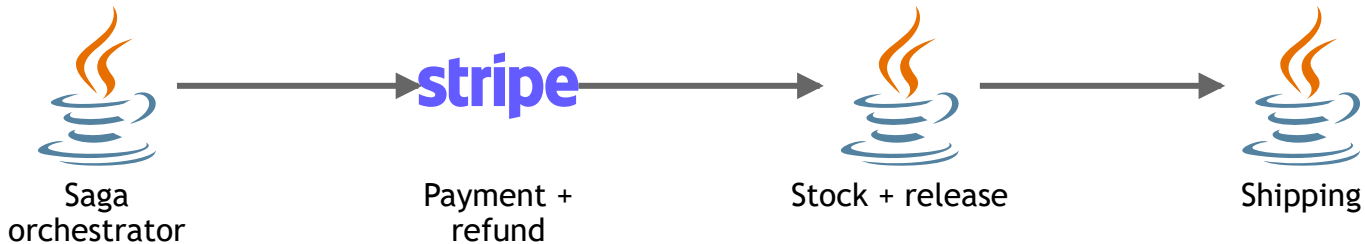
- **Scope:** the checkout flow only; the orchestrator can start inside the monolith, state in PostgreSQL
- **Change:** a persisted state machine runs charge, allocate stock and create shipment; on failure it compensates completed steps in reverse order
- **Safety net:** steps and compensations are idempotent and retried with a stable key; compensation is not atomic rollback, as the customer sees a charge, then a refund
- **Escape route:** a feature flag routes new checkouts through the previous flow; running sagas finish
- **Deferred:** choreography, until steps are loosely coupled reactions; reservations, until stock must be held before payment; single transaction, only with one shared database

```
charge payment    compensation: refund payment
allocate stock    compensation: release stock
create shipment   compensation: cancel shipment
```

Verification: inject each intermediate failure

Check	Pass condition
Stock fails after payment	Payment is refunded and the order ends cancelled with the refund recorded
Shipping fails after stock	Stock is released and payment refunded, or the shipment is retried by the agreed policy
Orchestrator crash	After restart each saga resumes from its last recorded state; no step has a visible effect twice
Duplicate command	Replaying any step or compensation leaves at most one charge and one refund
Final states	Only two outcomes exist: fulfilled, or cancelled and refunded; none stays paid without goods
Stuck sagas	None older than the agreed time in an intermediate state; alert on saga age
Fitness function	A failure-injection test at every step boundary runs in CI and fails on any final state outside the allowed set

Solution architecture



Same three systems, one new coordinator: arrows show step order, and the orchestrator runs the compensations in reverse when a later step fails.

Glossary

Used in this step

Term	Definition
Saga pattern	Coordinates distributed business transactions using local transactions and compensations
Saga orchestration	A central orchestrator tells each participant which local step to run next
Saga choreography	Participants react to each other's events and no one holds the whole flow
Orchestrator	Component that holds the flow state, calls the steps and triggers compensations
Compensating action	A new business action that semantically counters an earlier step, such as a refund
Single transaction	One database transaction: an atomic execution boundary for related data modifications
Reservation pattern	Temporarily claim business resource pending workflow completion
Business invariant	Rule that must hold at the end of every flow, such as no charge without goods
Idempotent operation	Operation that has the same effect when it is executed more than once

Related concepts

Term	Definition
Message broker	Routes and delivers messages between producers and consumers
Event bus	Infrastructure for distributing events between components
Transactional outbox	Atomically records business changes and outgoing events in the same database transaction
Idempotency key	Stable identifier recognizing multiple attempts at the same logical command
Eventual consistency	Participants reflect a change after a delay, not in the same transaction

A27 · Messaging and distributed workflows

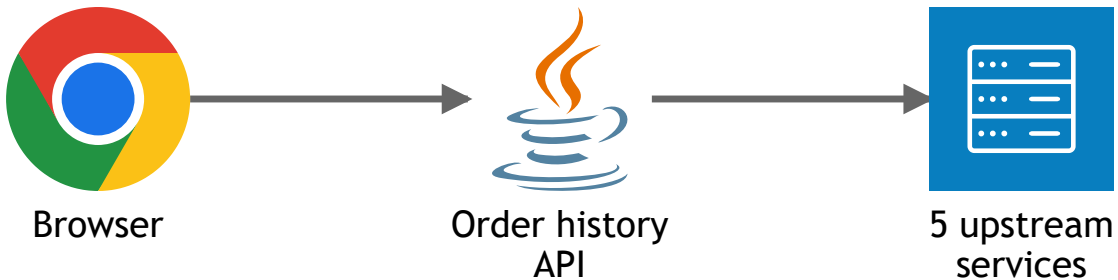
Eliminate query-time service fanout

Order history p95 exceeds 500 ms because every request makes five remote calls.

Order history is assembled from five services at request time

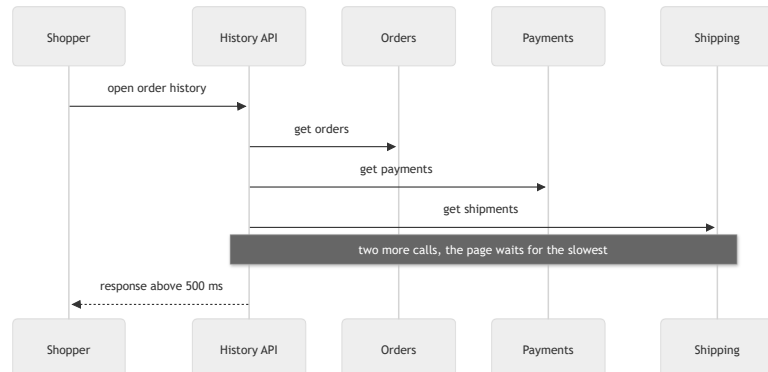
Component	What it is	What it does
Browser / SPA	Customer-facing web client	Opens the order history page
Order history API	Endpoint that builds the history page	Calls each upstream service on every request and joins the answers
Upstream services	Five services that each own part of an order, for example orders, payments and shipping	Answer one remote call each per request
Remote call	Network request from one service to another	Adds network latency and a failure point to every page view
Request-time aggregation	Joining answers from several services while the user waits	Returns fresh data, but the page is as slow as its slowest call

Current architecture



Five calls, one slow answer

Opening order history fans out to five services and joins the results in the API. The slowest dependency sets the pace, and one failing service degrades the whole page.



p95 above 500 ms
order history latency, caused by remote calls

5 remote calls
made for every history request

Below 500 ms with 1 dependency blocked
target: the history SLO holds even when an upstream service is down

Three ways to answer order history without the wait

Solution	What it is	What it does
CQRS projection	A separate read model, updated from events and shaped for one query	Serves order history with one local read instead of five remote calls
Materialized view	A precomputed query result stored as a table and refreshed on demand or on a schedule	Serves the stored result and recomputes it only at refresh time
Request-time aggregation	Calling every owning service live and joining the answers in the API	Returns current data from each owner on every request

Speed comes from staleness, freshness from latency and failure

Solution	Gains	Costs
CQRS projection	One local read per request, so latency no longer depends on five services Reads keep working when a producing service is down	Stale: the view lags the events and is eventually consistent A projection to build, keep correct and rebuild
Materialized view	Joins and aggregates are paid once, so reads are cheap Set-based refresh with no event handling	Refresh lag: changes are invisible until the next refresh Data from separately owned services must first be copied into one store
Request-time aggregation	Always fresh: each owner answers with its current state No new store or pipeline	Latency is that of the slowest call and one failing service breaks the page Extra load on every owning service per page view

Smallest safe move

A read model for the history list

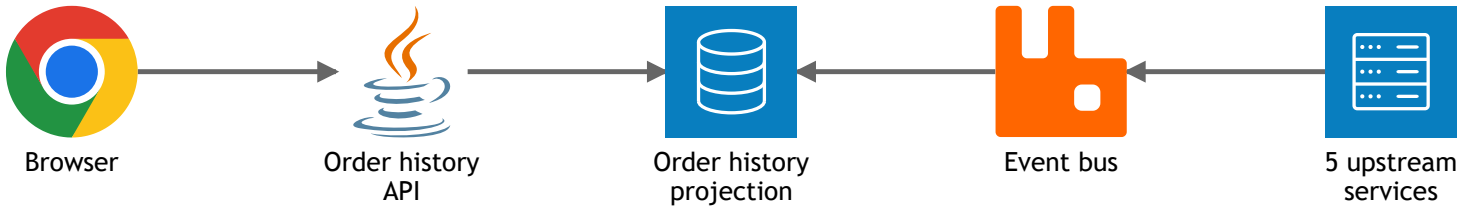
- **Scope:** the history list only; order detail and anything that must be fresh keep calling the owning services
- **Change:** an `order_history` table updated from order, payment and shipping events; the API reads it with one query
- **Safety net:** event ids and versions make updates idempotent; a shadow comparison with live aggregation tracks mismatches and lag, and above the staleness budget the API falls back to live calls
- **Escape route:** a feature flag returns the list to live aggregation; the table is derived and rebuildable
- **Deferred:** materialized view, until all five data sets share one database and a refresh schedule is enough

```
CREATE TABLE order_history (
  order_id bigint PRIMARY KEY, customer_id bigint, status text,
  payment_status text, shipment_status text, event_version bigint
);
CREATE INDEX ON order_history (customer_id);
```

Verification: history holds its SLO with a dependency blocked

Check	Pass condition
Latency	Order history p95 below 500 ms under replayed production traffic
Dependency blocked	With one upstream service blocked, history still answers inside the SLO
Remote calls	One local read per request on the list path, instead of five remote calls
Staleness	Projection lag p95 stays inside the agreed staleness budget
New order	A just-placed order appears in the list within the budget, or the page covers the gap
Correctness	Shadow comparison mismatches are explained and fall to 0 once lag has passed
Rebuild	Replaying events into an empty table produces the same rows

Solution architecture



Same browser and API; the services now publish events into a read model, and the API reads that model instead of calling five services.

Glossary

Used in this step

Term	Definition
CQRS	Separates models for reading data and modifying state
Materialized query projection	Read-optimized representation built from updates or events
Materialized view	Precomputed query result stored for efficient reads
Request-time aggregation	Assembling an answer from several services while the caller waits
Service fan-out	One request that triggers several remote calls
Eventual consistency	The read model reflects a change after a delay, not in the same transaction
Projection lag	Time between a change and its appearance in the read model
Staleness budget	Longest data age the business accepts for a given view
Event bus	Infrastructure for distributing events between components
Shadow comparison	Running the new path beside the old one and comparing results without serving them

Related concepts

Term	Definition
Event replay	Reconsume historical records to rebuild a projection
Transactional outbox	Atomically records business changes and outgoing events in the same database transaction
Backend for frontend (BFF)	API layer tailored to a specific frontend or client
GraphQL	Typed query interface allowing clients to request selected fields
Circuit breaker	Stops requests to failing dependencies temporarily
Message queue	Buffers messages for asynchronous processing by consumers

A28 · Messaging and distributed workflows

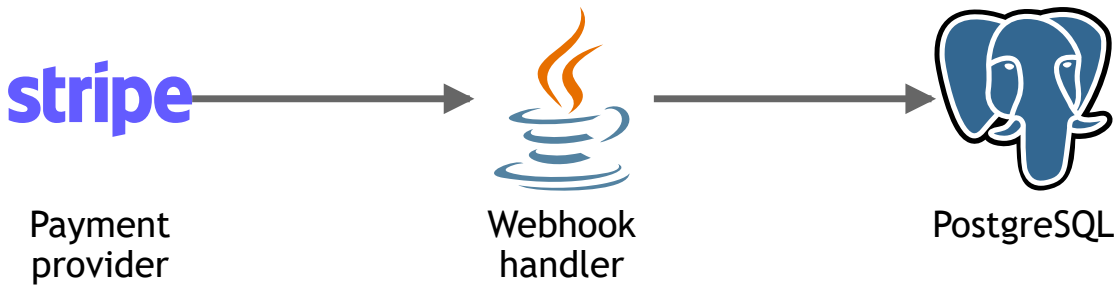
Make webhook retries safe

A payment callback may be delivered five times, yet each payment must cause one charge and one fulfillment effect.

Every delivery of the callback is treated as a new payment

Component	What it is	What it does
Payment provider	External system that charges customers	Sends a webhook on payment success and resends it until it sees a success reply
Webhook handler	HTTP endpoint and payment logic in the monolith	Records the charge and releases the order for fulfillment on every delivery
PostgreSQL	Relational transactional database	Stores charges and order state; has no rule against recording a payment twice
Delivery attempt	One send of the callback by the provider	May repeat the same payment up to five times
Success reply	Response, typically HTTP 2xx, that ends the provider’s retries	Is missing on timeouts and errors, which triggers a retry

Current architecture



One payment, five charges

The provider resends a callback when it gets no timely success reply. Deliveries can overlap, and the handler treats each one as new, so one payment is recorded and fulfilled several times.

Up to 5 deliveries

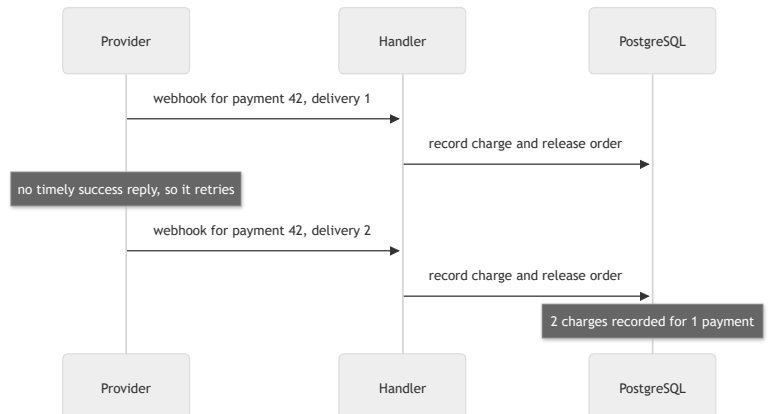
one callback may reach the handler five times

100 concurrent duplicates

the load the fix is tested with: the same webhook delivered at once

1 charge, 1 fulfillment

target: one business effect per payment



Three ways to stop a repeated callback from repeating its effect

Solution	What it is	What it does
Idempotency key + unique constraint	A stable identifier of the logical operation, such as the provider's event id, stored under a unique constraint	Lets the first delivery do the work and recognizes later deliveries as duplicates
Single writer	One process or partition that is the only writer for a given payment	Serializes handling, so deliveries for the same payment never run in parallel
Distributed lock	A lock shared across instances, usually held with an expiring lease	Lets one handler at a time work on a payment

Every option trades retention, throughput or lease risk for safety

Solution	Gains	Costs
Idempotency key + unique constraint	Safe under concurrency: the database rejects the second insert atomically One table, no extra infrastructure	Keys must be kept as long as duplicates can arrive, then expired Key and effect must commit in one transaction
Single writer	No concurrent duplicates by construction Simple reasoning about order	Throughput is capped per writer or partition: a bottleneck Does not by itself recognize a redelivery after the first one completed
Distributed lock	Coordinates handlers across instances without routing Familiar mutual exclusion	The lease can expire while the holder still works, so two holders may run A lock service becomes a new dependency; no deduplication after release

Smallest safe move

A unique key on the provider’s event id

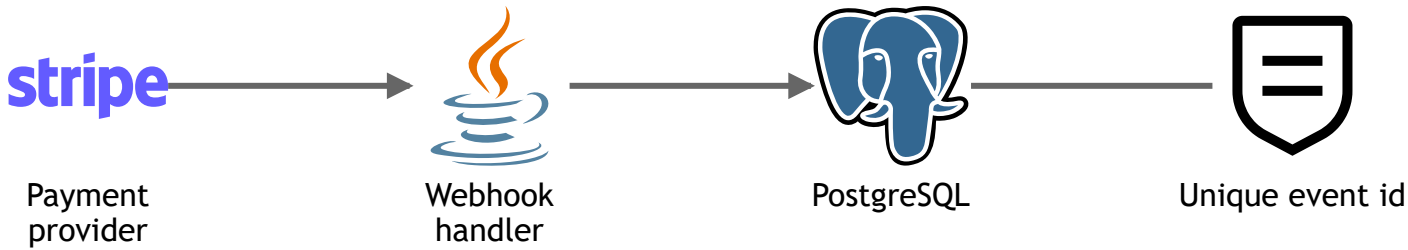
- **Scope:** the payment webhook endpoint only
- **Change:** insert the provider’s event id under a unique constraint in the same transaction as the charge and order release; a duplicate gets a success reply and no effect
- **Safety net:** the key identifies the payment event, not the delivery attempt; side effects outside the database are driven from rows committed in that transaction
- **Escape route:** a feature flag returns to the previous handler; the key table can stay
- **Deferred:** single writer, until throughput needs partitioning by payment anyway; distributed lock is not planned, since a lease can expire while its holder still works

```
CREATE TABLE processed_webhook (
  event_id text PRIMARY KEY, received_at timestamptz NOT NULL DEFAULT now()
);
INSERT INTO processed_webhook (event_id) VALUES (:id) ON CONFLICT DO NOTHING;
```

Verification: the same webhook 100 times at once

Check	Pass condition
Concurrent duplicates	The same webhook delivered 100 times concurrently leaves exactly 1 charge record and 1 fulfillment effect
Late redelivery	A delivery after the first completed returns success and changes nothing
Distinct payments	Every different payment is processed; none is suppressed as a false duplicate
Crash mid-handling	A crash before commit leaves no key and no effect, so the retry completes it; after commit the retry is a no-op
Provider reply	Duplicates get a success status so retries stop; handler p95 stays inside the budget
Key retention	Keys outlive the provider’s documented retry window; an expiry job keeps the table size flat
Fitness function	A parallel-duplicate test runs in CI and fails on more than one effect; alert on spikes in the duplicate rate

Solution architecture



Same provider, handler and database; a unique constraint on the event id now lets exactly one delivery through.

Glossary

Used in this step

Term	Definition
Webhook	HTTP callback that one system sends to another when an event happens
At-least-once delivery	Delivery semantics allowing duplicate deliveries in exchange for retries
Idempotency key	Stable identifier recognizing multiple attempts at the same logical command
Unique constraint	Database rule that rejects a second row with the same key
Exactly-once effect	Application-visible effect occurs once despite repeated attempts under defined guarantees
Single writer	One process or partition is the only writer for a given record
Distributed lock	Mutual exclusion shared by several processes, usually held with an expiring lease
Lease	Time-limited right to hold a lock or role that expires unless renewed
Key retention	How long recorded keys are kept to recognize late duplicates

Related concepts

Term	Definition
Transactional outbox	Atomically records business changes and outgoing events in the same database transaction
Write fencing	Prevent stale writers from modifying state after ownership changes
Retry with backoff	Repeats failed operations with increasing delays
Circuit breaker	Stops requests to failing dependencies temporarily
Data reconciliation	Compare systems using keys, checksums and domain invariants to find divergence

Phase 4

Specialized data and database scale

Steps in this phase

Step	Forcing constraint
29 Scale read-intensive database	Optimized queries and cache cannot meet read p95
30 Address primary write saturation	Primary write log or CPU saturated; replicas do not remove write load
31 Adapt heterogeneous product schema	New categories require new sparse attributes weekly
32 Support relationship traversal	Frequent multi-hop graph traversals become costly
33 Store massive telemetry	100M time series points per day
34 Isolate analytical workloads	Month-end reports slow checkout
35 Retain raw behavioral evidence	Data science needs raw clickstream, logs and events

A29 · Specialized data and database scale

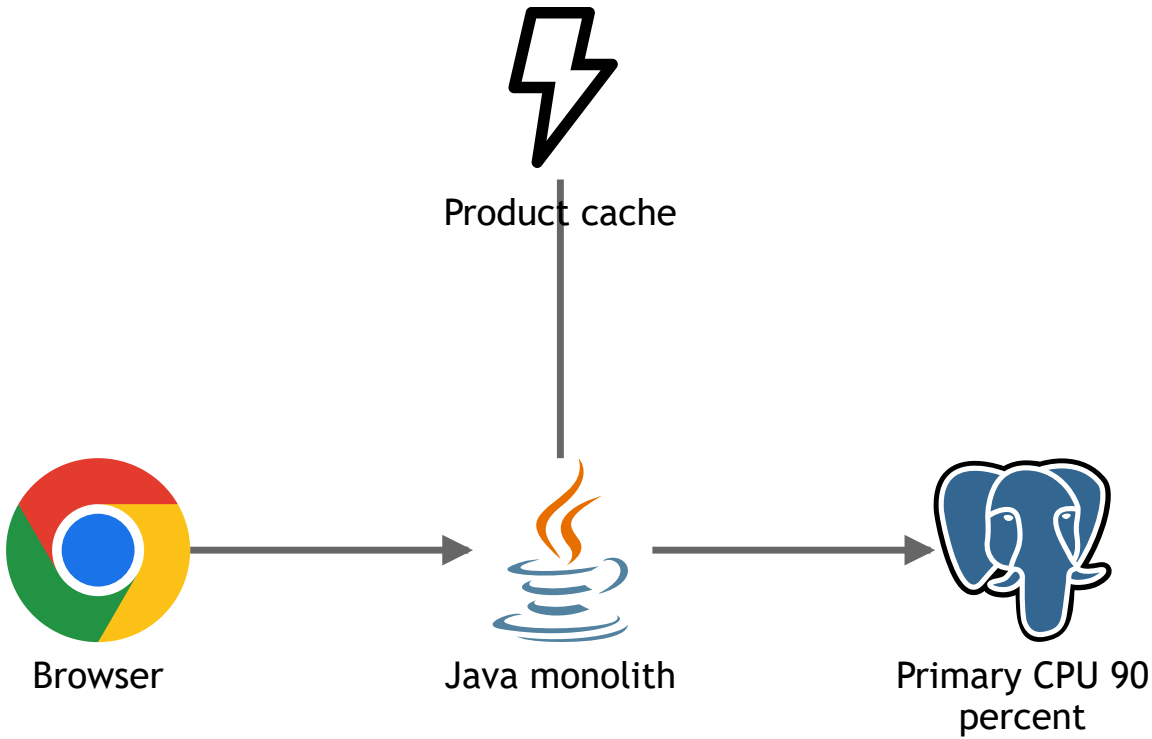
Scale read-intensive database

Indexes and a cache are in place, yet read p95 still misses its target while the primary runs at 90% CPU from reads.

One primary still answers every read the cache misses

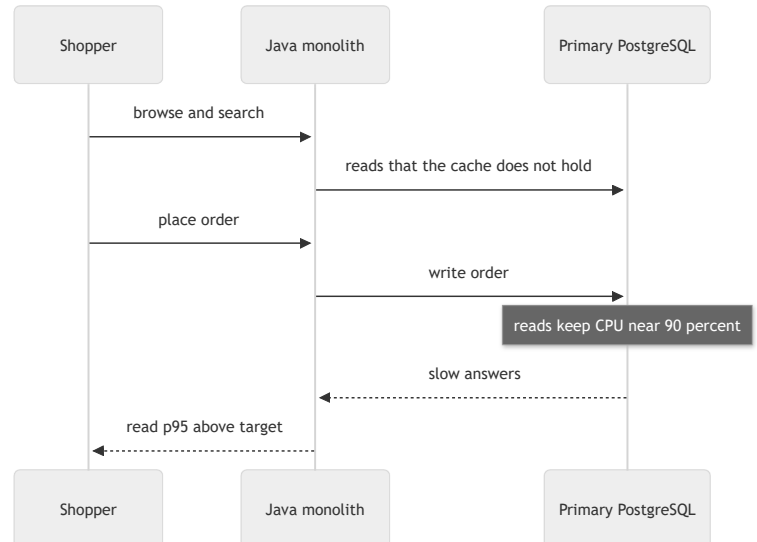
Component	What it is	What it does
Browser / SPA	Customer-facing web client	Sends browse, search and checkout requests
Java monolith	One deployable application	Sends every read and every write to the same database
Product cache	Cache-aside copy of hot catalog records	Absorbs repeated reads; misses still query PostgreSQL
Database indexes	Access paths on the slowest queries	Keep each query cheap, but do not reduce the number of queries
Primary PostgreSQL	The one writable database copy	Runs every write and every uncached read at about 90% CPU

Current architecture



Optimized reads still saturate the primary

Indexes make each query cheaper and the cache absorbs repeated reads, but the remaining reads still reach the primary. It stays near 90% CPU, read p95 misses its target and checkout writes compete for the same CPU.



90% primary CPU

caused by reads, with indexes and cache already in place

1200 QPS, 5% writes

read-heavy load; prices change every 30 seconds

p95 at or below 200 ms

target: the latency SLO, with replica lag inside the agreed staleness

Three ways to add read capacity

Solution	What it is	What it does
Read Replica	Database copy serving read operations to distribute query load	Receives the primary's changes asynchronously and answers lag-tolerant reads on another server
Distributed Cache	Shared cache accessible by multiple application instances, such as Redis	Serves repeated reads from memory so they never run SQL
Vertical Scaling	Add CPU or memory to one host	Gives the single primary more capacity with no architecture change

Read capacity is bought with staleness, invalidation or a ceiling

Solution	Gains	Costs
Read Replica	Scales reads: each replica adds read capacity Runs any SQL query, including ones the cache cannot hold	Replication lag: a read can miss a committed write Adds no write capacity; the application must route by freshness need
Distributed Cache	Avoids SQL: a hit uses no database CPU One shared copy for every application instance	Invalidation: entries can be stale until evicted Helps only repeated reads; an outage sends the full load to the primary
Vertical Scaling	Simple: no code, routing or consistency change No lag: one authoritative copy	Ceiling: one host has a physical and cost limit Resizing may need a restart, and saturation returns as load grows

Deciding signal: whether the bottleneck is read QPS, write rate or dataset size; cache hit rate and maximum acceptable staleness; headroom on the largest affordable host.

Smallest safe move

One asynchronous read replica for lag-tolerant queries

- **Scope:** browse, search and listing reads that tolerate lag; checkout, stock and a shopper's own just-written data stay on the primary
- **Change:** a routing rule sends read-only catalog queries to the replica; everything else goes to the primary
- **Safety net:** replica lag is monitored; above the staleness budget, reads fall back to the primary
- **Escape route:** a routing flag sends all reads back to the primary; the replica is a derived copy
- **Deferred:** distributed cache, until repeated reads dominate again; vertical scaling, a stopgap with a ceiling; sharding, until writes or data size saturate the primary

```
// routing rule: read-only and lag within budget goes to the replica
@Transactional(readOnly = true)
List<ProductView> browse(Filter filter) { ... }
```

Verification: read p95 and replica lag, before and after

Check	Pass condition
Read p95	At or below 200 ms under replayed production traffic
Primary CPU	Below the 90% baseline at the same 1200 QPS, with headroom for the write peak
Replica lag	Maximum lag under write bursts stays inside the staleness budget, for example a few seconds against prices that change every 30 seconds
Routing correctness	0 checkout, stock or read-after-write queries served by the replica
Replica loss	With the replica stopped, reads fall back to the primary and stay correct
Fitness function	Replay test fails the build when lag exceeds the budget or a critical query is routed to the replica

Rollback: the routing flag restores primary-only reads; the replica is a derived copy, so no state is reconciled. Promoting a replica after a primary failure is a state change: writes accepted since promotion need reconciliation.

Solution architecture

Same components, one new node: the replica receives the primary’s changes and serves the reads that tolerate lag.

Glossary

Used in this step

Term	Definition
Read replica	Database copy serving read operations to distribute query load
Primary	The one database copy that accepts writes and is the source of truth
Asynchronous replication	Acknowledge commit before all replicas confirm persistence
Replication lag	Delay between a commit on the primary and its visibility on the replica
Staleness budget	Maximum age of data a read may show before it counts as wrong
Read routing	Rule that sends each read to the replica or to the primary
Read-your-own-writes	A user sees their own committed change on the next read
Distributed cache	Shared cache accessible by multiple application instances
Cache invalidation	Removing or replacing cached copies after the source changes
Vertical scaling	Add CPU or memory to one host

Related concepts

Term	Definition
Database sharding	Partitions data across multiple database instances
Horizontal scaling	Distribute workload across more instances
Database index	Data structure supporting efficient access paths
Cache-aside pattern	Application explicitly reads from and populates a cache
Materialized view	Precomputed query result stored for efficient reads

A30 · Specialized data and database scale

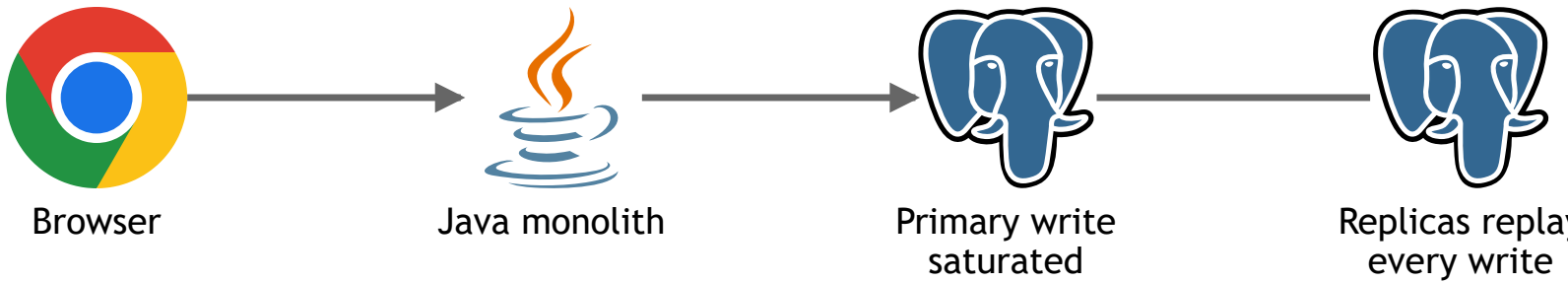
Address primary write saturation

The primary’s write log or CPU is saturated, and replicas do not remove write load.

One primary takes every write, and replicas only repeat them

Component	What it is	What it does
Java monolith	One deployable application	Writes rows to the primary one statement at a time
Primary PostgreSQL	The only writable database copy	Applies every insert and update and records it in the write-ahead log
Write-ahead log	Sequential record of changes, written before the data pages	Makes commits durable and feeds replication
Read replicas	Read-only copies kept current from the log	Serve reads and replay every write the primary makes
Indexes	Access paths on the product and order tables	Speed up reads; every insert and update also maintains them

Current architecture



More replicas add no write capacity

Write volume has grown until the primary's write log or CPU is the limit, not read activity. Replicas copy the primary's changes, so they take reads but add replay work, not write capacity.

Write log or CPU saturated

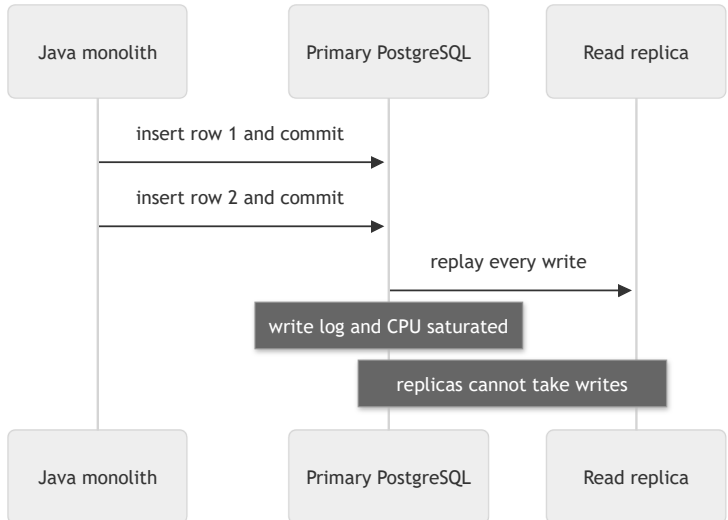
the primary is limited by writes, not reads

0 writes on replicas

every insert and update still runs on the primary

Capacity above peak writes

target: commit latency within the agreed limit at the peak write rate



Three ways to raise write throughput

Solution	What it is	What it does
Batching	Process groups of work together	Writes many rows in one transaction, so statement and commit overhead is paid once per batch
Data Partitioning	Divide a logical dataset into smaller sections, often within one database	Splits a large table into partitions in the same instance, so queries and maintenance touch one section
Database Sharding	Partitions data across multiple database instances	Gives each shard its own primary, write log and CPU, so writes run in parallel

Write capacity is bought with delay, same-engine limits or distributed transactions

Solution	Gains	Costs
Batching	Efficient: fewer commits and log flushes per row No new infrastructure, schema or engine	Latency: each row waits for its batch to fill or time out One bad row can fail a batch; unflushed rows are lost on a crash
Data Partitioning	Pruning: queries and maintenance touch one partition Cheap retention: drop an old partition instead of deleting rows	Same engine: one primary and one log, so no independent write capacity Queries without the partition key scan every partition
Database Sharding	Parallel writes: each shard has its own log and CPU Capacity grows by adding shards	Distributed transactions or sagas for writes that span shards Shard key choice, hot shards, resharding and cross-shard queries

Smallest safe move

Fewer, larger writes through a batch writer

- **Scope:** the one write stream that dominates log volume, found by profiling, for example row-by-row inserts; checkout and stock confirmation stay synchronous
- **Change:** a batch writer groups rows into one multi-row transaction, flushed at a row count or time limit, for example 100 rows or 50 ms
- **Safety net:** callers are acknowledged only after the commit; idempotency keys make retries safe; indexes and replicas are counted first, because they amplify writes
- **Escape route:** a flag restores direct writes once the buffer drains
- **Deferred:** partitioning, until table size or retention hurts; sharding, until profiled write load exceeds one primary even after batching

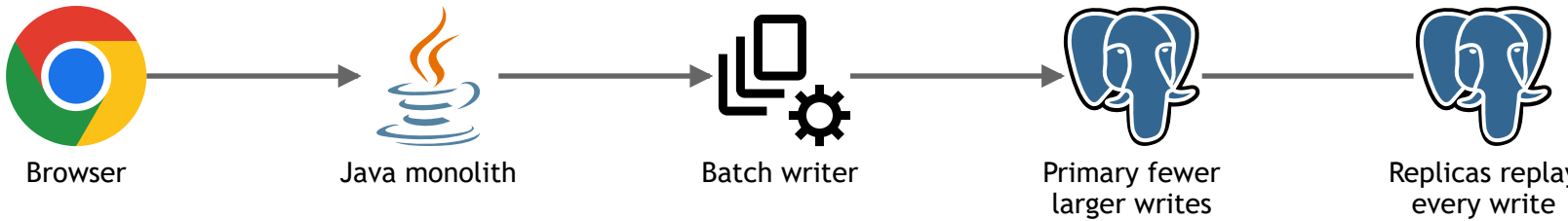
```
for (Row r : rows) { ps.setLong(1, r.id()); ps.addBatch(); }
ps.executeBatch();   // one round trip for the whole batch
conn.commit();       // one commit, one log flush
```

Verification: write profile before and after batching

Check	Pass condition
Write profile	Baseline names the dominant write path, with log volume, commits per second and CPU on the primary
Write throughput	Peak write rate sustained with log and CPU below saturation
Added delay	Per-row delay stays within the agreed batch window
Data integrity	Row counts and unique keys match a row-by-row run; a crash test loses no acknowledged row
Replica lag	Lag stays inside the staleness budget while commits arrive in larger bursts
Fitness function	Replay test fails the build when write p95, replica lag or an integrity check regresses

Rollback: the flag restores row-by-row writes after the buffer drains; committed rows are identical either way. Moving data to shards would be a state change that needs reconciliation and a copy back.

Solution architecture



Same components, one new node: the batch writer turns many small writes into fewer, larger transactions on the primary.

Glossary

Used in this step

Term	Definition
Write saturation	Primary limited by update and write-log throughput, not read activity
Write-ahead log	Sequential record of changes, written before data pages, for durability and replication
Write amplification	Extra work per logical write, such as index updates and replay on every replica
Read replica	Database copy serving read operations to distribute query load
Batch processing	Process groups of work together
Batch window	Row count or time limit at which a batch is flushed
Data partitioning	Divide a logical dataset into smaller sections, often within one database
Partition pruning	Skipping the partitions a query cannot match
Database sharding	Partitions data across multiple database instances
Shard key	Column that decides which shard stores a row
Cross-shard transaction	Change that spans shards and needs a distributed transaction or a saga

Related concepts

Term	Definition
Saga pattern	Coordinates distributed business transactions using local transactions and compensations
Idempotency key	Identifier that lets a repeated request take effect only once
Message queue	Buffers messages for asynchronous processing by consumers
Horizontal scaling	Distribute workload across more instances
Database index	Data structure supporting efficient access paths

A31 · Specialized data and database scale

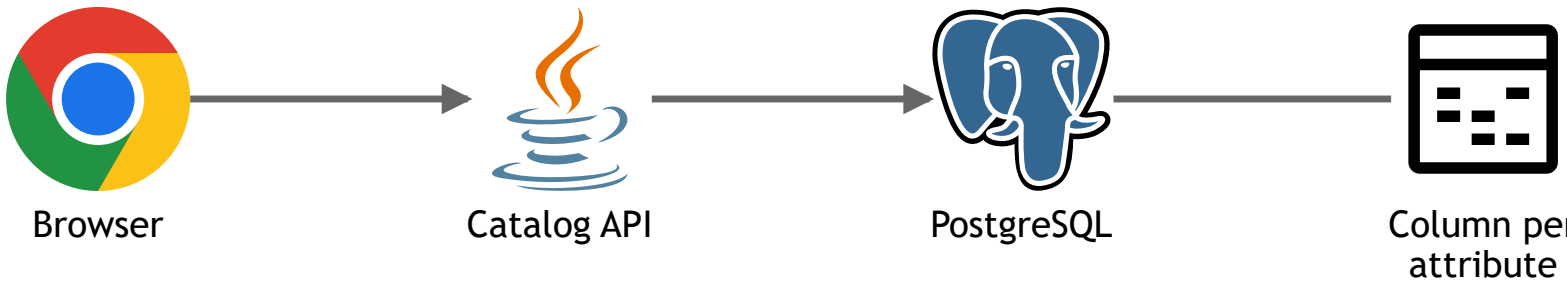
Adapt heterogeneous product schema

New categories bring new sparse attributes every week, and each one needs a schema change.

Every new category adds columns to the product table

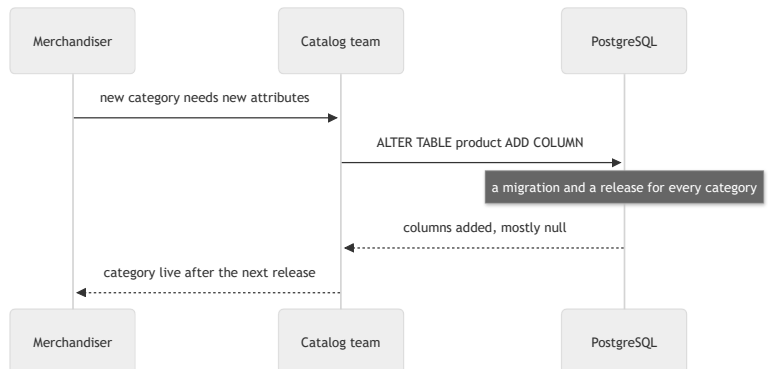
Component	What it is	What it does
Catalog API	Catalog module of the Java monolith	Reads and writes product rows through fixed columns
Product table	Relational table with one column per attribute	Holds mostly nulls, because each category fills only its own columns
Schema migration	Versioned script that changes table structure	Adds columns for each new category and ships with a release
PostgreSQL	Relational transactional database	Enforces types, constraints and transactions on every column
Attribute filters	Shopper-facing facets such as size or color	Query the columns directly, so every filter needs its own column

Current architecture



Each category waits for a schema release

Merchandisers launch a category with attributes no other category has. Every attribute needs a column, a migration and a release, so catalog growth moves at the pace of schema deployments.



Weekly schema change

new categories bring new sparse attributes every week

Mostly null rows

each product fills only the columns of its own category

0 migrations per category

target: a new category ships without a schema change and the invariants still hold

Three ways to hold attributes that differ by category

Solution	What it is	What it does
JSONB	PostgreSQL binary JSON format with indexing and query support	Stores a product's sparse attributes in one column next to its relational columns
Document Database	Stores flexible JSON-like documents, such as MongoDB	Holds each whole product as one document with its own attributes
Key-Value Store	Retrieves values using unique keys with minimal query overhead, such as Redis	Returns a product's attribute set by product id

Flexibility is bought with index design, consistency or query power

Solution	Gains	Costs
JSONB	Flexible attributes inside SQL: a new attribute needs no migration Same database, transactions and joins keep working	Indexing needs design: an index per query pattern Types and required keys are not enforced without constraints or validation
Document Database	Natural documents: a product is stored and read as one unit Schema flexibility is the default	Consistency spans two stores: product data, stock and orders no longer share a transaction Queries across products and relations are limited or move into the application
Key-Value Store	Fast, simple access by product id Minimal query overhead	Query limits: filtering by attribute needs a separate structure A second store to run, and consistency with the relational core

Smallest safe move

One JSONB column for one category

- **Scope:** the sparse attributes of one category; id, name, price, category and stock stay relational columns
- **Change:** add a nullable `attributes` JSONB column, backfill that category from its old columns, then switch its reads
- **Safety net:** old columns stay populated and results are compared side by side; per-category validation and an index on the filtered keys protect quality
- **Escape route:** reads switch back to the old columns; nothing is dropped until the category has run clean for an agreed period
- **Deferred:** document database and key-value store, until products no longer share transactions with stock and orders

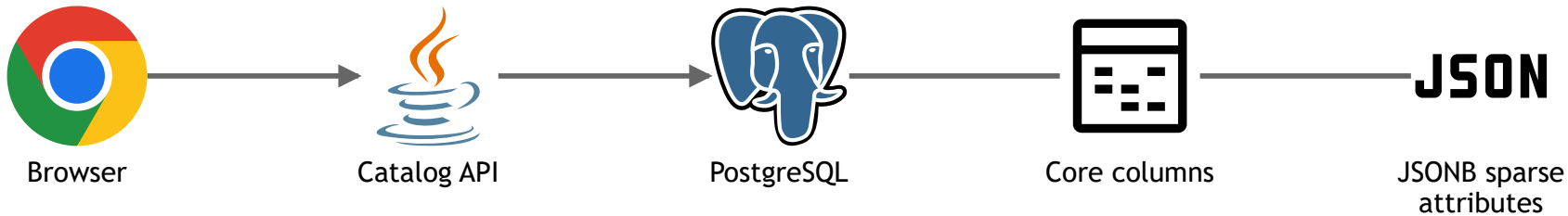
```
ALTER TABLE product ADD COLUMN attributes jsonb;
CREATE INDEX CONCURRENTLY idx_product_attributes
    ON product USING gin (attributes jsonb_path_ops);
-- filter: WHERE attributes @> '{"color": "red"}'
```

Verification: one category, old and new side by side

Check	Pass condition
Result parity	Attribute queries for the migrated category return the same product ids from JSONB as from the old columns
Query p95	Filtered attribute queries at or below the 200 ms search target, with the index used in the plan
Weekly change	A new attribute for the category ships with no schema migration and no schema release
Invariants	Price, stock and order transactions behave as before; the existing invariant tests pass unchanged
Data quality	Every stored attribute set passes the category's validation; 0 invalid rows after the backfill
Fitness function	Test on sample data fails the build when a result set differs or the plan stops using the index

Rollback: switching reads back to the old columns restores behavior. Attribute values written only to JSONB during the trial are state and must be copied back, so the old columns are kept in step until cutover.

Solution architecture



Same database and transactions, one new column: sparse attributes live in JSONB beside the relational core.

Glossary

Used in this step

Term	Definition
Sparse attribute	Property that only some products have, such as screen size
Schema migration	Versioned change to table structure, deployed with a release
Relational database (RDBMS)	Structured data stored in tables with SQL queries and transactions
Database transactions	Atomic execution boundary for related data modifications
JSONB	PostgreSQL binary JSON format with indexing and query support
GIN index	Inverted index that makes containment queries on JSONB efficient
Database index	Data structure supporting efficient access paths
CHECK constraint	Database rule that rejects rows whose values break a condition
Backfill	Copying existing data into a new structure before reads switch
Document database	Stores flexible JSON-like documents

Related concepts

Term	Definition
Entity-attribute-value	Relational layout that stores each attribute as a row of entity, attribute name and value
Eventual consistency	Copies converge after updates stop; reads may be stale meanwhile
Data normalization	Reduce redundancy using disciplined relation design
Data denormalization	Duplicate or reshape data to simplify high-frequency queries
Polyglot persistence	Using different database types for different data in one system

A32 · Specialized data and database scale

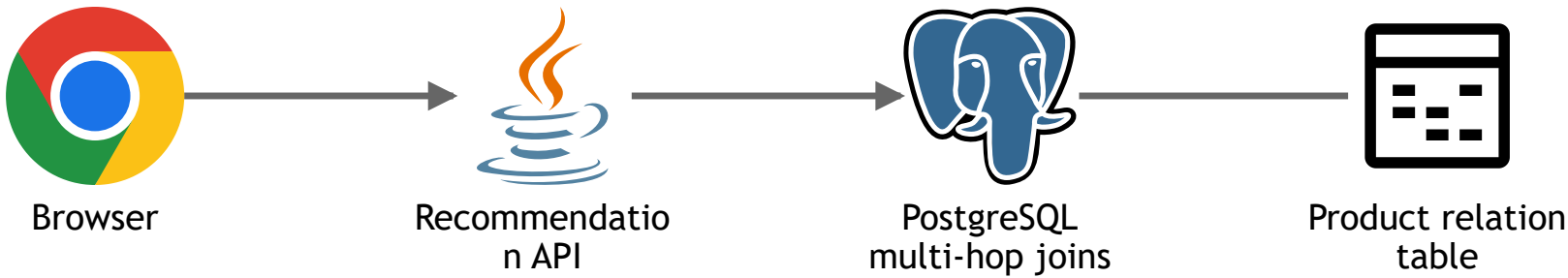
Support relationship traversal

Frequent multi-hop traversals of product relationships have become costly in SQL joins.

Recommendations are joins over a relation table

Component	What it is	What it does
Recommendation API	Module of the Java monolith	Builds the related-products list for a product page
Product relation table	Rows linking a product to a related product, such as bought together	Holds the edges that every traversal walks
Multi-hop join query	One SQL query with a join per hop	Walks 1, 2 or 4 hops by joining the relation table to itself
PostgreSQL	Relational transactional database	Stores the catalog and the relations; executes every traversal

Current architecture



Each extra hop multiplies the work

Related-product requests walk the relation table hop by hop, and every hop joins it again. Intermediate rows multiply with the fan-out, so deep traversals cost far more than shallow ones and the page waits.

1, 2 and 4 hops

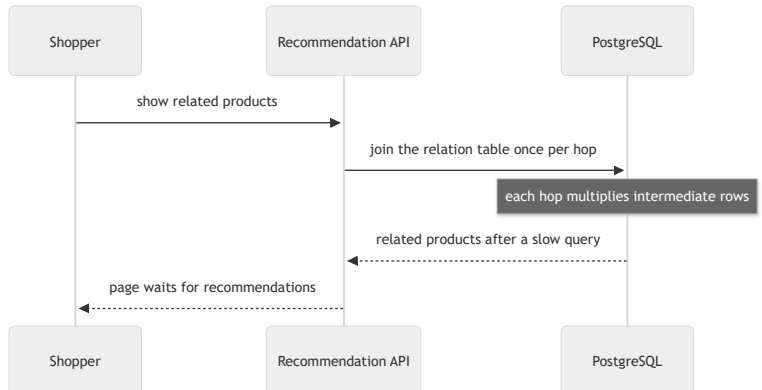
benchmark depths; cost grows with every hop

Frequent traversals

recommendation requests run multi-hop queries often

4-hop p95 within SLO

target: deep traversals meet the same read target as shallow ones



Three ways to answer multi-hop questions

Solution	What it is	What it does
Graph Database	Stores entities and relationships optimized for graph traversal, such as Neo4j	Follows stored relationships from node to node instead of joining tables
SQL recursive queries	Query that repeats a step over its own output, `WITH RECURSIVE` in PostgreSQL	Walks relationships of variable depth in one query inside the existing database
Precompute edges	Store results of expensive computations for later fast retrieval	Computes related products ahead of time, so a request is one lookup

Traversal speed is bought with a new store, unpredictable cost or stale results

Solution	Gains	Costs
Graph Database	Efficient traversal: cost follows the visited neighborhood, not table size Expressive queries for variable depth and paths	A new database to run, secure, back up and learn Loaded from PostgreSQL, so it lags the source of truth and ingest cost is paid on every change
SQL recursive queries	Reuse: same database and data, no new component One query for any depth, always current	Variable performance: depends on depth, fan-out and the planner Needs a depth limit; deep or hub-heavy traversals may still be slow
Precompute edges	Fast reads: one indexed lookup per request The expensive traversal runs off the request path	Stale: results lag changes until the next refresh Storage, refresh jobs and a rebuild when the logic changes

Smallest safe move

One depth-limited recursive query on an indexed relation table

- **Scope:** the one recommendation traversal that dominates cost; source tables and checkout are untouched
- **Change:** replace the join chain with one recursive query, a depth limit and an index on the relation table
- **Safety net:** duplicates are discarded at each step; the join version runs in shadow and results are compared before the switch
- **Escape route:** a flag returns to the join query; only an index was added and it can be dropped
- **Deferred:** precompute edges, when 4-hop p95 still misses the target and bounded staleness is acceptable; graph database, when traversal depth or variety rules out precomputing

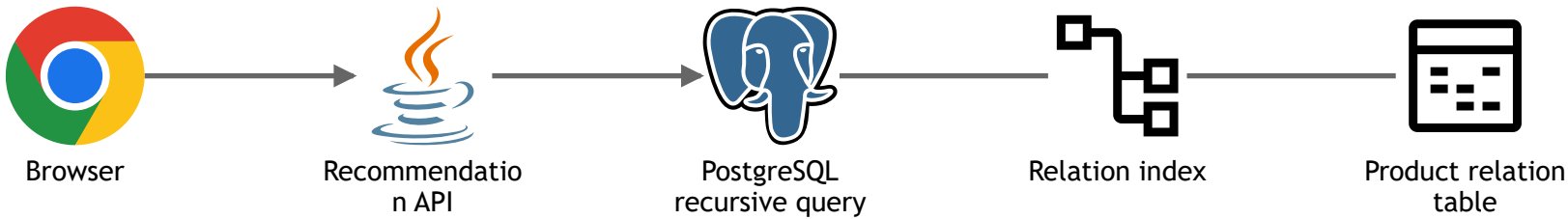
```
WITH RECURSIVE hop(id, depth) AS (
  SELECT related_id, 1 FROM product_edge WHERE product_id = :id
  UNION
  SELECT e.related_id, h.depth + 1 FROM hop h
  JOIN product_edge e ON e.product_id = h.id WHERE h.depth < 4)
SELECT DISTINCT id FROM hop;
```

Verification: 1, 2 and 4 hops against the join baseline

Check	Pass condition
Benchmark	p95 at 1, 2 and 4 hops measured for both versions at the same data volume and request rate; 4-hop within the read SLO
Result parity	Join and recursive versions return the same related products for sampled start products at each depth
Worst case	The product with the largest fan-out stays within the target, and cyclic data terminates
Ingest cost	Insert latency on the relation table, with the new index, stays within the agreed write budget
Database load	Primary CPU and checkout p95 are no worse at the same traffic
Fitness function	Benchmark on representative data fails the build when 4-hop p95 exceeds the target or the plan scans the table

Rollback: the flag returns to the join query (routing rollback); no data changed, so no state is reconciled. A precomputed table or graph database would add state that must be rebuilt and compared.

Solution architecture



Same database and data, one new access path: one depth-limited recursive query reads the relation index instead of a join per hop.

Glossary

Used in this step

Term	Definition
Multi-hop traversal	Query that follows relationships through several steps, such as product to product to product
Hop and fan-out	One relationship followed from a node; fan-out is the neighbors reached per hop
Edge table	Table whose rows are relationships between two entities
Graph database	Stores entities and relationships optimized for graph traversal
SQL recursive query	Query that repeats a step over its own output until a condition stops it
Depth limit	Maximum number of hops a recursive query may follow
Database index	Data structure supporting efficient access paths
Query precomputation	Store results of expensive computations for later fast retrieval
Staleness	Age of a result compared with the current data
Ingest cost	Work to load and keep a second store in step with its source
Source of truth	The store whose data wins when copies disagree

Related concepts

Term	Definition
Relational database (RDBMS)	Structured data stored in tables with SQL queries and transactions
Materialized view	Precomputed query result stored for efficient reads
Materialized query projection	Read-optimized representation built from updates or events
Eventual consistency	Copies converge after updates stop; reads may be stale meanwhile

A33 · Specialized data and database scale

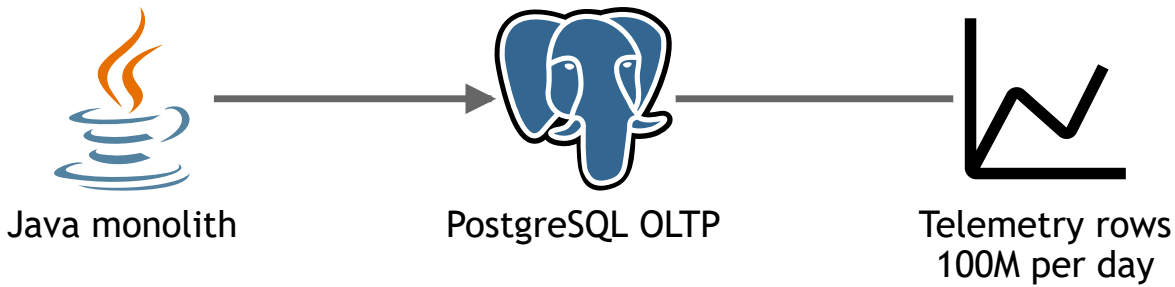
Store massive telemetry

100M time-series points per day, about 1,157 per second on average, land in the OLTP database that also runs checkout.

Application measurements are stored in the OLTP database

Component	What it is	What it does
Java monolith	One deployable application	Serves checkout and catalog, and emits measurements such as request timings and counters
Telemetry point	One timestamped value with a metric name and tags	Arrives all day: 100M per day, about 1,157 per second on average
PostgreSQL	Relational OLTP database	Stores orders and catalog, and every measurement as one more row
Measurements table	Append-only table ordered by metric and time	Grows with every point, at this rate by 36.5 billion rows a year
Dashboard query	Range query over recent points	Reads, for example, the last hour of one metric or a 30-day average

Current architecture



Every measurement becomes a row beside the orders

Measurements arrive all day as timestamped points. Stored next to orders and catalog, they grow the same tables, indexes, backups and write-ahead log, and retention deletes compete with checkout work.

36.5 billion rows a year

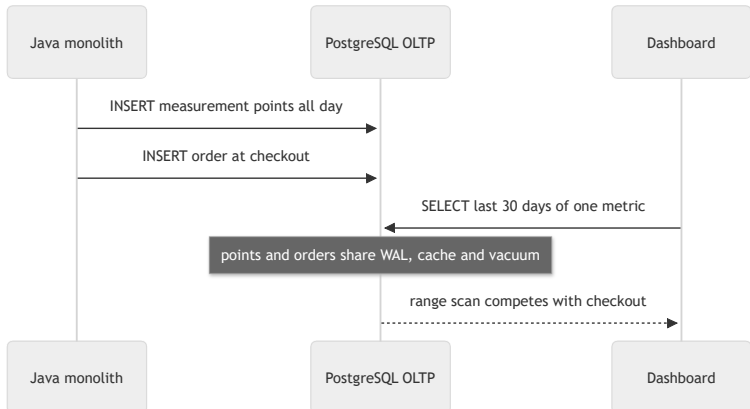
symptom: table growth if every point is one row

100M points a day

load: about 1,157 points per second on average, before peaks

Sample query within SLO

target: the range query meets its SLO at full ingest with retention applied



Three ways to store a continuous stream of points

Solution	What it is	What it does
Time-series database	Engine specialized for timestamped measurements, such as InfluxDB, TimescaleDB or the store of a metrics system	Compresses points by time, applies retention and downsampling, and answers range queries
Partitioned SQL	PostgreSQL table split by time range, for example one partition per day	Keeps the same engine and SQL, prunes partitions by time filter and drops old partitions whole
Object storage	Scalable bucket of files, for example columnar files per time window	Stores points cheaply for a long time; a separate query engine reads them

Every option trades reuse, query speed or cost

Solution	Gains	Costs
Time-series database	Compression and built-in retention Range queries and rollups designed for time	Specialized: one more engine to run, back up and learn Telemetry cannot join orders in one SQL transaction
Partitioned SQL	Reuse of PostgreSQL skills, tools and backups Dropping a partition is cheap compared with row-by-row delete	Tuning: partition size, indexes, autovacuum, partition creation No independent write capacity: points still share the primary
Object storage	Lowest cost per gigabyte and practically unlimited capacity Durable long-term archive	Slower querying: needs a query engine and a file layout Poor fit for low-latency recent-data dashboards

Deciding signal: points per second against the primary's write headroom, retention period, and access frequency with acceptable retrieval latency, which place data in hot, warm or cold storage.

Smallest safe move

Telemetry in a time-series database, orders untouched

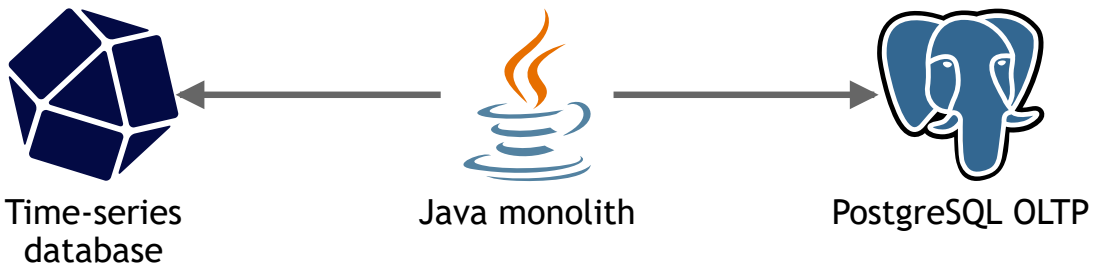
- **Scope:** application measurements only; orders and catalog stay in PostgreSQL
- **Change:** a batching writer sends points to a time-series database with a retention rule and a downsampled rollup
- **Safety net:** dual write while validating, so the PostgreSQL table stays current; telemetry failure never fails checkout
- **Escape route:** dashboards read the PostgreSQL table again
- **Deferred:** object storage as cold archive beyond the hot window; partitioned SQL as fallback if the primary absorbs the writes

```
retention:
  raw_points: 14d    # illustrative
  rollup_1m: 400d    # illustrative, one-minute averages kept longer
```

Verification: ingest, retention and a sample query

Check	Pass condition
Ingest	100M points per day accepted with no dropped points; a peak test at a multiple of the 1,157 per second average, for example 5x, also passes
Sample query	A fixed range query, for example the last hour of one metric, meets its latency SLO while ingest runs at full rate
Retention	Raw points older than the retention window disappear on schedule; the rollup stays queryable
Storage	Bytes per point and the projection for 36.5 billion points a year are recorded against the PostgreSQL baseline
Business invariants	Order write latency and checkout p95 do not regress; no business data exists only in the telemetry store
Fitness function	Scheduled load test fails on dropped points or a sample-query SLO breach; alert on writer buffer depth

Solution architecture



Same application and OLTP database, now holding orders and catalog only; the 100M points per day go to a separate time-series database.

Glossary

Used in this step

Term	Definition
Time-series database	Optimized for timestamped measurements and events
Ingest rate	Points written per second, a point being one timestamped measurement; 100M a day is about 1,157 per second
OLTP	Operational database workload dominated by frequent short transactions
WAL	Append transaction log records before database pages are modified
Data partitioning	Divide a logical dataset into smaller sections, often within one database
Partition pruning	Planner skips partitions that cannot match the time filter of a query
Retention policy	Rule that deletes or moves data after a set age
Downsampling	Replacing raw points with coarser aggregates, a rollup such as one-minute averages
Object storage	Stores files and binary objects in scalable buckets
Hot, warm and cold storage	Tiers set by access frequency and acceptable retrieval latency

Related concepts

Term	Definition
Metrics collection	Captures numerical measurements of system performance and behavior
Batch processing	Process groups of work together
Database sharding	Partitions data across multiple database instances
Data warehouse	Optimized for analytical queries across structured historical data
Audit log	Recorded business or security action with actor, time and outcome

A34 · Specialized data and database scale

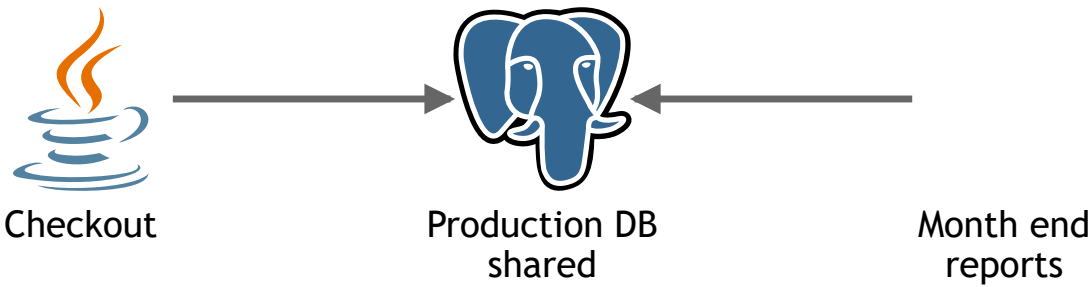
Isolate analytical workloads

Month-end reports run on the production database and slow checkout.

Business reports query the production database

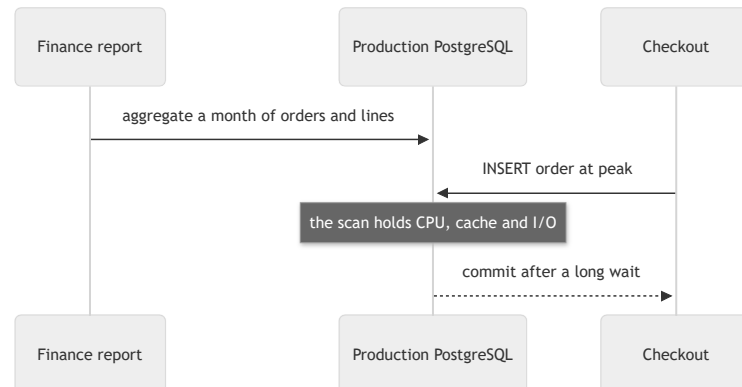
Component	What it is	What it does
Checkout	Capability of the Java monolith	Writes orders and reads stock, prices and carts in short transactions
Production database	One PostgreSQL instance serving the OLTP workload	Answers checkout and report queries from the same server
Month-end reports	BI dashboards and finance exports	Scan and aggregate months of orders, lines and payments
Report users	Finance and operations analysts	Open the heaviest reports together, at month end
Shared resources	CPU, buffer cache and disk I/O of one server	Go to whichever queries run, including long scans

Current architecture



One database, two very different workloads

Finance runs month-end reports directly on the production database. Large scans and aggregations occupy the CPU, cache and I/O that short checkout transactions also need, so checkout waits behind them.



Checkout p95 rises

symptom: checkout latency climbs while month-end reports run

Scans and aggregations

load: reports read months of orders, not single rows

Report and checkout together

target: checkout p95 stays within its target while a report runs at peak

Three ways to take reports off the checkout path

Solution	What it is	What it does
Data warehouse	Separate analytical store for structured historical data	Runs month-end aggregations on its own compute, loaded from production by ETL or CDC
Read replica	Copy of the production database that serves reads	Runs reports as ordinary SQL on the copy, away from the primary
Materialized view	Query result stored as a table and refreshed on a schedule	Serves a known report from the stored result without re-running the query

Isolation is paid in freshness, reuse or compute

Solution	Gains	Costs
Data warehouse	Isolates OLAP from OLTP: report load cannot reach checkout Columnar storage and long history suit large scans	Ingestion delay: reports show data as of the last load A pipeline and a model to build, monitor and pay for
Read replica	Reuses SQL, schema and BI connections unchanged Moves report reads off the primary	Shared replica compute: heavy reports compete with each other Row-oriented engine; lag and long-query conflicts with replication
Materialized view	Fast known reports: the aggregate is paid once No new component	Refresh lag and refresh cost on the primary Covers only predefined queries; a new question needs a new view

Deciding signal: runtime and volume of the heaviest report, fixed or ad hoc questions, the data age finance accepts, and how often and how long the report capacity sits idle.

Smallest safe move

A warehouse loaded incrementally, one report moved first

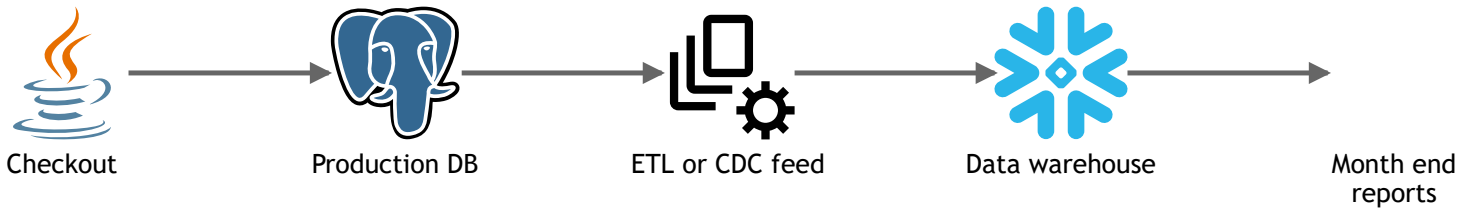
- **Scope:** month-end reports only; checkout keeps using production
- **Change:** load the tables one heavy report needs into a warehouse by incremental ETL or CDC, and run that report there
- **Safety net:** old and new report side by side with totals compared; extraction throttled so it does not become checkout load
- **Escape route:** point the BI connection back at production; the warehouse is derived data
- **Deferred:** read replica, if reports need fresher data than the load interval; materialized view, if few fixed reports remain

```
SELECT * FROM orders
WHERE updated_at > :last_loaded_at   -- high-water mark of the previous load
ORDER BY updated_at;
```

Verification: report and peak checkout together

Check	Pass condition
Isolation	With a month-end report and peak checkout running together, checkout p95 matches its baseline without reports
Report correctness	Warehouse totals equal production totals for the same closed period, by count and sum
Report runtime	The heaviest report finishes within the agreed window
Freshness	Ingestion delay is measured and below the accepted age, for example the previous night; alert when exceeded
Extraction cost	Primary CPU and I/O during extraction stay within the agreed budget at peak
Fitness function	Scheduled test of report plus checkout load fails on checkout p95 regression; reconciliation job fails on a total mismatch

Solution architecture



Checkout and the production database are unchanged; reports read a separate warehouse that a feed fills from production.

Glossary

Used in this step

Term	Definition
OLAP	Analytical workload dominated by aggregations and large scans
OLTP	Operational database workload dominated by frequent short transactions
Data warehouse	Optimized for analytical queries across structured historical data
ETL	Extract, transform and load data into a target system
Change data capture	Detects and propagates changes made to database records
Read replica	Database copy serving read operations to distribute query load
Materialized view	Precomputed query result stored for efficient reads
Incremental extract	Loads only rows changed since a stored high-water mark
Data reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Ingestion delay	Age of warehouse data relative to production

Related concepts

Term	Definition
Data lake	Stores large volumes of raw structured and unstructured data
Data lakehouse	Object-storage analytical architecture with table semantics
Batch processing	Process groups of work together
Asynchronous replication	Acknowledge commit before all replicas confirm persistence
Materialized query projection	Read-optimized representation built from updates or events

A35 · Specialized data and database scale

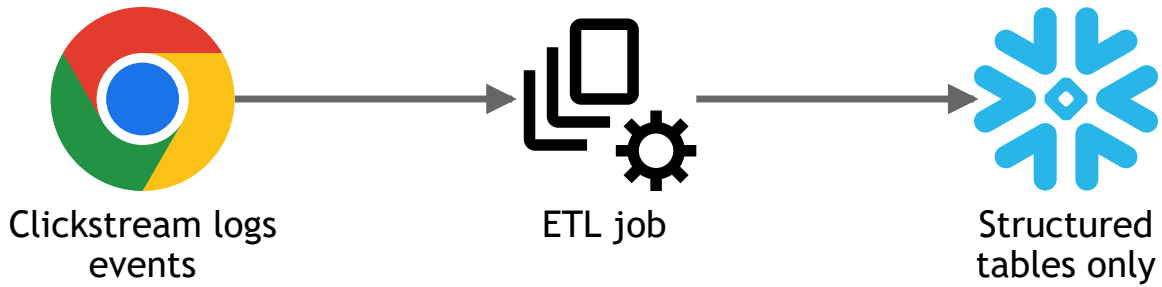
Retain raw behavioral evidence

Data science needs raw clickstream, logs and events, but the warehouse keeps only structured tables.

The warehouse keeps only structured tables

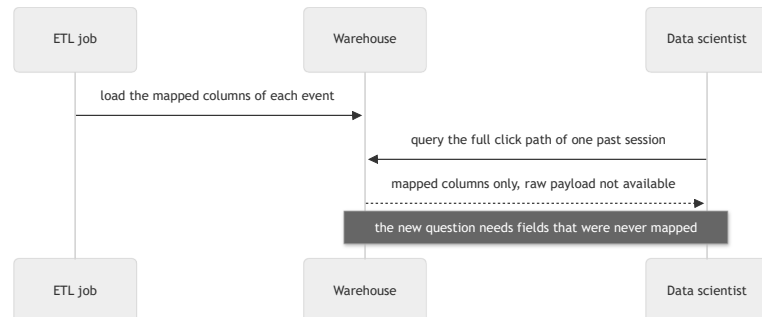
Component	What it is	What it does
Clickstream, logs and events	Behavioral signals from the browser, the application and its services	Record page views, clicks, searches, errors and domain events as semi-structured payloads
ETL job	Extract, transform and load pipeline	Maps selected fields to table columns and leaves the rest behind
Data warehouse	Analytical store of curated structured tables	Answers known questions with fast SQL
Table schema	Agreed column set per table	Fixes at design time which fields survive
Data scientists	Analysts who build models and explore behavior	Need the original events, not only the columns chosen in advance

Current architecture



A new question needs a field nobody kept

A data scientist asks about a past customer session. The ETL job kept only the columns defined earlier, so fields outside the table schema cannot be recovered and the original events are gone.



Unmapped fields lost

symptom: anything outside the table schema cannot be recovered

Clickstream, logs, events

load: semi-structured payloads from many sources that change with the product

One historical session

target: its raw events are recovered unchanged

Three ways to keep the original events

Solution	What it is	What it does
Data lake	Raw structured and unstructured data kept as files in object storage	Stores every event unchanged and applies a schema only when it is read
Warehouse ETL	Extract, transform and load into curated warehouse tables	Widens the column mapping so more fields reach structured tables
Long-retention stream	Event streaming platform that keeps events for months, not days	Lets consumers replay the ordered log from any earlier position

Keeping raw data trades curation for flexibility and cost

Solution	Gains	Costs
Data lake	Flexible: any later question can read the originals Cheap storage, with columnar formats and compression	Governance: ownership, catalog, quality, access and deletion Ungoverned raw data is hard to find and slow to query
Warehouse ETL	Curated, consistent tables with fast SQL Quality and privacy rules applied before data lands	Transforms early: unmapped fields stay lost, so each new question changes the pipeline History already dropped cannot be backfilled
Long-retention stream	Replay: consumers rebuild state from the ordered log One event feed serves real-time and later consumers	Costly: broker storage and compute sized for months of retention Replay is not ad hoc analytical scanning

Smallest safe move

A raw copy of every event in object storage, existing load untouched

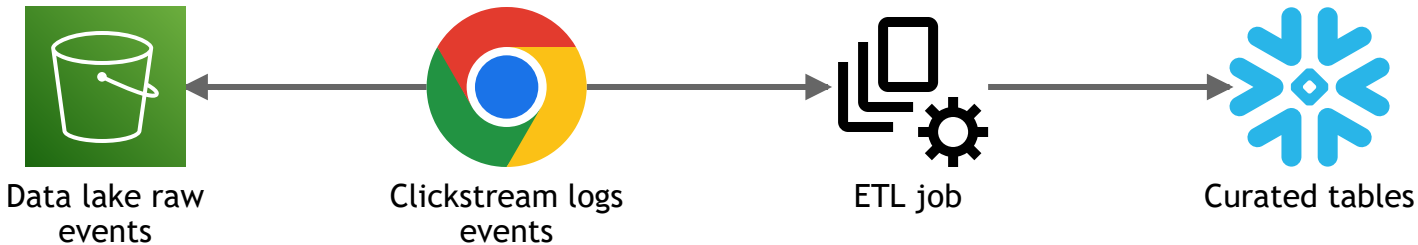
- **Scope:** clickstream, logs and domain events; warehouse tables and the ETL job stay as they are
- **Change:** the event collector also writes each raw event, unmodified, to object storage as compressed files partitioned by date and source
- **Safety net:** personal data minimized and access limited first; a failed object write never blocks a page request
- **Escape route:** stop the extra write; the warehouse path never changed
- **Deferred:** long-retention stream until low-latency replay is needed; lakehouse tables until updates or several writers need them

```
raw/clickstream/dt=2026-11-02/hour=14/part-0007.json.gz
raw/logs/dt=2026-11-02/hour=14/part-0003.json.gz
```

Verification: recover one historical session

Check	Pass condition
Session recovery	All raw events of one past customer session are found by session id and match what the collector received
Completeness	Event counts in the lake equal collector counts for sampled hours, with no missing events
Warehouse unchanged	Existing warehouse tables and report totals are identical before and after
Governance	Access is limited to named roles, personal-data rules are applied, objects past retention are removed on schedule
Cost	Storage growth per day is recorded and projected against the agreed budget
Fitness function	A daily job fetches one historical session and fails when its raw events cannot be recovered; counts are reconciled each day

Solution architecture



Same sources, ETL job and warehouse; one new node receives every raw event unchanged, for data science to read later.

Glossary

Used in this step

Term	Definition
Clickstream	Ordered record of user actions such as page views, clicks and searches
Raw event	An event stored exactly as emitted, before any transformation
Data lake	Stores large volumes of raw structured and unstructured data
Schema-on-read	Structure is applied when data is queried, not when it is stored
Object storage	Stores files and binary objects in scalable buckets
Data warehouse	Optimized for analytical queries across structured historical data
ETL	Extract, transform and load data into a target system
Data governance	Ownership, access control, quality and retention rules for stored data
Event streaming platform	Stores ordered event streams for independent consumers
Data lakehouse	Object-storage analytical architecture with table semantics

Related concepts

Term	Definition
Event sourcing	Persists state changes as an append-only sequence of events
Change data capture	Detects and propagates changes made to database records
OLAP	Analytical workload dominated by aggregations and large scans
Batch processing	Process groups of work together
Time-series database	Optimized for timestamped measurements and events

Phase 5

DDD boundaries and service separation

Steps in this phase

Step	Forcing constraint
36 Establish domain boundaries	Pricing changes cause unrelated regressions and release conflicts
37 Protect domain from ERP terminology	ERP calls shipments orders and corrupts model semantics
38 Scale one capability independently	Recommendations need 10x compute and separate team release
39 End cross-service schema coupling	Order migration changes billing and fulfillment schemas
40 Tailor APIs per consumer	Three clients need distinct payloads and releases
41 Find changing service instances	Hardcoded hostnames fail during rescheduling
42 Unify service policy at ingress	Security policy drift causes missed checks
43 Automate workload placement	Manual scheduling, restart and rollout are unreliable
44 Decouple frontend team release cadence	Shared frontend release blocks unrelated areas

A36 · DDD boundaries and service separation

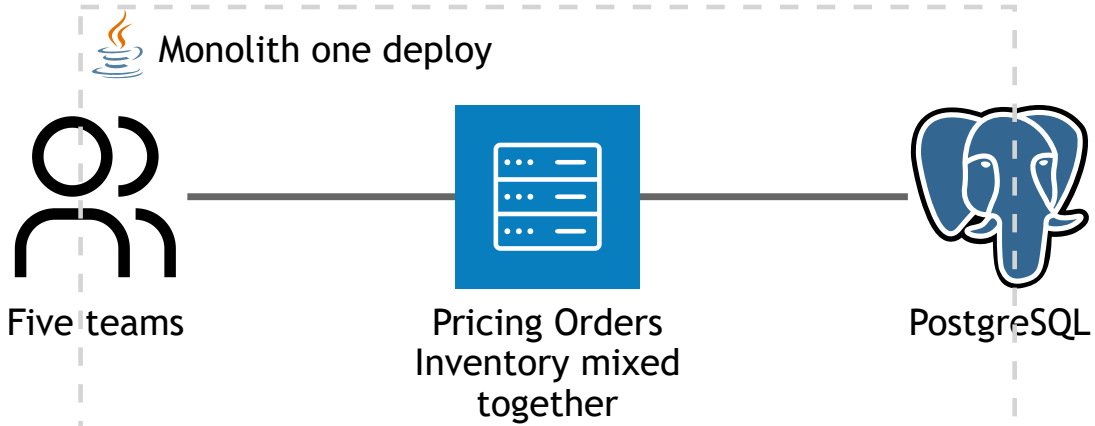
Establish domain boundaries

Pricing changes cause unrelated regressions and release conflicts for five teams in one deployment.

Five teams change one codebase and ship one deployment

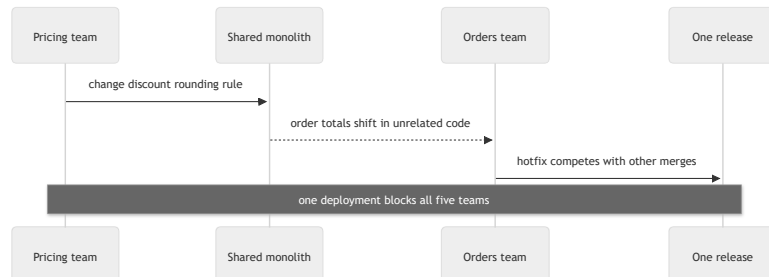
Component	What it is	What it does
Five teams	Feature teams working in one repository	Change the same codebase and merge into the same release
Java monolith	One deployable application	Hosts every business capability
Mixed business code	Pricing, orders and inventory logic without explicit boundaries	Calls and reads each other’s classes and tables directly
PostgreSQL	Relational transactional database	Stores all tables; any code can query any table
Release pipeline	One build, one test run, one deployment	Ships every team’s change together; a failure blocks all five

Current architecture



One pricing change breaks unrelated code

A pricing rule changes, and order totals shift in code the pricing team never touched. All five teams merge into one release, so a conflict or a rollback delays everyone.



Cross-module imports

counted from the code today; each one is a hidden dependency

5 teams, 1 deployment

every change ships in the same release

Pricing change in one module

target: no edits outside it and a falling coordinated change rate

Three ways to separate business areas

Solution	What it is	What it does
DDD + Modular Monolith	Business areas modeled as bounded contexts and built as modules in one deployment	Gives each context its own model, public interface and data ownership
Hexagonal Architecture	Business logic isolated from infrastructure through ports and adapters	Lets domain code change and be tested without touching databases or external systems
Microservices	Independently deployable services organized around business capabilities	Gives each capability its own release, runtime and data; calls cross the network

Stronger boundaries buy autonomy and cost coordination

Solution	Gains	Costs
DDD + Modular Monolith	Explicit model: names, rules and owners per context A wrong boundary is fixed by refactoring, with no network involved	Shared deployment: releases and runtime stay coupled Needs deep domain collaboration and discipline to keep modules apart
Hexagonal Architecture	Isolated logic: testable without infrastructure A database or an external system can be swapped behind a port	Adapters: more interfaces and mapping code Isolates technology, not teams, so release coupling remains
Microservices	Autonomy: each team deploys and scales on its own Runtime faults stay inside one service	Network complexity: latency, retries, partial failure, tracing A wrong boundary becomes chatty remote calls and distributed transactions

Deciding signal: coordinated change frequency, need for independent deployment, operations skill and invariants that span business areas.

Smallest safe move

Pricing as the first module inside the monolith

- **Scope:** pricing only, where the regressions start; one deployment and one database stay
- **Change:** model pricing as a bounded context behind one public interface; other code stops importing its classes and reading its tables
- **Safety net:** an architecture test, for example ArchUnit, fails the build on a forbidden dependency; aggregates keep each invariant inside one module
- **Escape route:** modules are packages in one codebase; reverting the refactor commits restores the old structure
- **Deferred:** hexagonal ports, until a module must swap infrastructure; microservices, until a boundary needs its own release or scaling that pays for network calls

```
noClasses().that().resideOutsideOfPackage("..pricing..")
    .should().dependOnClassesThat()
    .resideInAPackage("..pricing.internal..")
    .check(importedClasses);
```

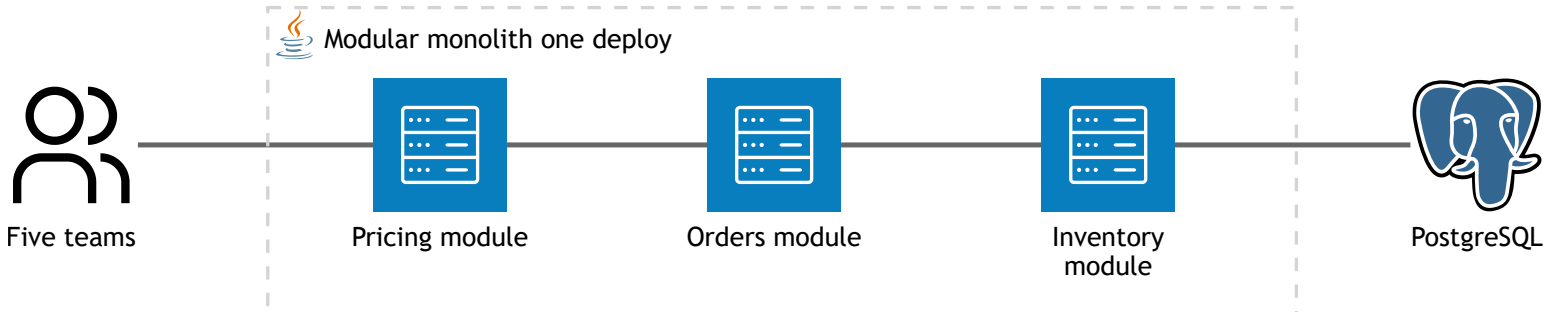
Verification: imports and coordinated changes

Check	Pass condition
Cross-module imports	Imports into pricing internals recorded as a baseline, then zero outside its public interface
Coordinated change rate	Share of pricing changes that also touch another module or need another team falls against the baseline, over the same period
Regression check	A pricing rule change leaves order and inventory tests unchanged and passing
Business invariants	Reference orders price to identical totals before and after the refactor; no oversell test changes
Fitness function	Architecture test in the build fails on any new import across a module boundary

Rollback: reverting the refactor restores the previous structure; no routing or data changed, so no state is reconciled.

Stop: when pricing changes no longer touch other modules, no service is extracted; the next module needs its own change data first.

Solution architecture



Same single deployment and database, one change: pricing, orders and inventory are separate modules that call each other only through public interfaces.

Glossary

Used in this step

Term	Definition
DDD	Domain-driven design aligns software models with business concepts
Bounded context	Boundary inside which a domain model and vocabulary have specific meaning
Modular Monolith	Single deployment with clearly separated internal modules
Monolith	Entire application deployed as a single unit
Hexagonal Architecture	Business logic isolated from infrastructure through ports and adapters
Microservices	Independently deployable services organized around business capabilities
Aggregate	Group of entities and values forming an invariant and transaction boundary
Schema ownership	Named authority to evolve data structure and writes
Event storming	Collaborative mapping of domain events, commands, decisions and policies
Fitness function	Measurable or executable architectural quality rule

Related concepts

Term	Definition
Strangler Fig	Incrementally replace legacy functionality by directing selected calls to new implementations
Team autonomy	Team can change and release its owned capabilities independently
Conway's Law	Software structure tends to reflect organizational communication structure
Context map	Description of relationships between bounded contexts
Shared database	Several modules or services use the same persistence system or schema

A37 · DDD boundaries and service separation

Protect domain from ERP terminology

The ERP calls shipments orders, and its vocabulary corrupts the meaning of the internal order model.

The order model and the ERP use one word for two things

Component	What it is	What it does
Order model	Domain model of a customer purchase	Holds lines, prices and the customer's decision to buy
Fulfillment	Code that dispatches goods to customers	Hands confirmed orders to the ERP and reads back dispatch records
Legacy ERP	Enterprise system with its own vocabulary	Records every dispatched parcel as an order
ERP client	Code that calls the ERP interface	Passes ERP records and status codes into the domain as they are
Domain language	Terms agreed by domain experts and developers	Says order for a purchase and shipment for a dispatch

Current architecture



The same word means two things

The ERP sends an order for every parcel it dispatches. The domain code reads it as the customer's order, so ERP states overwrite order states and one purchase can look like several.

1 purchase, 3 ERP orders

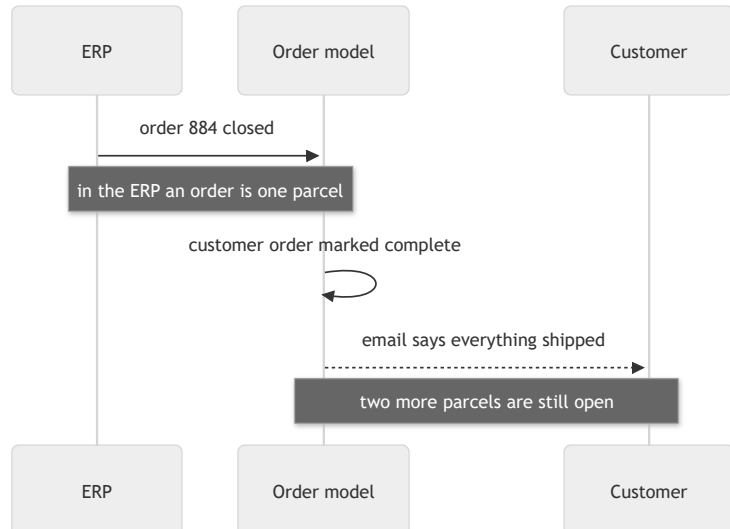
for example, one customer order dispatched as three parcels

ERP words in domain code

counted by search; each one couples the model to the vendor

20 ERP edge cases

target: all mapped with no ERP term entering the domain



Three ways to connect to a system with a different model

Solution	What it is	What it does
Anti-Corruption Layer	Layer that protects a domain model from incompatible external models	Converts ERP records into domain concepts, such as shipments, at the boundary
SOA + ESB	Services with standardized interfaces, integrated by a central middleware bus	Routes and transforms messages between systems in one shared place
Direct adapter	Client code that calls the ERP and returns its data without translation	Gives the domain the ERP’s records, field names and status codes as they are

Every option decides where the ERP’s meaning lives

Solution	Gains	Costs
Anti-Corruption Layer	Translation: the domain keeps its own language and only the layer knows ERP terms A different ERP changes one place	Code: the mapping is written, tested and kept current Interface indirection and mapping need retirement when the ERP goes
SOA + ESB	Enterprise reuse: one integration serves many consumers Central routing, transformation and monitoring	Central complexity: one more platform to run and a shared bottleneck A canonical model for all can blur the two meanings again
Direct adapter	Cheap: least code and no extra component Quickest to build	Leakage: ERP terms and codes spread through domain code Replacing the ERP means editing the domain

Deciding signal: degree of language and model mismatch, number of consumers of the ERP data and whether the ERP is expected to change.

Smallest safe move

One translation layer for the shipment flow

- **Scope:** the one ERP flow that reports dispatches
- **Change:** a domain-owned port plus one adapter that translates each ERP order into a domain shipment; the context map marks the ERP upstream
- **Safety net:** 20 ERP edge cases run as tests; the build fails when domain code imports an ERP type
- **Escape route:** a flag keeps the direct call; the layer first runs in shadow on the live feed without persisting
- **Deferred:** SOA + ESB, until several systems need the same integration; the mapping is retired with the ERP

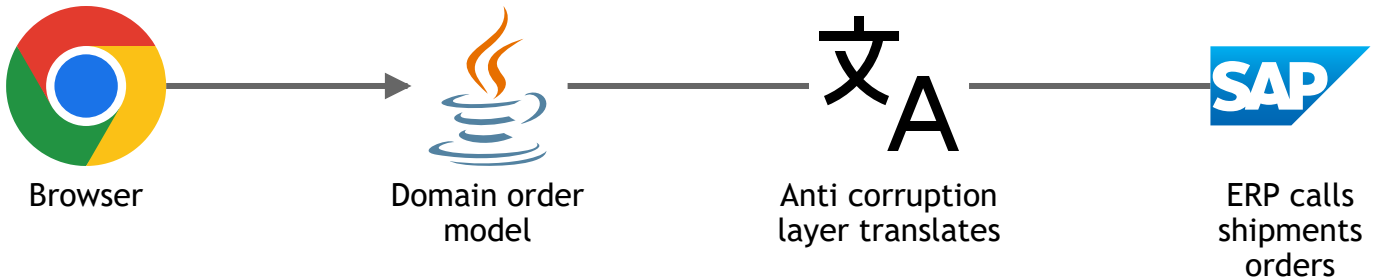
```
Shipment toShipment(ErpOrder o) {
    return new Shipment(ShipmentId.of(o.docNumber()),
        OrderId.of(o.customerRef()), statusFor(o.state()));
}
```

Verification: 20 ERP edge cases

Check	Pass condition
Edge cases	20 realistic ERP records, such as split, partial, duplicate and out-of-order, translate with no ERP term in the result
Semantics	One customer order with three ERP records yields one order and three shipments
Dependency rule	Build fails when a class outside the layer imports an ERP type or status code
Shadow parity	On the same live feed, old and new paths agree, and every difference is explained
Business invariants	Order status and shipment counts match ERP dispatch records after reconciliation
Fitness function	Edge-case suite and dependency rule run in every build

Rollback: the flag routes back to the direct call; shipments already written by the layer stay valid, and re-running the translation over the ERP feed repairs any mismatch.

Solution architecture



Same domain and unchanged ERP, one new layer between them: ERP terms stop at the layer and the domain keeps its own words.

Glossary

Used in this step

Term	Definition
Anti-Corruption Layer	Protects a domain model from incompatible external system models
Ubiquitous language	Shared terminology used by domain experts and developers
Canonical model	One shared data model imposed on every system that integrates through a bus
Bounded context	Boundary inside which a domain model and vocabulary have specific meaning
Context map	Description of relationships between bounded contexts
Upstream context	Context providing domain model or contract consumed by another
Downstream context	Context adapting to another bounded context's model or events
SOA	Services communicating through standardized interfaces
ESB	Central middleware for integrating heterogeneous enterprise systems
Direct adapter	Client code that passes an external system's data on without translation

Related concepts

Term	Definition
Hexagonal Architecture	Business logic isolated from infrastructure through ports and adapters
Branch by Abstraction	Introduce stable interface then change the implementation behind it gradually
Fitness function	Measurable or executable architectural quality rule
Golden master	Capture existing observable outputs as compatibility reference
Domain Translation Adapter	Translate one system's terms and records into another's semantics
Domain event	Statement that an important business fact occurred

A38 · DDD boundaries and service separation

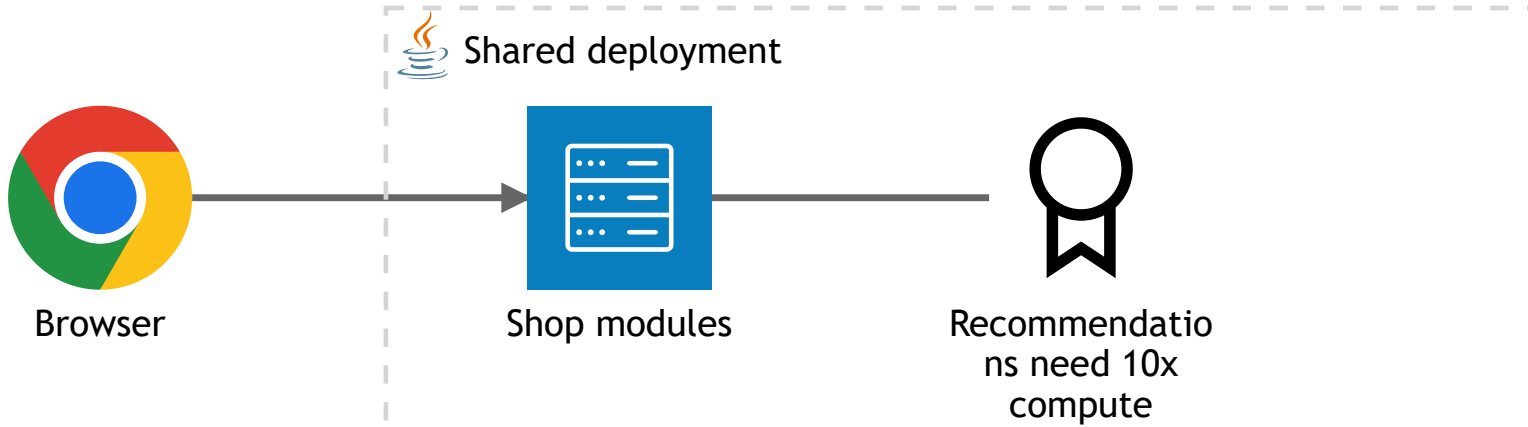
Scale one capability independently

Recommendations need 10x compute and a release cycle separate from the rest of the shop.

The recommendation processor runs inside the shared deployment

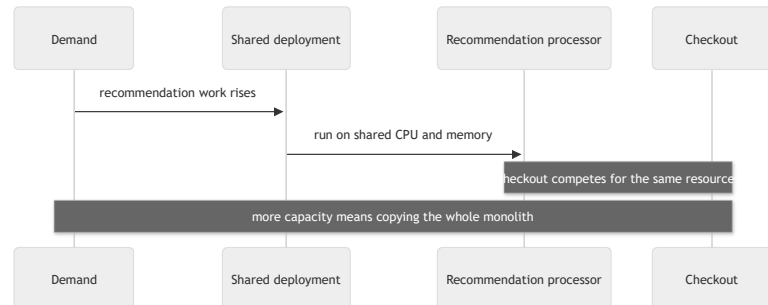
Component	What it is	What it does
Shop modules	Catalog, pricing, orders and other capabilities in one Java monolith	Serve customer requests from one deployment
Recommendation processor	Compute-heavy module that ranks related products	Shares processors, memory and release with every other module
Shared deployment	One build deployed as one unit	Gains capacity only by adding copies of the whole monolith
Recommendation team	Team that owns ranking logic and models	Waits for the shared release to ship a change

Current architecture



One hot module sets the capacity of everything

Recommendations need ten times the compute of any other module, and their team wants its own release schedule. Capacity can only be added by copying the whole monolith, and every change waits for the shared release.



Separate team release

recommendation changes wait for the shared release

10x compute

recommendations need ten times the capacity of other modules

Unrelated services flat

target: their p95 and CPU do not move when recommendations scale

Three ways to give one capability more compute

Solution	What it is	What it does
Microservices	Independently deployable services organized around business capabilities	Runs recommendations as its own service with its own release, scaling and owners
Separate worker process	Background process built from the same codebase, fed by jobs from a queue	Moves the compute to its own processors, but still shares code and release with the monolith
Scale monolith	More instances of the whole application behind a load balancer	Adds capacity for every module at once, recommendations included

Capacity is bought with distributed operations or with waste

Solution	Gains	Costs
Microservices	Independent scaling: only recommendations get 10x Independent release by its own team; faults stay inside the service	Distributed operations: deployment, monitoring, versioning and on-call for one more service Every call crosses the network; a wrong boundary becomes chatty calls
Separate worker process	Isolated CPU: recommendation work stops competing with checkout Little new infrastructure; same code and tooling	Some coupling: shared code, schema and release, so a separate release is not met A queue or job table between web and worker to run and monitor
Scale monolith	Simple: no code change, one deployment, one pipeline No new network boundary	Inefficient: ten copies of every module to feed one hot module Release stays coupled; every copy adds database connections and needs statelessness

Smallest safe move

One recommendation service behind the existing interface

- **Scope:** recommendations only; every other module stays in the monolith
- **Change:** a new service implements the recommendation interface and is owned, released and scaled by its own team; the shop makes one coarse call per page
- **Safety net:** a timeout and a popular-products fallback keep pages rendering; the service reads its inputs read-only
- **Escape route:** a flag, with a small canary share first, routes calls back to the in-process code, which stays deployed during validation
- **Deferred:** worker process and scaling the monolith; serverless, until load is bursty with long idle time

```
recommendations:
  mode: remote        # local = in-process code
  timeoutMs: 150      # illustrative
  fallback: popular-products
```

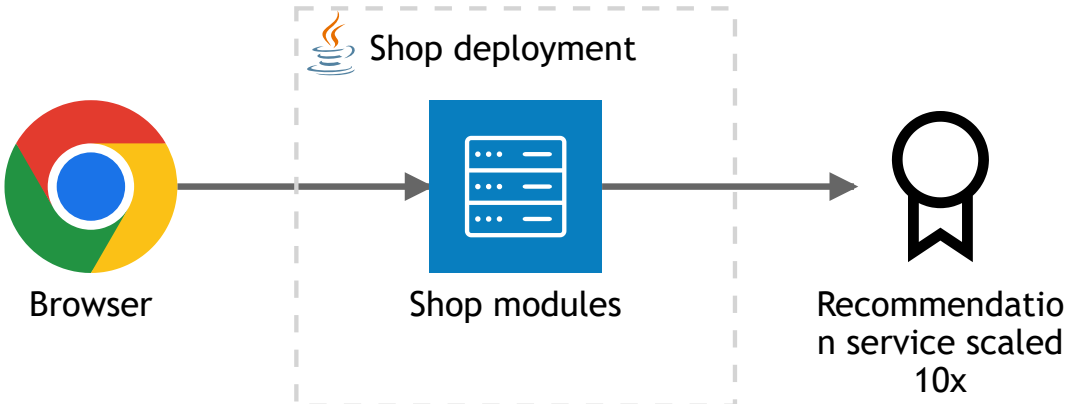
Verification: scale recommendations only

Check	Pass condition
Scale test	Recommendation capacity raised to 10x while the shop deployment stays unchanged
Unrelated services	p95 and CPU of checkout and catalog stay within their baseline during the test
Release independence	A recommendation change ships through its own pipeline without a monolith release
Call cost	One call per page; remote p95, timeout rate and fallback rate are reported
Business invariants	Order, stock and payment results are unchanged; recommendation failure never blocks checkout
Fitness function	Load test with the service stopped: pages render with the fallback and checkout p95 stays in SLO

Rollback: the flag returns calls to the in-process code; recommendations are derived data, so nothing needs reconciliation.

Stop: when the service holds 10x capacity and releases on its own, no other module is split without its own measured reason.

Solution architecture



Same browser and shop modules, one capability moved out: the recommendation service has its own deployment, scaling and release.

Glossary

Used in this step

Term	Definition
Microservices	Independently deployable services organized around business capabilities
Monolith	Entire application deployed as a single unit
Separate worker process	Background process built from the same codebase that runs jobs away from the web processes
Horizontal scaling	Distribute workload across more instances
Load balancer	Distributes incoming requests across multiple servers or instances
Team autonomy	Team can change and release its owned capabilities independently
Feature flag	Runtime control enabling behavior without deploying new code
Canary routing	Shift a small controlled segment of live traffic to a new implementation
Graceful degradation	Retain core capability when optional dependencies fail
Timeout	Limits how long an operation may wait for completion

Related concepts

Term	Definition
Serverless Function (FaaS)	Executes short-lived functions triggered by events or requests
Autoscaling Group	Automatically adjusts compute capacity based on demand
Vertical scaling	Add CPU or memory to one host
Circuit breaker	Stops requests to failing dependencies temporarily
Message queue	Buffers messages for asynchronous processing by consumers

A39 · DDD boundaries and service separation

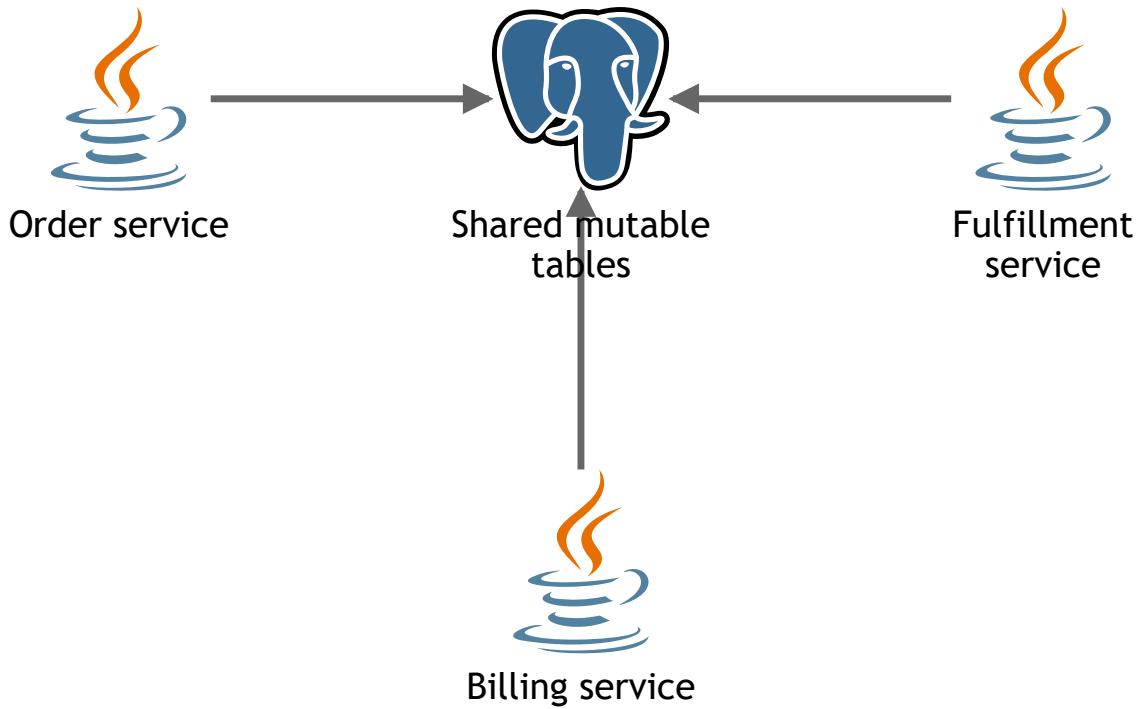
End cross-service schema coupling

An order migration changes the billing and fulfillment schemas, so three services must deploy together.

Three services share one set of mutable tables

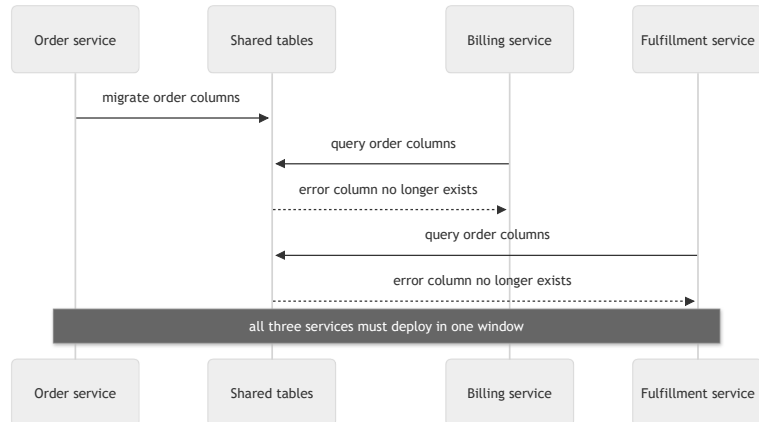
Component	What it is	What it does
Order service	Service for the customer purchase lifecycle	Writes order rows and reads payment and shipment status from the shared tables
Billing service	Service that charges and invoices customers	Reads order rows and writes payment and invoice rows in the same tables
Fulfillment service	Service that picks and dispatches goods	Reads order rows and writes shipment rows in the same tables
Shared mutable tables	One PostgreSQL schema that all three services write	Acts as the only contract between the services, so a column change reaches every reader
Schema migration	Versioned change to table structure, such as a renamed column	Works only if every service that queries the table changes in the same window

Current architecture



One migration forces three releases

The order migration renames a column that billing and fulfillment also query. Their SQL fails until each service ships a matching change, so three independently deployed services release in one window.



3 services, 1 release window

one order migration pulls billing and fulfillment into the same deploy

2 dependent schemas

billing and fulfillment schemas change with every order migration

0 coordinated deploys

target: an order schema change ships without deploying billing or fulfillment

Three ways to give each service a schema it can change alone

Solution	What it is	What it does
Database per service	Each service owns its private persistence and schema	Puts the order tables behind the Orders API and events; no other service can query them
Shared database with ownership	Several services use one database, each owning its own schema	Keeps one database with one writing owner per table; others read through views, not base tables
CDC local projections	Change data capture feeding a read-only copy of another service's data	Streams the owner's row changes into tables that the consumer queries locally

Schema autonomy is paid with remote reads, shared infrastructure or lag

Solution	Gains	Costs
Database per service	Ownership: only the owner reads or changes its tables, so schema and deployment are autonomous Load and faults in one database stay there	Remote joins: another service's data arrives through an API or events, not SQL No cross-service ACID transaction; consistency between services becomes eventual
Shared database with ownership	Simpler: one database to run; joins and transactions across services still work No new infrastructure: ownership is added with schemas, grants and views	Coupled: services still share one engine, its upgrades, locks and connection limits Ownership is held by grants and review, so a direct read can creep back
CDC local projections	Read decoupling: a consumer queries its own copy without calling the owner Local joins stay possible and reads survive an owner outage	Lag: the copy trails the owner and can be read stale Raw row changes expose the owner's table structure, so a column rename still reaches consumers

Smallest safe move

Order tables private to the Order service

- **Scope:** order tables only; billing and fulfillment keep their shared tables for now
- **Change:** Orders gets its own database; billing and fulfillment read order data through the Orders API or events, not cross-schema SQL
- **Safety net:** expand and contract: a one-way CDC stream keeps the old order tables current for unmigrated readers; counts are reconciled; consumers expect eventual consistency
- **Escape route:** a feature flag points Orders back at the old tables after the stream drains
- **Deferred:** shared database with ownership, until an invariant needs one ACID commit across services; CDC local projections, until Orders API reads break a consumer's p95

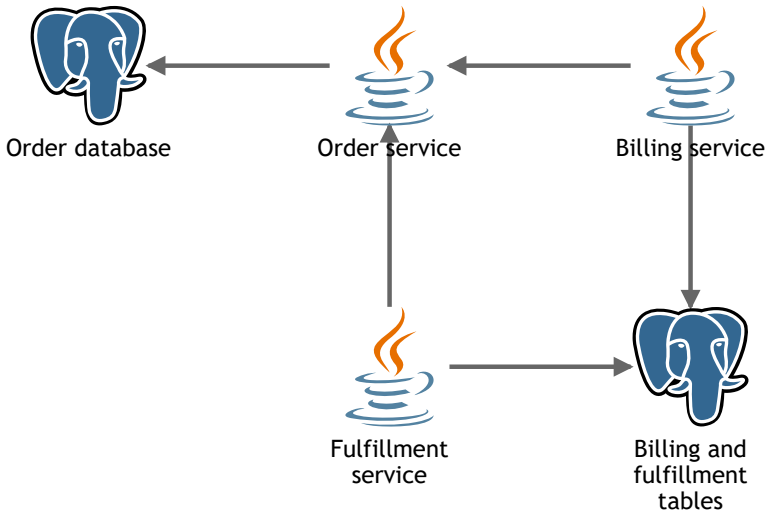
```
-- contract step, after the last reader has moved
REVOKE ALL ON orders, order_lines
    FROM billing_app, fulfillment_app;
```

Verification: change the order schema alone

Check	Pass condition
Schema change	A renamed order column ships with only the Orders service deployed; billing and fulfillment have no release
Direct access	Grants and query logs show no reads of order tables by billing or fulfillment
Data parity	Old and new order tables match on row counts and checksums until the old tables are dropped
Read cost	Orders API p95 and error rate are recorded; billing and fulfillment p95 stay within their SLO
Business invariants	Every confirmed order has one invoice and one fulfillment request; no payment is charged twice
Fitness function	Build fails when a service queries another service’s schema or breaks the Orders API contract

Rollback: the flag returns Orders to the old tables, which is routing rollback; orders written to the new database since the cutover are state, so drain the capture stream and reconcile, or repair forward.

Solution architecture



Same three services, one new database: Orders owns its tables, and billing and fulfillment reach order data through the Order service instead of its tables.

Glossary

Used in this step

Term	Definition
Database per service	Each service owns its private persistence and schema
Shared database	Several modules or services use same persistence system or schema
Schema ownership	Named authority to evolve data structure and writes
Database view	Named query presenting an abstraction over underlying tables
Cross-schema read	Query by one service on tables that another service owns
Change data capture (CDC)	Detects and propagates changes made to database records
Local projection	Read-only copy of another service's data, kept current from its changes
Expand-contract	Add compatible representation, migrate consumers, then remove old representation
Data reconciliation	Compare systems using keys, checksums and domain invariants to find divergence
Eventual consistency	Copies converge after a delay, so a reader may briefly see older data

Related concepts

Term	Definition
Saga pattern	Coordinates distributed business transactions using local transactions and compensations
Transactional outbox	Atomically records business changes and outgoing events in the same database transaction
Database transactions	Atomic execution boundary for related data modifications
Message queue	Buffers messages for asynchronous processing by consumers
Bounded context	Boundary inside which a domain model and vocabulary have specific meaning
Forward fix	Repair the new implementation without restoring the old system

A40 · DDD boundaries and service separation

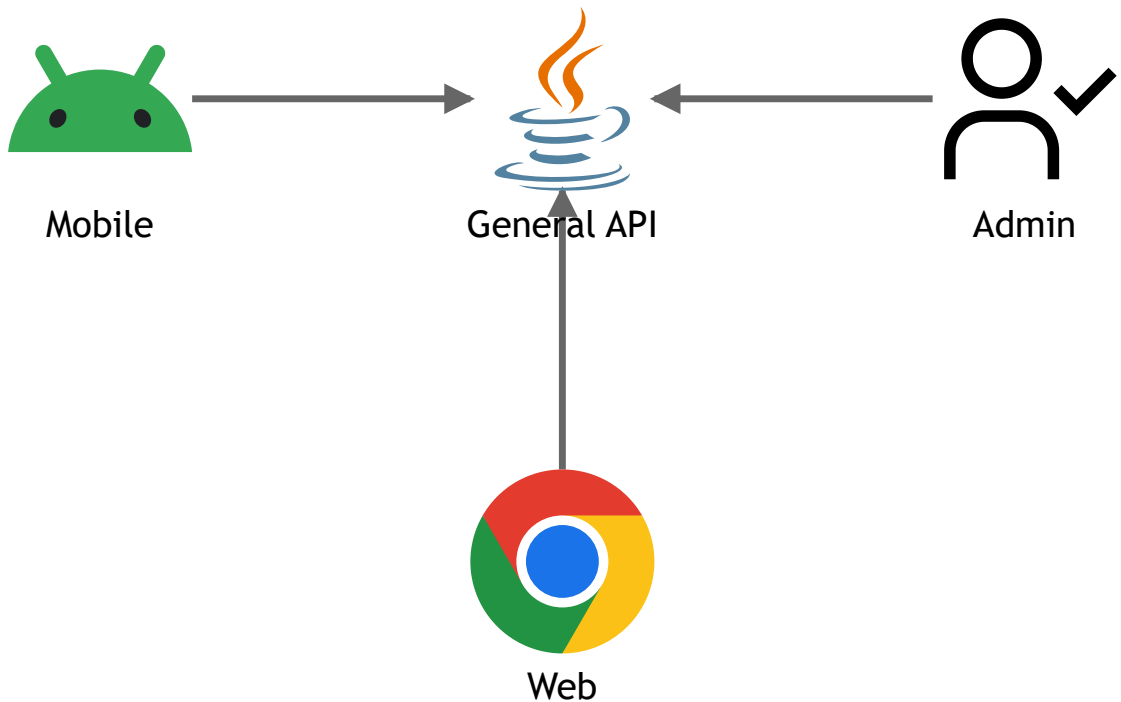
Tailor APIs per consumer

Web, mobile and admin clients need distinct payloads and releases, but one general API serves all three.

One general API serves web, mobile and admin

Component	What it is	What it does
Web client	Browser single-page app for shoppers	Loads full product and order views from the general API
Mobile client	Native app installed from an app store	Calls the same endpoints over mobile networks; old app versions stay in use
Admin client	Internal back-office tool	Reads and edits full records through the same endpoints
General API	One service with one payload per endpoint	Returns the same fields to every client
Shared release	One deployment of the general API	Ships every client’s field changes at the same time

Current architecture



One payload and one release for three clients

A field that mobile needs reaches web and admin too, and every client must retest the release. Old mobile app versions stay in use after a release, so fields cannot be reshaped or removed freely.

1 payload, 3 clients

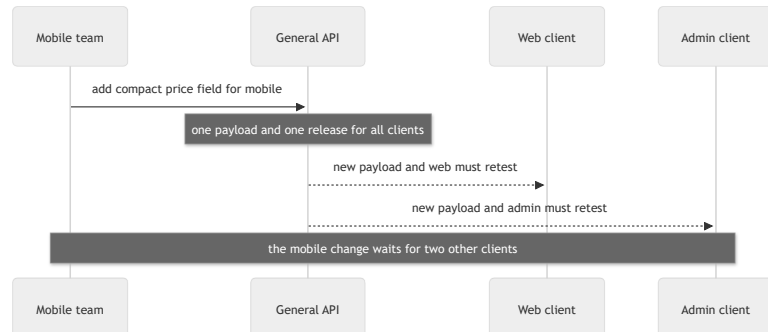
every client receives the same fields, needed or not

1 shared release

a change for one client ships with the other two

0 web changes

target: a mobile-specific field ships without any web change



Three ways to serve each client what it needs

Solution	What it is	What it does
BFF	Backend for Frontend: an API layer built for one specific client	Maps the core API into the payload that client needs and ships with that client's team
GraphQL	Typed query interface allowing clients to request selected fields	Serves each client exactly the fields it asks for from one endpoint
API Gateway	Central entry point for routing, authentication, rate limiting and API policies	Routes each request to a service and applies shared policy in one place; it does not compose client-specific responses

Tailoring is paid with duplication, query policy or no composition

Solution	Gains	Costs
BFF	Tailored: payload, calls per screen and release date fit one client The client team owns it, so a mobile change never touches web	Duplication: similar mapping and aggregation code repeats in each BFF One more service per client to run, monitor and secure
GraphQL	Flexible: each client selects its own fields from one endpoint No over-fetching; new fields do not break existing queries	Query policy: depth, complexity and rate limits are needed to cap expensive queries Caching is harder than for plain URLs; resolvers can multiply backend calls
API Gateway	Central routing: one entry point for all clients Authentication, rate limiting and logging enforced once	No composition: each route still returns one payload, so client shapes stay unsolved A shared dependency: its outage or misconfiguration reaches every client

Smallest safe move

One BFF for the mobile client

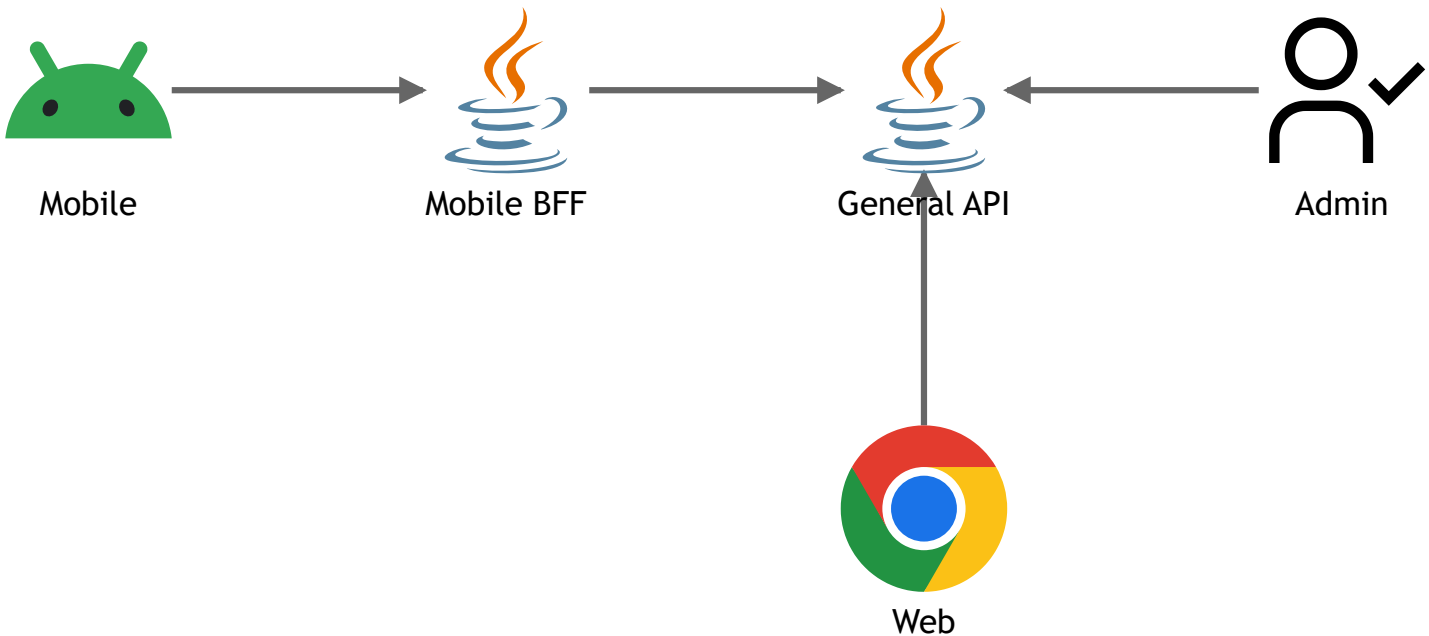
- **Scope:** mobile only, the slowest-releasing client; web and admin keep the general API
- **Change:** a mobile BFF in front of the core API serves a compact payload with one call per screen; the mobile team owns it
- **Safety net:** the BFF holds no business rules or state; contract tests pin the core API; mobile fields stay additive for old app versions
- **Escape route:** a canary route switch sends mobile to the BFF; switching back returns it to the general API
- **Deferred:** web and admin BFFs, until their payloads diverge; GraphQL, until field variants outgrow a BFF; gateway, until edge policy repeats

```
routes:
  - path: /mobile/**
    upstream: mobile-bff    # was: general-api
    canary: 5%              # illustrative
```

Verification: ship a mobile field alone

Check	Pass condition
Release independence	A mobile-specific field ships through the mobile pipeline; web and admin have no build, deploy or retest
Payload	Mobile response bytes per screen fall against baseline, to a target agreed with the mobile team
Round trips	Calls per mobile screen are recorded before and after and fall against baseline
Latency	Mobile p95 with the extra BFF hop stays at or below baseline
Business invariants	Prices, stock and order totals shown in mobile equal the values the general API returns for the same request
Fitness function	Contract tests fail the build when a core API change breaks the BFF; alert when a mobile change triggers a web build

Solution architecture



Same clients and general API, one new node: mobile traffic passes through its own BFF, while web and admin still call the API directly.

Glossary

Used in this step

Term	Definition
BFF	API layer tailored to a specific frontend or client
API Gateway	Central entry point for routing, authentication, rate limiting and API policies
GraphQL	Typed query interface allowing clients to request selected fields
Payload	The data a response carries to a client: its fields and their size
Over-fetching	Response carries fields or calls that a client does not need
Aggregation	Combining several backend calls into one response for a client
Additive change	New fields only; nothing existing is removed or renamed, so old clients keep working
Query cost limit	Cap on the depth or complexity of a GraphQL query
Consumer-driven contract test	Consumer expectations verified against a provider
Canary routing	Shift a small controlled segment of live traffic to a new implementation

Related concepts

Term	Definition
REST	Resource-oriented interface style using uniform HTTP semantics
Reverse proxy	Intermediary that forwards client requests to backend services
Microfrontend	Frontend divided into independently developed and deployed modules
CQRS	Separates models for reading data and modifying state
Team autonomy	Team can change and release its owned capabilities independently

A41 · DDD boundaries and service separation

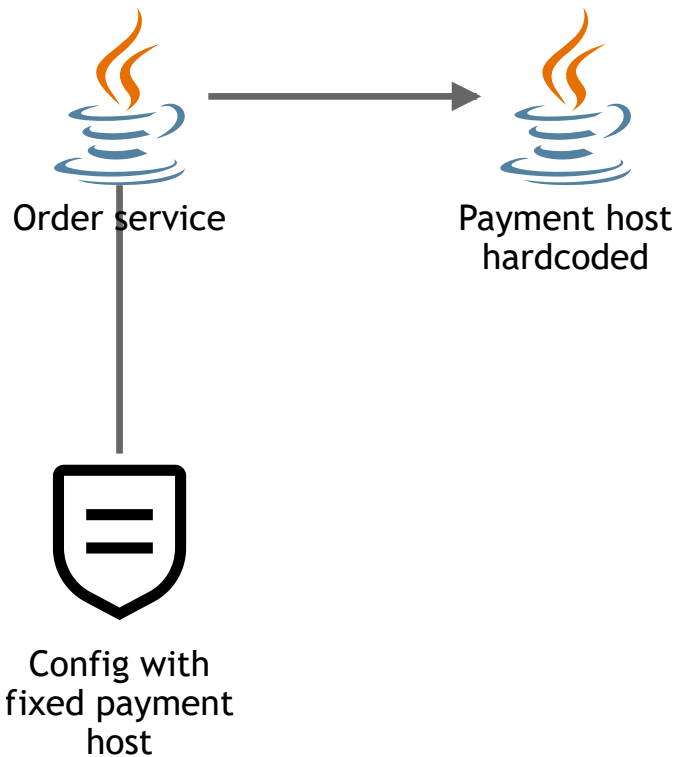
Find changing service instances

Hardcoded hostnames fail when 15 moving service deployments are rescheduled.

Every caller holds a fixed address for its target

Component	What it is	What it does
Calling service	One of 15 separately deployed services, such as the Order service	Reads the payment host name from its own configuration
Target service	A separately deployed service, such as the Payment service	Runs as one or more instances on hosts that change
Hardcoded hostname	A fixed host name or IP address in configuration	Points a caller at one specific instance address
Rescheduling	A restart, move or replacement of an instance on another host	Gives the instance a new address without telling any caller
Configuration release	An edit and redeploy of every affected caller	The only way a caller learns that an address changed

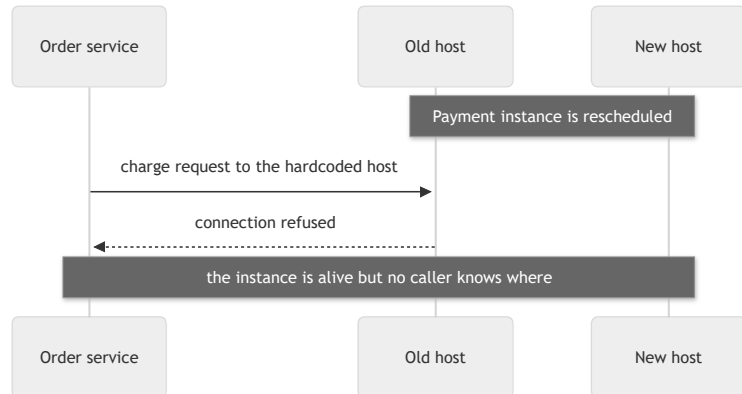
Current architecture



The order service reads a fixed payment hostname from configuration, which breaks when the payment deployment moves.

A moved instance breaks every caller

Payment is redeployed and starts on another host with a new address. Order still calls the old address, so requests fail until each caller’s configuration is edited and released.



15 moving deployments
services whose addresses change across releases and failures

Connection refused
symptom: calls to a stale address fail while an instance moves

0 routing errors
target while backend instances are rolled

Three ways to stop depending on a fixed address

Solution	What it is	What it does
Service Registry	Directory of currently reachable service instances	Instances register on start; callers look up current addresses before calling (client-side discovery)
Internal DNS	Name service for private names inside the network	Resolves a stable service name to current instance addresses; changes spread as cached answers expire
Reverse Proxy	Intermediary that forwards client requests to backend services	Gives callers one stable address and forwards to the current instances from its own configuration (server-side discovery)

Every option moves the address problem somewhere else

Solution	Gains	Costs
Service Registry	Addresses follow instances automatically Health checks remove failing instances from lookups	Control dependency: callers need the registry to find targets Stale or wrong entries route traffic to dead instances
Internal DNS	Simple: standard protocol, no client library Works for any language and tool	TTL: resolvers and clients keep old answers until expiry No health awareness unless the DNS service adds it
Reverse Proxy	Stable target: callers never see instance addresses One place for timeouts, retries and TLS	Central configuration must track every instance Extra network hop and a shared component to replicate

Deciding signal: instance churn per hour, the longest acceptable stale-address window compared with DNS TTL, and operational capacity to run one more highly available component.

Smallest safe move

One route resolved through a registry

- **Scope:** the Order to Payment route only; all other routes keep their hardcoded hosts
- **Change:** Payment instances register on start; Order looks up their addresses and caches the last known list
- **Safety net:** health checks drop failing instances from lookups; the old host stays as a fallback behind a flag
- **Escape route:** switch the flag back to the hardcoded host; no data is touched
- **Deferred:** internal DNS and reverse proxy, until a caller cannot embed a discovery client or a measured TTL window proves short enough

```
payment:
  discovery: registry          # static falls back to the old host
  cache-last-known: true
  static-host: payment-host-1  # fallback only
```

Verification: roll the instances and count routing errors

Check	Pass condition
Rolling restart	Every Payment instance is restarted or moved under Order traffic with 0 routing errors
Address freshness	A new instance receives traffic and a stopped one receives none within the agreed time, for example 30 seconds
Registry outage	With the registry stopped, Order keeps calling the last known healthy addresses
Success and latency	Checkout success ratio and p95 stay at baseline; lookup time recorded separately
Fitness function	A rolling-restart test fails the build on any routing error against a stale address

Rollback: switching the flag back restores the hardcoded host and is a routing rollback only. A charge that failed mid-flight may already be applied, so retries carry an idempotency key; that is state repair, not routing.

Stop: when a rolling restart shows 0 routing errors on the route, no gateway or mesh is added; further routes move one at a time.

Solution architecture

Same caller and target. The registry now holds current addresses, and the configuration keeps only the registry’s stable name.

Glossary

Used in this step

Term	Definition
Service instance	One running copy of a service, with its own address
Rescheduling	Restart or move of an instance on another host, usually with a new address
Hardcoded hostname	Fixed host name or IP address written into a caller's configuration
Service discovery	Dynamically identifies available service instances and endpoints
Service Registry	Directory of currently reachable service instances
Client-side discovery	The caller looks up instances in a registry and picks one itself
Server-side discovery	The caller uses one stable address and an intermediary picks the instance
Health check	Detects whether an application instance is functioning correctly
Internal DNS	DNS for private service names, resolved only inside the network
TTL	How long a resolver may keep an answer before asking again

Related concepts

Term	Definition
Load Balancer	Distributes incoming requests across multiple servers or instances
Service Mesh	Manages service-to-service traffic, security, and observability
API Gateway	Central entry point for routing, authentication, rate limiting, and API policies
Kubernetes Cluster	Group of machines orchestrating containerized applications
Managed Container Platform	Cloud provider operates the scheduling and runtime platform

A42 · DDD boundaries and service separation

Unify service policy at ingress

Security policy drift across 12 public endpoints causes missed checks.

Each service enforces its own authentication and throttling

Component	What it is	What it does
Client	Browser, mobile app or partner system calling the public API	Sends a token with each request to any of the public endpoints
Public endpoint	One of 12 externally reachable routes across the services	Is protected only by the service behind it
Service-local authentication	Token validation code or library inside each service	Checks identity and scope with a copy that differs between services
Service-local throttling	Rate-limit configuration inside each service	Limits request rates with thresholds chosen per service
Policy change	An edit and redeploy in every affected service	Reaches services one by one, so some lag behind

Current architecture

Each service enforces its own authentication and rate limits, so the policies drift apart.

One missed check is an open path

A tightened token check reaches most services, but one still runs the old copy. Clients find the unchecked path first, and no single place shows which endpoints enforce what.

12 public endpoints

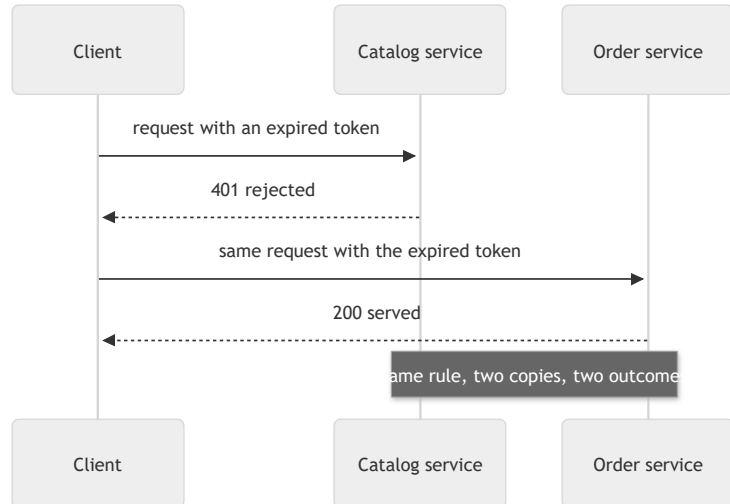
each with its own copy of authentication and throttling

Missed checks

symptom: some paths accept requests that policy says to reject

Same policy on every path

target: authentication and rate limit audited as consistent on all public paths



Three places to enforce policy once

Solution	What it is	What it does
API Gateway	Central entry point for routing, authentication, rate limiting, and API policies	Validates tokens and enforces rate limits once for all public routes before forwarding to the services
Ingress Controller	Routes external traffic into Kubernetes services	Maps hosts and paths to services at the cluster edge and terminates TLS; policy is limited to what its configuration and plugins offer
Service Mesh	Manages service-to-service traffic, security, and observability	Enforces traffic and security policy on every service-to-service hop through proxies next to the services

Central policy trades drift for a shared dependency

Solution	Gains	Costs
API Gateway	Shared edge enforcement: one policy change covers every public route The audit surface shrinks to one enforcement point	Dependency: its outage or bad rule affects every route Services still need object-level authorization behind it
Ingress Controller	Simple declarative routing and TLS at the cluster edge Cheap to adopt when the platform already runs on Kubernetes	Fewer application policies: token claims and per-client quotas need extra plugins Presumes Kubernetes, which ties the edge to the orchestration choice
Service Mesh	Uniform policy and mutual TLS on internal traffic without code changes Telemetry for every hop	Complexity: control plane and proxies to run and upgrade Extra latency and resources per hop; public client policy is still a separate concern

Smallest safe move

One public route behind the gateway

- **Scope:** one public route, for example catalog search; other endpoints keep their own checks
- **Change:** route it through an API gateway that validates the token and applies a shared rate limit
- **Safety net:** the gateway runs in log-only mode first and its verdicts are compared with the service's; services keep their own checks, including object-level authorization
- **Escape route:** point the route back at the service
- **Deferred:** ingress controller and service mesh, until cluster-edge routing or east-west controls are justified

```
route: /api/catalog/**
upstream: catalog-service
auth: jwt            # signature, expiry and scope
rate-limit: 100/min  # per client, for example
```

Verification: audit every public path

Check	Pass condition
Path inventory	All 12 public endpoints are listed and routed through the gateway
Authentication	Missing, expired and wrong-scope tokens are rejected on every public path
Rate limit	A burst above the limit gets 429 on every public path; normal traffic passes
Bypass	Direct requests to service addresses from outside the trusted network are refused
Latency and availability	Added p95 from the gateway hop stays within the agreed budget; a gateway failure drill is recorded
Fitness function	A policy test replays the audit against every public route on each deploy and fails on any path missing a check

Rollback: pointing a path back at its service is a routing rollback, and the policy is versioned configuration that can be restored. Policy changes no business state, but requests wrongly rejected by a faulty rule are not replayed; clients must

Solution architecture

Same client and services. The gateway now enforces authentication and rate limits once, and the services keep object-level authorization.

Glossary

Used in this step

Term	Definition
Security policy drift	Copies of one rule diverge until some paths enforce an old or missing version
API Gateway	Central entry point for routing, authentication, rate limiting, and API policies
Ingress Controller	Routes external traffic into Kubernetes services
Service Mesh	Manages service-to-service traffic, security, and observability
Authentication	Verifies who the caller is, for example by validating a signed token
JWT	Signed token that carries identity and scopes, checked for signature, expiry and scope
Rate limiter	Controls the number of requests allowed within a time window; throttling rejects the excess
North-south traffic	Traffic between external clients and the platform
East-west traffic	Traffic between services inside the platform
Object-level authorization	Check that the caller may access this specific object, such as their own order

Related concepts

Term	Definition
Reverse Proxy	Intermediary that forwards client requests to backend services
WAF (Web Application Firewall)	Filters malicious HTTP traffic
OAuth 2.0	Framework for delegated authorization
Zero Trust	Require verification for access rather than assuming network trust
Backend for Frontend (BFF)	API layer tailored to a specific frontend or client

A43 · DDD boundaries and service separation

Automate workload placement

Manual scheduling, restart and rollout of 30 containers across eight machines are unreliable.

Operators place, restart and update containers by hand

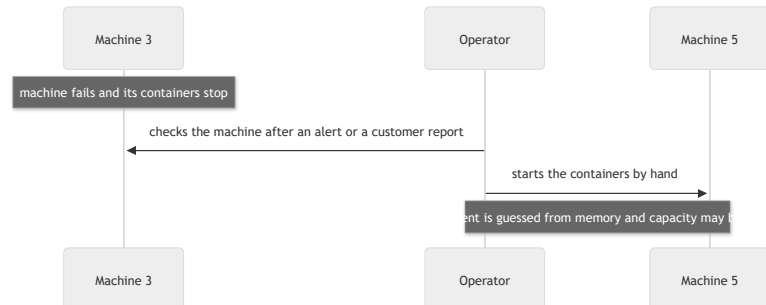
Component	What it is	What it does
Container	Lightweight isolated environment packaging an application and its dependencies	Runs one instance of a service with everything it needs
Machine	One of eight servers that host the containers	Runs several containers chosen by an operator
Operator	An engineer or script working from a runbook	Picks the machine, then starts, restarts and updates each container by hand
Placement	The decision of which machine runs which container	Lives in runbooks and memory; nothing enforces or checks it
Rollout	Replacement of old container versions with new ones	Runs as a manual sequence, machine by machine

Current architecture

An operator places, restarts and rolls out containers on each machine by hand.

A dead machine stays dead until someone notices

A machine fails during a release and its containers stop. They stay down until a person notices and restarts them elsewhere, and the manual rollout skips or repeats a machine.



30 containers, 8 machines
placed, restarted and updated by hand

Manual restart
symptom: a failed container stays down until an operator acts

0 operator actions
target: the workloads of a killed node run again elsewhere

Three ways to automate placement, restart and rollout

Solution	What it is	What it does
Kubernetes	Group of machines orchestrating containerized applications	A scheduler places containers by declared resources, and controllers restart failed ones and replace versions gradually until the declared state holds
Managed Container Platform	Cloud provider operates the scheduling and runtime platform	Takes the image and the desired count, then places, restarts and updates the containers without a cluster for the team to run
Docker Compose	Declarative multi-container application management, often single host	Describes the containers in one file and starts and restarts them on one machine

Automation is bought with operations, control or reach

Solution	Gains	Costs
Kubernetes	Orchestration: placement, restart and rolling updates from one declared state Portable across providers, with a large ecosystem	Steep overhead: cluster operations, upgrades, networking and security Many new concepts for every team to learn
Managed Container Platform	Simpler: no cluster to operate Lower operational staffing	Less control: provider limits on networking, runtime and configuration Platform-specific setup makes leaving harder
Docker Compose	Simple: one file, little to learn Fits local and modest deployments	Limited fleet: it does not schedule across machines A dead machine is still recovered by hand

Deciding signal: fleet size and churn, the recovery time required after losing a machine, and platform-team capacity compared with the provider limits the workloads can accept.

Smallest safe move

One stateless service under a scheduler

- **Scope:** one stateless service, for example recommendations; databases and all other containers stay on the machines
- **Change:** join some existing machines to a cluster as nodes and declare the service with replicas, a readiness probe and a rolling update
- **Safety net:** the old containers keep running while traffic shifts gradually; spare capacity covers one node loss
- **Escape route:** shift traffic back to the old containers and scale the declaration to zero
- **Deferred:** managed container platform, once cluster operations outgrow the team and provider limits are acceptable; Docker Compose does not schedule across machines

```
spec:
  replicas: 3          # desired state, for example
  strategy:
    rollingUpdate: { maxUnavailable: 0, maxSurge: 1 }
```

Verification: kill a node and watch rescheduling

Check	Pass condition
Node loss	One node is stopped under normal traffic and its replicas run again on the remaining nodes with no operator action
Recovery time	Time from node loss to ready replacements is recorded and within the agreed target
Rolling update	A new image rolls out with 0 failed requests while old replicas serve until new ones are ready
Capacity headroom	The remaining nodes hold the lost node’s workloads within their resource limits
Fitness function	A scheduled node-drain test in staging fails the pipeline when rescheduling or rollout exceeds the target

Rollback: shifting traffic back to the old containers is a routing rollback. A stateless service holds no data to restore; stateful workloads would need their volumes handled separately, so none is moved in this step.

Solution architecture

Same operator and machines, two of the eight shown. The operator now declares the desired state and a scheduler places, restarts and updates containers to match it.

Glossary

Used in this step

Term	Definition
Container	Lightweight isolated environment packaging an application and its dependencies
Node	A machine that runs containers and is managed by the orchestrator
Placement	The decision of which machine runs which container
Scheduler	Component that assigns containers to nodes by declared resources and constraints
Orchestration	Automated placement, restart and update of containers across machines
Desired state	Declared replicas and versions that the platform keeps true
Kubernetes Cluster	Group of machines orchestrating containerized applications
Managed Container Platform	Cloud provider operates the scheduling and runtime platform
Docker Compose	Declarative multi-container application management, often single host
Rolling Deployment	Replace application instances incrementally

Related concepts

Term	Definition
Kubernetes Pod	Smallest Kubernetes scheduling unit, containing one or more containers
Serverless Function (FaaS)	Executes short-lived functions triggered by events or requests
Autoscaling Group	Automatically adjusts compute capacity based on demand
Blue-Green Deployment	Maintain two release environments and switch traffic
Health Check	Detects whether an application instance is functioning correctly

A44 · DDD boundaries and service separation

Decouple frontend team release cadence

One web bundle shared by three UI teams blocks unrelated areas from releasing.

Three teams build and ship one web bundle

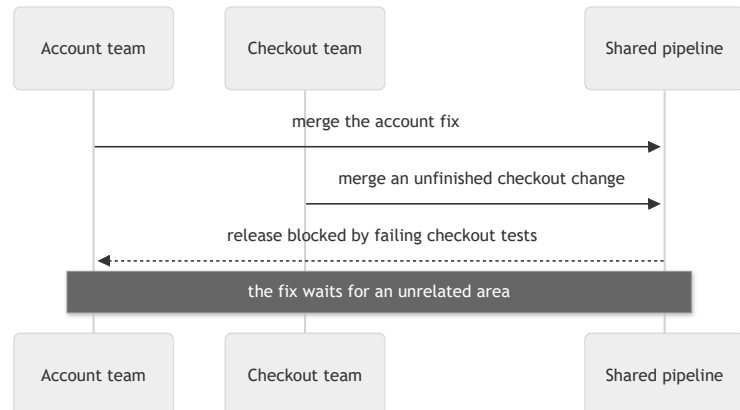
Component	What it is	What it does
Web bundle	One single-page application artifact	Contains the catalog, account and checkout screens and ships as one release
Three UI teams	The catalog, account and checkout teams	Commit to one repository and one build
Shared pipeline	One build, test and deploy run for the whole bundle	Gates every change behind the tests of all three areas
Release schedule	One release at a time for the whole bundle	Ships only when every merged change passes together
Backend APIs	The services the web client calls	Serve all three areas of the bundle

Current architecture

Catalog, account and checkout UI teams ship through one shared bundle and one release.

An unrelated area decides when a fix ships

The account team has a small fix ready, but the same release holds an unfinished checkout change. The bundle ships only when all three areas pass together, so the fix waits.



1 bundle, 3 teams

every change ships in the same release

Blocked release

symptom: a fix in one area waits for unrelated areas

Account UI ships alone

target: released without rebuilding the catalog UI

Three ways to organize a frontend for several teams

Solution	What it is	What it does
Microfrontend	Frontend divided into independently developed and deployed modules	A shell loads each area’s UI at runtime from its own deployment, so each team ships on its own
Modular Frontend	One application organized into modules with enforced boundaries	Keeps one build and one release; teams work apart but ship together
Build-time composition	Areas published as versioned packages that a host application assembles	Gives each package an owner; the host rebuilds and redeploys to pick up a change

Independence costs runtime complexity, ownership without it costs coordination

Solution	Gains	Costs
Microfrontend	Independence: each team builds, tests and deploys its own area A release touches one area's artifact only	Runtime complexity: loading, shared dependency versions and duplicated framework code Look, navigation and cross-area state need an agreed contract
Modular Frontend	Simplicity: one build, one runtime and one dependency set Compile-time checks across modules	Shared release: a blocked module still blocks the others Build and test time grow with every team's code
Build-time composition	Module ownership: one package, one owning team, explicit versions Each package is tested on its own	Coordinated deploy: the host rebuilds and redeploys for any package change Version skew between host and packages needs management

Smallest safe move

One area as a runtime module

- **Scope:** the account UI only; catalog and checkout stay in the current bundle
- **Change:** build the account UI as a separate deployable module; the existing bundle becomes the shell and loads it at runtime
- **Safety net:** the in-bundle account UI stays behind a flag; framework and design system are shared singletons; contract tests cover what shell and module exchange
- **Escape route:** flip the flag back to the in-bundle account UI
- **Deferred:** catalog and checkout, until blocked releases are measured there; modular frontend stays right for areas that release together

```
new ModuleFederationPlugin({
  name: 'account',
  exposes: { './AccountRoutes': './src/AccountRoutes' },
  shared: { react: { singleton: true } },
})
```

Verification: ship the account UI alone

Check	Pass condition
Independent release	The account UI reaches production with no rebuild or redeploy of the catalog or checkout UI
Lead time	Merge-to-production time for the account area is shorter than the bundle release cycle, against a recorded baseline
Page load	p95 load and navigation time within the agreed budget, for example no more than 10 percent above baseline
Failure handling	With the account module unreachable, the shell shows the fallback and the other areas keep working
Contract	Contract tests between shell and module pass before each deploy
Fitness function	The pipeline fails when the account build needs changes in another area’s artifact or a shared dependency leaves the agreed version range

Solution architecture

Same three UI areas, now separate deployments loaded by a shell at runtime instead of one bundle with one release.

Glossary

Used in this step

Term	Definition
Web bundle	Single built artifact of the web application, released as one unit
Release coupling	Independent areas cannot be released without coordinated deployments
Microfrontend	Frontend divided into independently developed and deployed modules
Modular Frontend	One application organized into modules with enforced boundaries, built and released together
Build-time composition	Versioned packages assembled into one build by a host application
Shell	Host application that loads and mounts the area UIs at runtime
Module federation	Bundler feature that loads modules from other builds at runtime
Shared singleton	Dependency loaded once for shell and modules, such as the UI framework
Design system	Shared components and tokens that keep separate UIs consistent

Related concepts

Term	Definition
Backend for Frontend (BFF)	API layer tailored to a specific frontend or client
Modular Monolith	Single deployment with clearly separated internal modules
Microservices	Independently deployable services organized around business capabilities
DDD	Domain-driven design aligns software models with business concepts
API Gateway	Central entry point for routing, authentication, rate limiting, and API policies

Phase 6

Regional resilience and cost

Steps in this phase

Step	Forcing constraint
45 Authenticate service identities	Compromised service must not access payment data
46 Produce tamper-resistant audit evidence	Auditors need actor, time, operation, outcome and integrity
47 Survive region failure	Full region outage; checkout RTO at most 5 minutes
48 Avoid loss of acknowledged regional writes	RPO=0 for acknowledged payments under region loss
49 Isolate enterprise noisy neighbor	One tenant consumes 70% capacity and violates others' p95
50 Optimize transaction unit economics	Cost per 1000 orders must drop 30% with unchanged SLO
51 Verify recovery instead of assuming it	Drills exceed contracted RTO

A45 · Regional resilience and cost

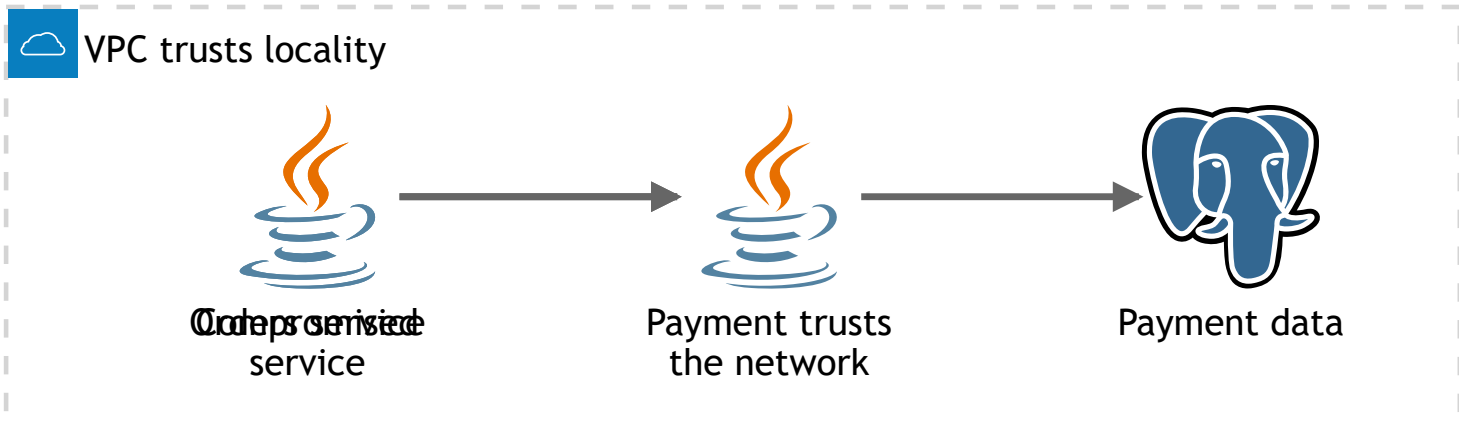
Authenticate service identities

Every service inside the VPC is trusted, so one compromised service can reach payment data.

Internal services trust the network, not the caller

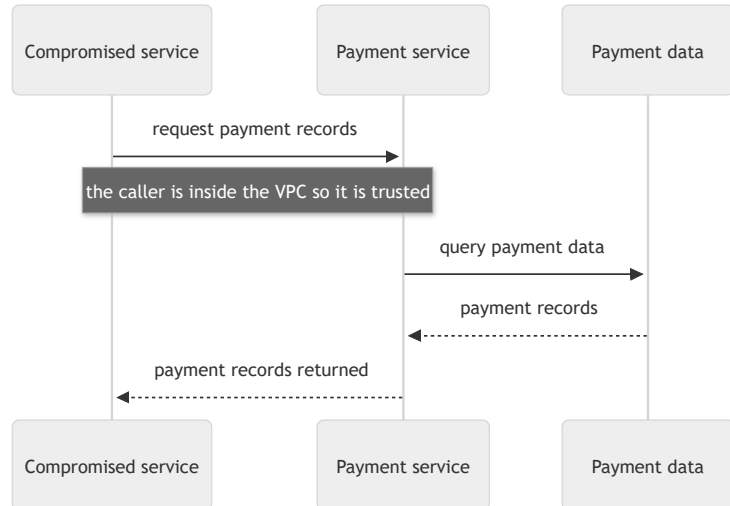
Component	What it is	What it does
API gateway	Single entry point for outside traffic	Authenticates shoppers and routes requests to services
Internal services	Containerized services on the container platform	Call each other directly over the internal network
Payment service	Service that owns payments and their data	Accepts every request that arrives from inside the VPC
Payment data	Database holding payment records	Accepts connections from anywhere inside the VPC
VPC	Private network around the platform	Treats being inside as proof of identity

Current architecture



One breach reaches payment data

An attacker takes over a low-risk internal service, for example through a vulnerable dependency. The payment service sees a request from inside the VPC and answers it, so there is no identity to check, limit or revoke.



Unauthorized call succeeds

a service with no payment role reads payment data today

Trust by location

every internal caller is accepted as it is

0 successful unauthorized calls

target: only verified and permitted callers get through

Three ways to verify the caller

Solution	What it is	What it does
Zero Trust with mTLS	Verification on every call: both sides prove a workload identity with certificates	Payment accepts only callers that hold a trusted certificate and are on its allow-list
Service mesh	A platform layer that manages service-to-service traffic, security and observability	Applies mTLS and access policy to every call from one place, without changing service code
Network isolation with RBAC	Network segments limit who can reach whom; roles limit what a caller may do	Blocks paths that no service needs and grants permissions through predefined roles

Stronger caller checks cost operations, platform or rules

Solution	Gains	Costs
Zero Trust with mTLS	Verified caller identity on every call Limits lateral movement after a compromise	Certificate operations: issuance, rotation and revocation An expired certificate becomes an outage
Service mesh	One uniform policy for all services No change to service code	Platform cost: control plane plus a proxy per workload Extra hop on every call and one more system to upgrade
Network isolation with RBAC	Layered controls with familiar tools No certificates to manage	Rules to write and keep current as services change Location is not identity: a compromised service in an allowed segment passes

Deciding signal: trust boundaries and permitted communication paths: which services may call payment, how many services share one policy and how much certificate rotation the platform team can run.

Smallest safe move

mTLS on the payment service only

- **Scope:** inbound calls to the payment service and its database; other service traffic stays as it is
- **Change:** short-lived workload certificates from an internal certificate authority; payment requires a client certificate and an allow-list of callers
- **Safety net:** report-only mode first, enforce when only expected callers remain; the database accepts only the payment service (least privilege)
- **Escape route:** return the policy to report-only; configuration only, no data changes
- **Deferred:** service mesh, until many services need the same policy; extra network rules and roles as added layers

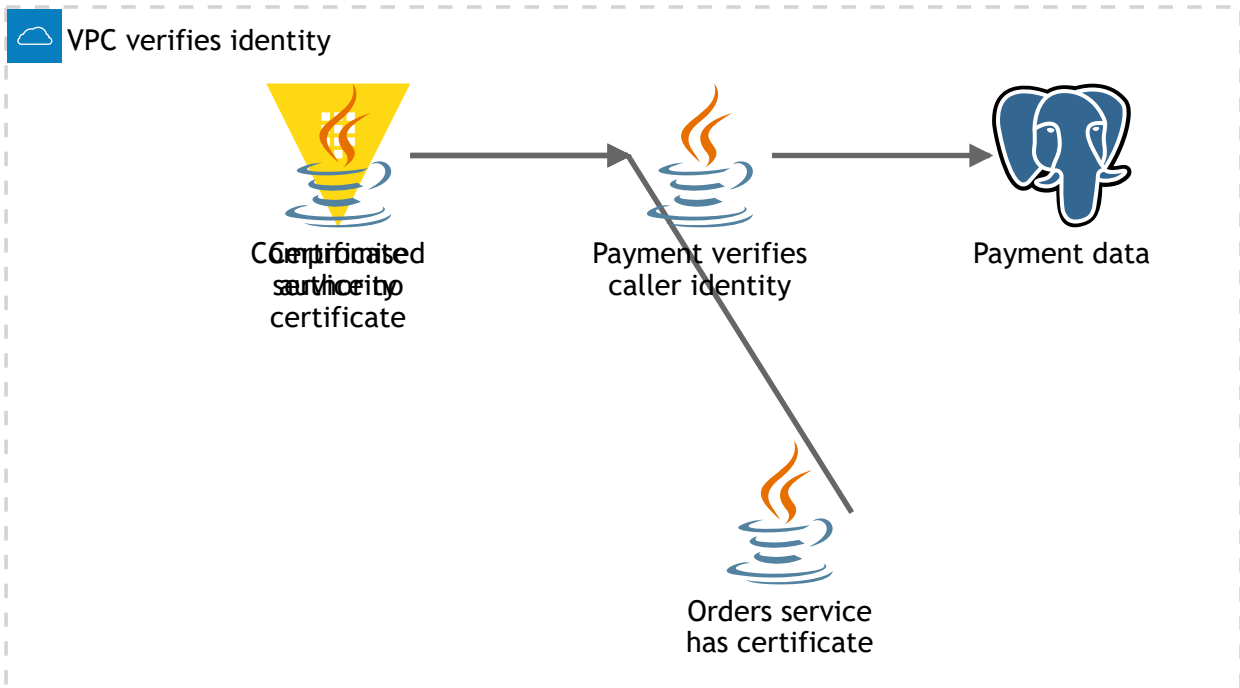
```
inbound:
  mode: report-only
  require-client-certificate: true
  trusted-ca: internal-ca
  allow-callers: [orders-service]
```

Verification: an unauthorized internal call

Check	Pass condition
Unauthorized call	A service without a certificate is refused at the handshake and receives no payment data
Wrong identity	A service with a valid certificate that is not on the allow-list is denied
Direct data access	A connection straight to the payment database from any other service fails
Legitimate path	Orders still reaches payment; checkout success rate and p95 stay at baseline, handshake cost recorded
Rotation	A forced certificate rotation completes with 0 failed calls; expiry alerts fire ahead of time
Fitness function	A scheduled job attempts an unauthorized internal call and fails the pipeline if one succeeds

Rollback: returning the policy to report-only restores trust by location; no business state is involved and issued certificates expire on their own.

Solution architecture



Same services and network; a certificate authority is new, and payment now verifies who is calling before it answers.

Glossary

Used in this step

Term	Definition
Zero Trust	Require verification for access rather than assuming network trust
mTLS	TLS in which both client and server present certificates and so prove their identity
Workload identity	Verifiable identity issued to a running service, not to a network address
Certificate authority	Issuer whose signature makes a certificate trusted by other services
Certificate rotation	Replacing certificates before they expire; short lifetimes limit a stolen one
Least privilege	Provide only necessary access for an actor or workload
Lateral movement	An attacker moving from one compromised system to others inside the network
Service mesh	Manages service-to-service traffic, security and observability
Network isolation	Segmenting the network so only needed communication paths exist
RBAC	Grants permissions through predefined roles

Related concepts

Term	Definition
TLS	Cryptographic transport protecting endpoints and data in transit
East-west traffic	Communication between components inside an infrastructure boundary
North-south traffic	Inbound or outbound communication crossing an infrastructure boundary
API gateway	Central entry point for routing, authentication, rate limiting and API policies
Secrets manager	Securely stores and distributes credentials and sensitive configuration
Audit log	Recorded business or security action with actor, time and outcome

A46 · Regional resilience and cost

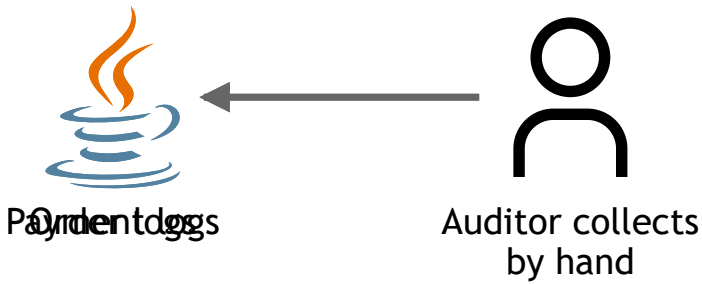
Produce tamper-resistant audit evidence

Auditors need actor, time, operation, outcome and integrity, but evidence is scattered across service logs.

Each service keeps its own logs

Component	What it is	What it does
Order service	Service that owns orders	Writes its own application log in its own format
Payment service	Service that owns payments	Writes a separate log with other fields and other retention
Service logs	Files or log streams kept per service	Hold what each developer chose to log, rotated after a short time
Auditor	Person who must reconstruct a financial operation	Collects logs from each team and joins them by hand
Log access	Operator and administrator privileges	Allow entries to be edited or deleted; nothing proves a log is complete

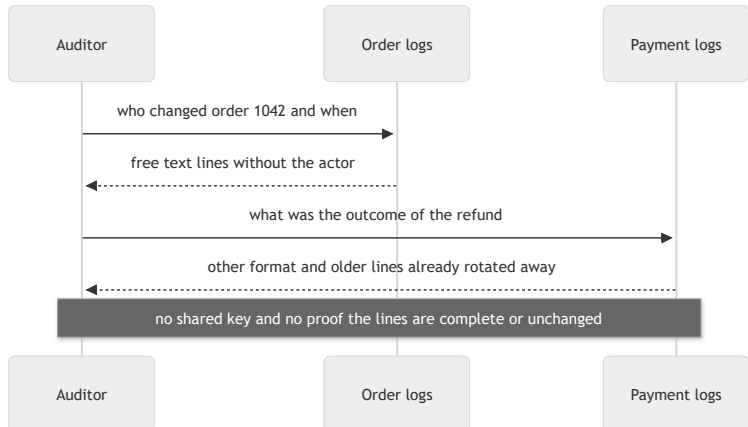
Current architecture



Order and payment logs stay inside their services, and the auditor assembles the evidence by hand.

Evidence exists, proof does not

An auditor asks who refunded an order, when, and whether it succeeded. Each service answers in a different format, and anyone with log access could have edited the lines. Sampled traces help debugging but are not a ledger.



10 days

audit preparation by hand today; the auditability driver aims for 2 days

5 required properties

actor, time, operation, outcome and integrity on every financial operation

99.99% within 60 sec

target: completed transactions with a queryable record

Three ways to reconstruct what happened

Solution	What it is	What it does
Immutable audit log	An append-only record of actor, time, operation and outcome, kept in storage that resists modification	Gives auditors one queryable ledger in which hash chaining makes tampering detectable
Event sourcing	Persisting state changes as an append-only sequence of events	Makes the event history the record of state, so any past state can be rebuilt and replayed
Distributed tracing	Tracking of requests across multiple services and dependencies	Shows the path and timing of each request, usually sampled and kept briefly

Evidence costs storage, schema discipline or completeness

Solution	Gains	Costs
Immutable audit log	Accountability with one place to query Tamper resistance from write-once storage and a hash chain	Retention and privacy: personal data in an append-only store cannot simply be deleted Storage grows with operations times retention
Event sourcing	Complete history, replayable to any point The audit trail follows from the data model	Event schema evolution: old events must stay readable for years The write model of a whole domain changes
Distributed tracing	Request visibility across services Quick to add when instrumentation already exists	Sampling leaves gaps and retention is short Not a legal ledger: records are not tamper evident

Deciding signal: retention requirement, audit granularity and event volume, and whether privacy rules require erasing recorded fields.

Smallest safe move

One append-only record per financial operation

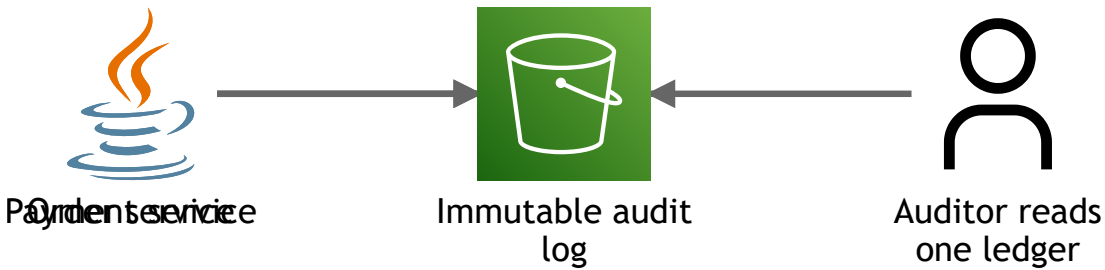
- **Scope:** payment and order state changes only; application logs and traces stay as they are
- **Change:** each service writes an audit record in the same transaction as the change, through a transactional outbox; a relay appends it to write-once storage with the hash of the previous record
- **Safety net:** the record commits with the business change, so no operation lacks one; records hold identifiers, not raw personal data
- **Escape route:** stop the relay and keep service logs; begin with a bypassable retention mode before the non-bypassable one
- **Deferred:** event sourcing, until state itself must derive from events

```
{ "seq": 5871, "actor": "user-77 via payment-service", "op": "refund order 1042",
  "outcome": "succeeded", "ts": "2026-11-02T09:14:05Z", "corr": "c-91f",
  "prev_hash": "9a3f0c", "hash": "c41e7b" }
```

Verification: audit a random transaction end to end

Check	Pass condition
End-to-end audit	A randomly chosen transaction yields actor, time, operation and outcome from the ledger alone, with no manual log collection
Completeness	Every completed financial transaction in the test window has a record; target at least 99.99% queryable within 60 sec
Integrity	Verifying the hash chain passes; a deliberately altered or removed record is detected
Immutability	An administrator’s delete or overwrite attempt is refused until retention ends
Privacy	A scan finds no card numbers or contact details in the records
Storage cost	Daily growth and the total at the retention horizon fit the agreed budget
Fitness function	A scheduled job verifies the chain and samples transactions; it fails on a gap or a hash mismatch

Solution architecture



Same services; one write-once ledger is new, and the auditor reads it instead of collecting logs from each service.

Glossary

Used in this step

Term	Definition
Audit log	Recorded business or security action with actor, time and outcome
Immutable storage	Data retention mechanisms resisting unauthorized modifications
WORM retention	Write once, read many: records cannot be changed or deleted until retention ends
Append-only	Records can be added but never edited or removed in place
Hash chain	Each record stores the hash of the previous one, so an edit or a gap breaks the chain
Tamper evidence	Ability to show that records were altered or removed after the fact
Transactional outbox	Atomically records business changes and outgoing events in the same database transaction
Event sourcing	Persists state changes as an append-only sequence of events
Distributed tracing	Tracks requests across multiple services and dependencies

Related concepts

Term	Definition
Centralized logging	Aggregates application logs from multiple services
Object storage	Stores files and binary objects in scalable buckets
Change data capture (CDC)	Detects and propagates changes made to database records
Event replay	Reconsume historical records to rebuild a projection
Least privilege	Provide only necessary access for an actor or workload

A47 · Regional resilience and cost

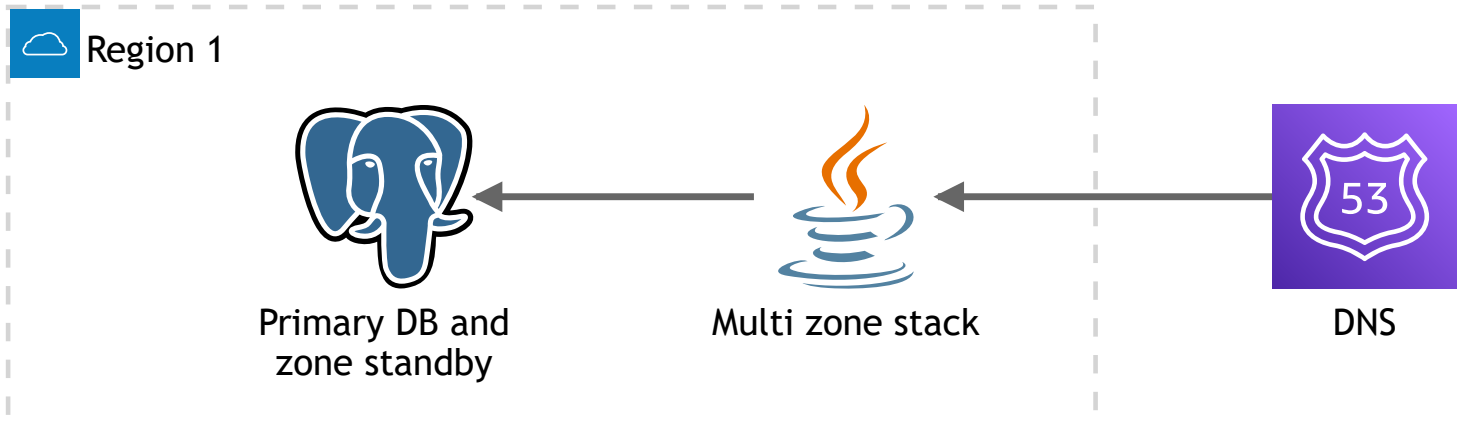
Survive region failure

A full region outage must not stop checkout for more than 5 minutes (RTO), yet the whole stack runs in one region.

One region, many zones, one failure domain

Component	What it is	What it does
DNS	Name resolution service	Resolves the shop’s domain to the single regional entry point
Regional entry point	Load balancer and gateway of the region	Spreads traffic over the availability zones
Service stack	Checkout, orders, payment and other services on the container platform	Runs across several zones of one region
PostgreSQL	Primary database with a standby in another zone	Survives the loss of one zone, not of the region
Cloud region	Geographic area containing several availability zones	Shares its network and control plane across all zones

Current architecture



Every zone fails together

A region-wide failure takes all zones down at once, so multi-AZ gives no protection. Checkout stays unavailable until the region returns, and nothing routes shoppers elsewhere.

Checkout down

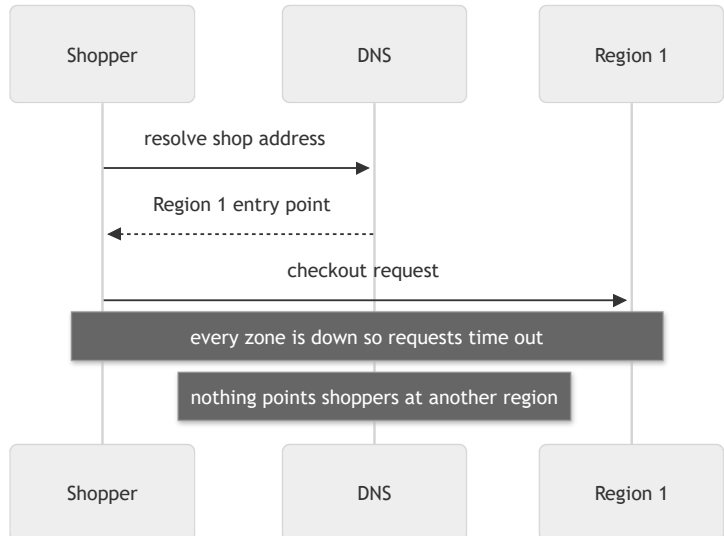
for the whole outage; multi-AZ survives a zone loss, not a region loss

1 region

one failure domain holds the whole stack and its data

RTO at most 5 min

target: checkout serves traffic from another region within 5 minutes



Three ways to run a second region

Solution	What it is	What it does
Warm standby with DNS failover	A second region with a scaled-down stack and a replicated database	Switches DNS on a failed health check, scales up the stack and promotes the replica
Hot standby	A second region with a fully ready, full-size copy of the stack	Takes over after a routing switch, with no start-up or scale-up wait
Active-active	More than one deployment actively processes traffic	Keeps serving from the surviving region with no switchover, while both regions share state

Faster recovery is bought with idle capacity or shared state

Solution	Gains	Costs
Warm standby with DNS failover	Moderate recovery time and cost A small standby idles at a fraction of the cost of a full copy	RTO adds up detection, DNS switch, scale-up and promotion, so it must be measured Resolvers may cache the old address until the TTL ends
Hot standby	Fast switchover: capacity is already running Fewer steps between failure and recovery	High infrastructure cost: a full second stack mostly idles Must be kept in sync and still needs a switch decision
Active-active	Rapid continuation during outages Both regions carry live traffic	Shared-state and conflict complexity: two regions accepting writes must reconcile Two full stacks to pay for and operate

Deciding signal: RTO and RPO targets, tolerance for two writers at once and the idle-capacity budget, each checked against measured failover time.

Smallest safe move

A warm standby region behind DNS failover

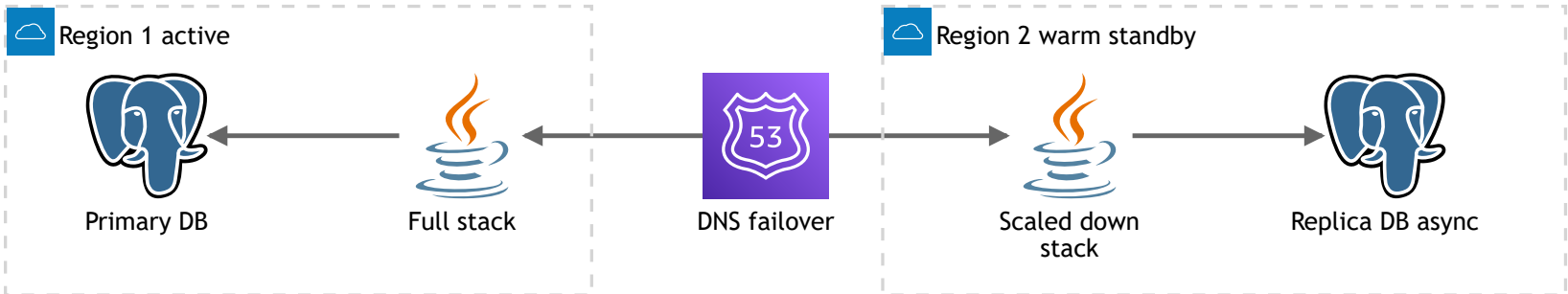
- **Scope:** the checkout path and its data; Region 1 stays the only region taking writes
- **Change:** deploy the checkout path to Region 2 from the same definitions at reduced capacity, replicate the database asynchronously and add health-checked DNS failover with a short TTL
- **Safety net:** promote the replica and fence the old primary, so two writers never coexist; a rehearsed runbook comes before any automation
- **Escape route:** point DNS back to Region 1; the standby keeps running
- **Deferred:** hot standby, until measured scale-up and promotion exceed the RTO; active-active, until single-writer failover cannot meet it

```
record: shop.example.com
ttl: 60   # seconds, for example
primary:   { target: region-1-entry, health-check: /healthz }
secondary: { target: region-2-entry, when: primary-unhealthy }
```

Verification: isolate the region and measure RTO

Check	Pass condition
Region isolation drill	Region 1 is cut off from traffic and its control plane; a synthetic checkout succeeds in Region 2 within 5 minutes
RTO breakdown	Time from first failed health check to first successful checkout, split into detection, DNS switch, scale-up and promotion
RPO	Acknowledged payments missing in Region 2 are counted separately from RTO; the recovery driver sets at most 30 sec
Capacity	After scale-up the standby carries the checkout peak load at normal p95
Client behavior	Share of clients still resolving Region 1 after the TTL is recorded
Failback	Return to Region 1 completes with no duplicate or missing orders and payments

Solution architecture



Same Region 1 stack and DNS; Region 2 is new, and health-checked DNS can move traffic to a scaled-down copy fed by an asynchronous replica.

Glossary

Used in this step

Term	Definition
Cloud region	Geographic area containing multiple cloud infrastructure facilities
Multi-AZ deployment	Run workload across separate availability zones
RTO	Maximum targeted recovery duration after disruption
RPO	Maximum targeted period of data loss
Failover	Switches operations to standby resources after failure
Warm standby	Partially initialized fallback environment
Hot standby	Fully ready fallback environment
Active-active	More than one deployment actively processes traffic
DNS	Resolves domain names to network addresses; the TTL sets how long answers may be cached
Health check	Detects whether an application instance is functioning correctly
Asynchronous replication	Acknowledge commit before all replicas confirm persistence
Write fencing	Prevent stale writers from modifying state after ownership changes

Related concepts

Term	Definition
Active-passive	One deployment processes traffic while another waits
Automated failover	Switch service or data authority on detected failure through orchestration
Disaster recovery site	Secondary infrastructure used to restore operations after major outages
Cold standby	Disaster recovery environment requiring substantial start or restoration
Multi-region replication	Maintain data copies across geographically separate cloud regions
Chaos engineering	Controlled experiments under injected failures

A48 · Regional resilience and cost

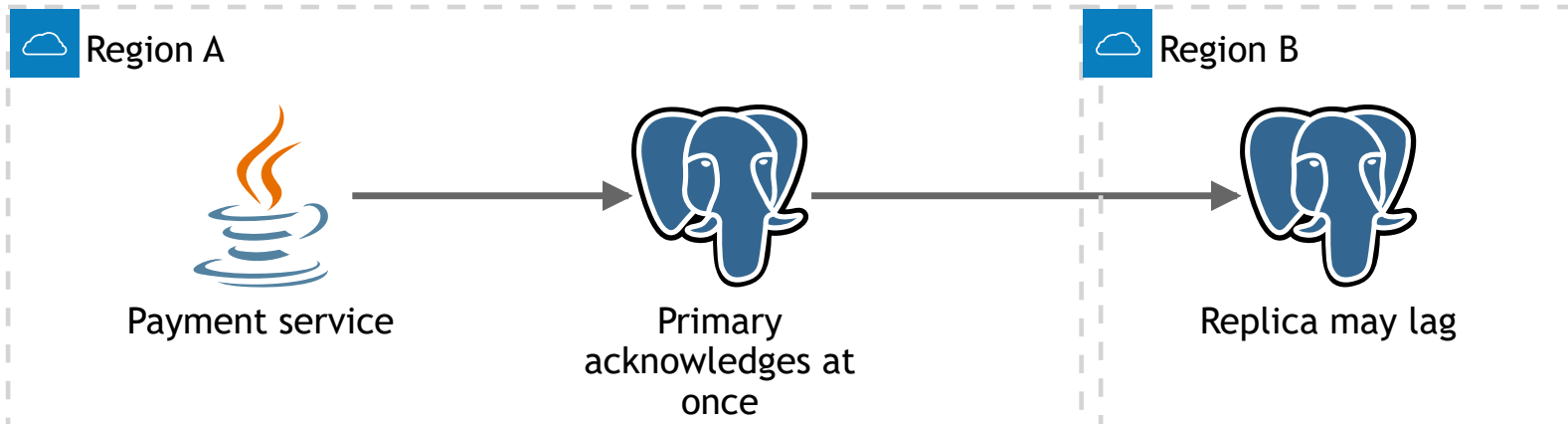
Avoid loss of acknowledged regional writes

Payments are acknowledged before the second region has them, but RPO=0 is required for acknowledged payments under region loss.

Payments are acknowledged before the other region has them

Component	What it is	What it does
Payment service	Service that records payments	Writes each payment to the primary database and confirms it to the customer
Primary database	PostgreSQL in Region A	Commits locally and acknowledges at once
Asynchronous replication	Background shipping of the transaction log to another region	Streams changes after the acknowledgement, so the replica lags
Replica database	PostgreSQL copy in Region B	Is promoted when Region A is lost; holds only what has arrived
Acknowledgement	The confirmation returned to the customer	Promises the payment is safe, although only Region A may hold it

Current architecture



A confirmed payment can vanish

Region A fails right after a payment is acknowledged. The replication stream had not shipped it yet, so Region B is promoted without it: the customer holds a confirmation and the system has no record.

Replica lag above 0

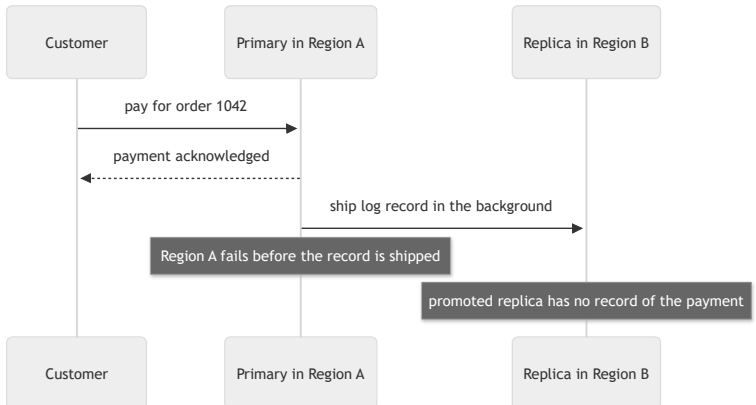
the window in which an acknowledged payment exists only in Region A

Region A lost

failure scope: the primary and its storage disappear together

0 acknowledged payments lost

target: RPO=0 under region loss



Three ways to treat an acknowledged payment

Solution	What it is	What it does
Synchronous cross-region quorum	A commit is acknowledged only after the required replicas, one in another region, persist it	Keeps every acknowledged payment through a region loss, but waits for an inter-region round trip
Asynchronous replication	A commit is acknowledged before replicas confirm; the log is shipped in the background	Keeps write latency local and the regions loosely coupled, while the replica lags behind
Read-only mode	A degraded mode that refuses payment writes while state cannot be trusted	Keeps reads and order lookups alive and stops new payments landing on possibly incomplete state

Zero loss costs latency, availability or service

Solution	Gains	Costs
Synchronous cross-region quorum	Protects against acknowledged-write loss within the supported failure model The surviving region holds the same payments, so failover needs no reconciliation	Higher write latency: every payment commit waits at least one inter-region round trip Reduced availability: writes stall or are refused when the other region or link is down
Asynchronous replication	Lower latency and weak coupling The primary keeps accepting writes while the link is down	Replication lag and possible loss: an acknowledged payment can vanish with the region RPO cannot be 0
Read-only mode	Correctness: no payment is taken on state that may miss acknowledged writes Reads and order history stay available	Degraded service: no new payments while it is active Does not recover writes that never left the failed region

Smallest safe move

Synchronous commit for payment writes only

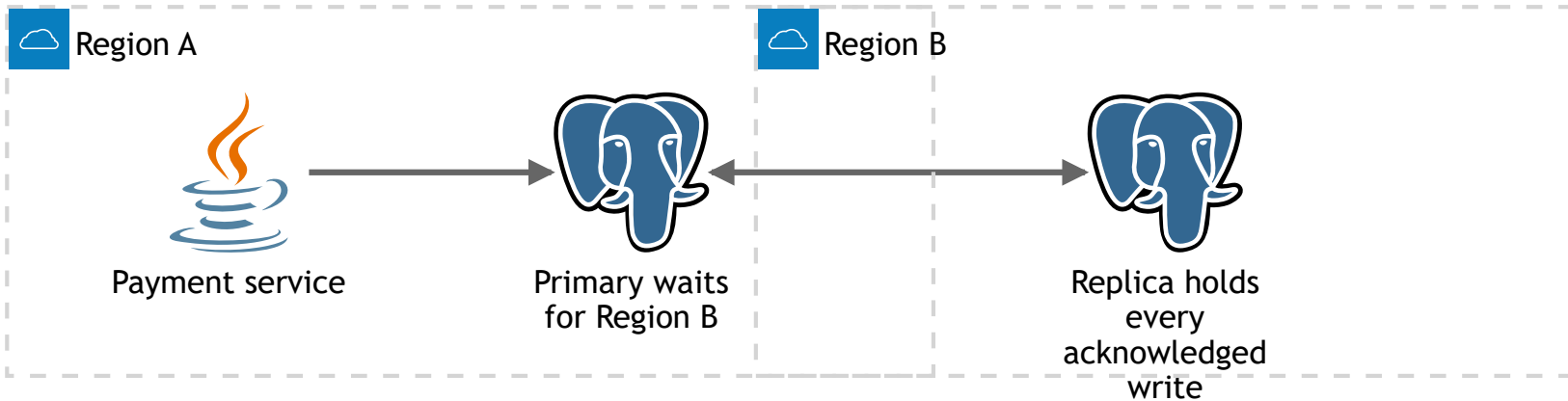
- **Scope:** the payment commit path; catalog, search and other data stay on asynchronous replication
- **Change:** a payment commit is acknowledged only after the replica in Region B has persisted it
- **Safety net:** fail closed: if Region B does not confirm within a timeout, the payment is rejected and retried with an idempotency key, never silently acknowledged
- **Escape route:** return the commit level to local; configuration only, and latency returns to its previous level
- **Deferred:** a third voting site and a planned read-only mode, until stalled payment writes during a link or region failure breach the availability target

```
-- synchronous_standby_names lists Region B, the default level is local
BEGIN;
SET LOCAL synchronous_commit = on;  -- this commit waits for Region B
INSERT INTO payment (order_id, amount) VALUES (1042, 59.90);
COMMIT;
```

Verification: a partition during commit

Check	Pass condition
Partition during commit	A region partition injected mid-commit leaves every acknowledged payment in the surviving region, 0 lost
Unacknowledged payments	A payment cut off before acknowledgement is absent or completes exactly once on retry, never twice
Region loss drill	Region B is promoted after losing Region A; all acknowledged payments are present and no customer is charged twice
Write latency	Payment commit p95 under replayed traffic stays within the agreed budget; the inter-region round trip is recorded as baseline
Write availability	Payment success rate during a link outage and time to resume after it heals are recorded; rejected payments return a clear error, not a hang

Solution architecture



Same payment service and databases; the primary now waits for Region B before acknowledging, so losing Region A cannot remove an acknowledged payment.

Glossary

Used in this step

Term	Definition
RPO	Maximum targeted period of data loss
Acknowledgement	Confirmation to the client that a write was accepted; it defines what the system promises to keep
Synchronous replication	Acknowledge a commit only after required replicas persist it
Asynchronous replication	Acknowledge commit before all replicas confirm persistence
Quorum	Number of replicas that must confirm a write before it is accepted
Replication lag	Delay between a commit on the primary and its arrival on a replica
Read-only mode	Degraded mode that refuses writes while state cannot be trusted
Idempotency key	Stable identifier recognizing multiple attempts at the same logical command
Fail closed	When state is in doubt, refuse the operation instead of proceeding unsafely
Network partition	Failure in which parts of a system cannot reach each other

Related concepts

Term	Definition
Multi-region replication	Maintain data copies across geographically separate cloud regions
WAL	Append transaction log records before database pages are modified
Database transactions	Atomic execution boundary for related data modifications
Point-in-time recovery	Restore a database to a selected moment using backups and logs
Failover	Switches operations to standby resources after failure
Write fencing	Prevent stale writers from modifying state after ownership changes

A49 · Regional resilience and cost

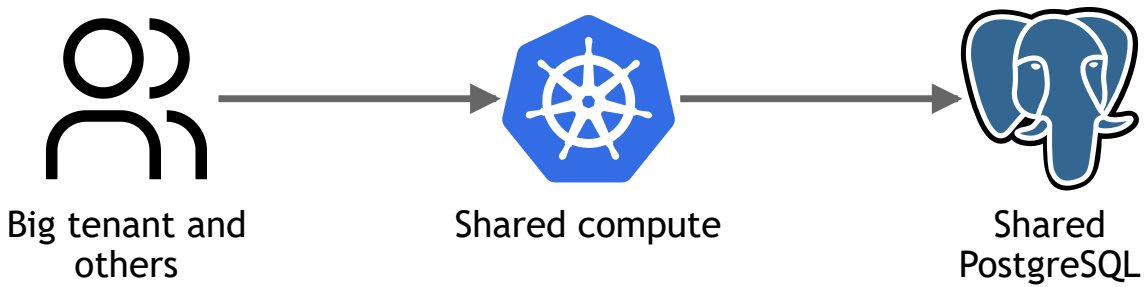
Isolate enterprise noisy neighbor

One tenant consumes 70% of shared capacity and pushes the p95 of every other tenant above its SLO.

All tenants share one compute pool

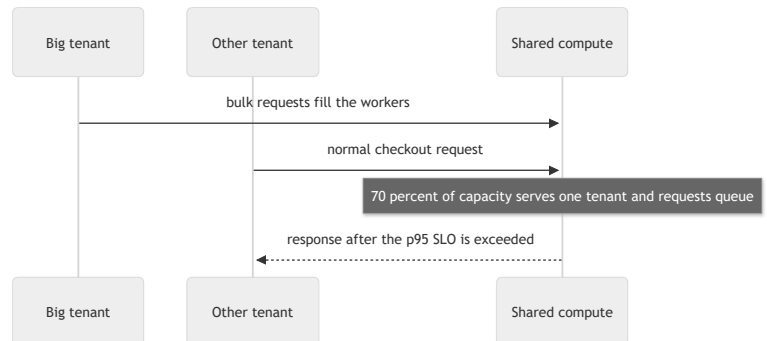
Component	What it is	What it does
Tenants	Business customers, each with many users, on one platform	Send catalog, checkout and order traffic under their own tenant id
Enterprise tenant	The largest customer account	Consumes 70% of the shared capacity
Shared compute	One pool of service replicas on the container platform	Runs every tenant’s requests on the same CPU, threads and connections
Shared PostgreSQL	One database serving all tenants	Executes every tenant’s queries within one connection budget
Tenant id	Identifier carried by each request	Names the caller; today nothing limits work by it

Current architecture



One tenant’s load becomes everyone’s latency

The enterprise tenant sends far more work than any other, and the shared pool serves it first come, first served. Requests of smaller tenants queue behind it, so their p95 rises although their own traffic did not change.



Others' p95 above SLO

symptom: tenants with unchanged traffic slow down

70% of capacity

consumed by a single tenant

Others' p95 within SLO

target: while one tenant sends abusive load

Three ways to keep one tenant from taking the shared capacity

Solution	What it is	What it does
Tenant bulkheads/quotas	Per-tenant limits on request rate and on concurrent work inside the shared pool	Counts work by tenant id and refuses the excess of a heavy tenant while others keep their share
Dedicated tenant pool	Compute, and optionally a database, reserved for one tenant	Moves the heavy tenant onto its own capacity, so it cannot take any from the shared pool
Tenant-based sharding	Several independent shards, each a full set of compute and data	Places each tenant on one shard, so a heavy tenant disturbs only the tenants on that shard

Isolation is bought with limits, idle capacity or rebalancing

Solution	Gains	Costs
Tenant bulkheads/quotas	Efficient: utilization of shared capacity stays high Limits are configuration, with no new infrastructure	Constrained: a heavy tenant is throttled at its quota even when others are idle Limits need tuning per tenant
Dedicated tenant pool	Strongest isolation: no shared CPU, memory or connections Scaling and maintenance timing per tenant	Cost: reserved capacity sits partly idle Another pool to deploy, patch and monitor in every region
Tenant-based sharding	Blast radius shrinks to one shard Shards grow independently	Rebalancing: moving a tenant moves its data and routing A shard shared with a heavy tenant still has a noisy neighbor

Smallest safe move

Per-tenant quotas and bulkheads in the shared pool

- **Scope:** requests carry a tenant id; the shared pool and database stay as they are
- **Change:** a per-tenant rate limit at the entry point plus a per-tenant cap on concurrent work in the service
- **Safety net:** log-only first; ceilings above each tenant's measured peak; excess refused before any state change with 429
- **Escape route:** limits are configuration; set log-only or raise them, and no data is touched
- **Deferred:** dedicated pool, until a contract demands isolation; sharding, until quotas cannot hold others' p95

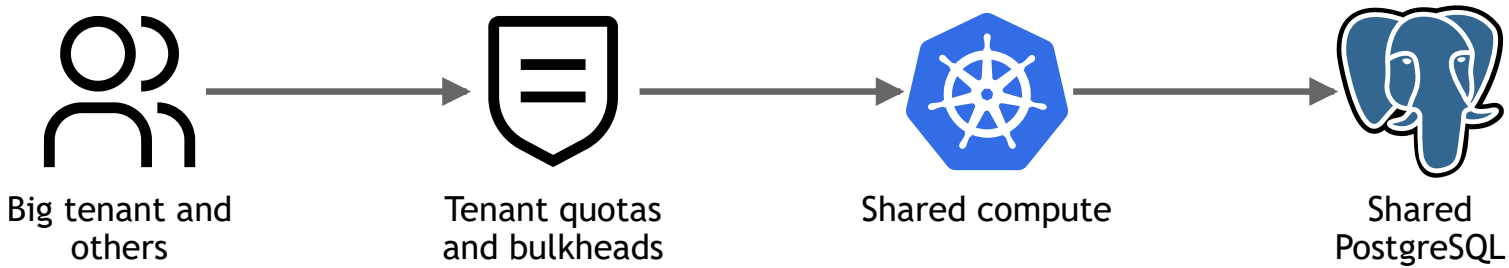
```
# illustrative ceilings for a pool of 200 workers
default:      { rate: 50/s,  maxConcurrent: 20 }
enterprise-1: { rate: 300/s, maxConcurrent: 80 }
```

Verification: one abusive tenant, everyone else’s p95

Check	Pass condition
Abusive tenant	Runs above its quota; its share of capacity stays at or below the quota instead of 70%
Other tenants’ p95	Within SLO during the abusive run, compared with a baseline recorded without abuse
Refusals	Excess of the heavy tenant answered with 429 and `Retry-After`; requests inside the quota succeed
Normal day	0 refusals when every tenant stays inside its limit; p95 unchanged from baseline
Utilization	Compute utilization recorded before and after, so the cost of isolation is visible
Fitness function	Pipeline replays one abusive tenant beside normal tenants and fails when another tenant’s p95 passes the SLO

Rollback: removing the limits restores shared behavior; no business state is involved, because refused requests never started. Moving a tenant to a dedicated pool or shard later is state and needs a data move back.

Solution architecture



Same shared compute and database; one new control point checks each request against its tenant's quota before it reaches the pool.

Glossary

Used in this step

Term	Definition
Noisy neighbor	One customer workload degrades others sharing resources
Multi-tenancy	Shares application resources among customers
Bulkhead pattern	Isolates resources so failures do not spread across components
Tenant quota	Ceiling on the request rate or concurrent work one tenant may use
Rate limiter	Controls the number of requests allowed within a time window
Concurrency cap	Maximum number of requests one tenant may have in progress at once
Dedicated tenant pool	Compute, and optionally data, reserved for one tenant
Tenant-based sharding	Database sharding by tenant; partitions data across multiple database instances
Blast radius	How many users or functions one failure affects
Capacity utilization	Share of provisioned capacity doing useful work; isolation tends to lower it

Related concepts

Term	Definition
API gateway	Central entry point for routing, authentication, rate limiting, and API policies
Data partitioning	Divide a logical dataset into smaller sections, often within one database
Horizontal scaling	Distribute workload across more instances
Autoscaling group	Automatically adjusts compute capacity based on demand
Container orchestrator	Automates container scheduling, scaling, and recovery
Reserved capacity	Committed use of resources or spend for discounted pricing

A50 · Regional resilience and cost

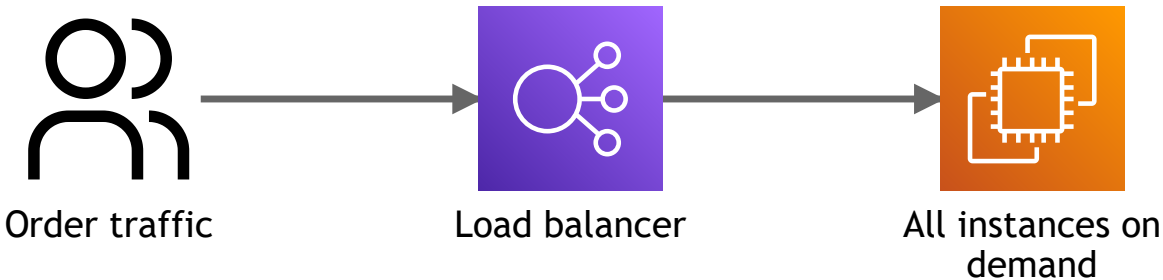
Optimize transaction unit economics

Cost per 1000 orders must drop 30% while the SLO stays unchanged.

Every instance-hour is bought on demand

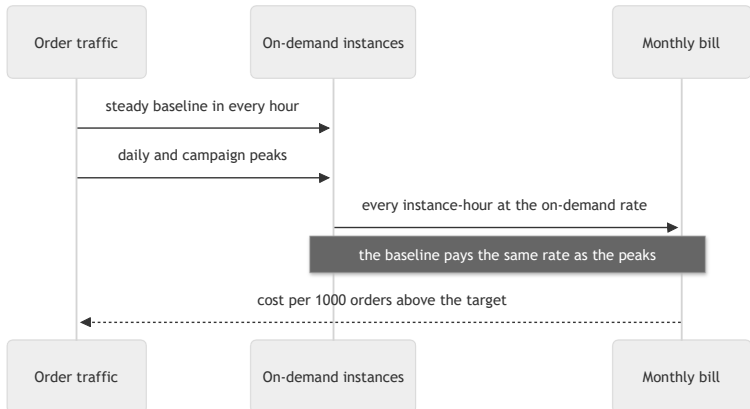
Component	What it is	What it does
Order traffic	Checkout and order requests from shoppers	Rises and falls with the daily cycle and with campaigns
Load balancer	Entry point that spreads requests over the fleet	Sends each request to an available instance
Instance fleet	Virtual machines running the platform’s services	Runs all workloads, sized for peak demand
On-demand pricing	Pay-as-you-go rate with no commitment	Charges the list price for every instance-hour, baseline included
Cost per 1000 orders	Allocated infrastructure spend divided by successful orders	The unit-economics metric that has to fall

Current architecture



The steady baseline pays the peak rate

A baseline of traffic runs in every hour of the month, with daily peaks and occasional campaigns on top. All compute is billed on demand, so the predictable baseline pays the same rate as the unpredictable peaks. Latency and availability targets cannot move.



Cost per 1000 orders

symptom: allocated spend divided by successful orders

100% on demand

every instance-hour at the list rate, baseline included

30% lower, same SLO

target: cost per 1000 orders falls while latency and availability hold

Four ways to pay less for the compute behind each order

Solution	What it is	What it does
Reserved Capacity	Committed use of resources or spend at a discounted price, for a fixed term	Discounts the steady baseline; usage above it is still billed on demand
Spot Instance workers	Discounted compute that the provider can interrupt	Runs fault-tolerant background workers cheaply and reschedules them after an interruption
Autoscaling/serverless	Capacity that follows demand: scaled instance groups or per-invocation functions	Removes idle capacity in quiet hours, so billing follows the load
Managed hosting	A service where the provider operates the platform, such as a managed container platform or database	Moves patching, scaling and backups to the provider, at a price that includes its margin

Each saving is paid for with commitment, interruption, variability or margin

Solution	Gains	Costs
Reserved Capacity	Cheap predictable baseline No architecture change: same instances, no interruption	Long-term commitment: unused hours are still paid Sized above the real baseline, it raises cost per order
Spot Instance workers	Low cost for fault-tolerant workloads Large discount against on-demand for interruptible work	Interruption and rescheduling: work must survive loss at any moment Capacity is not guaranteed, so it is unfit for the checkout path
Autoscaling/serverless	Pays for demand: idle capacity shrinks Absorbs spikes without pre-provisioning	Variable bill that is harder to forecast Scale-up lag and cold starts; serverless is expensive at sustained high load
Managed hosting	Less operations work: patching, backups, failover	Provider margin inside the price Provider constraints and less control

Smallest safe move

Reserved capacity for the measured baseline only

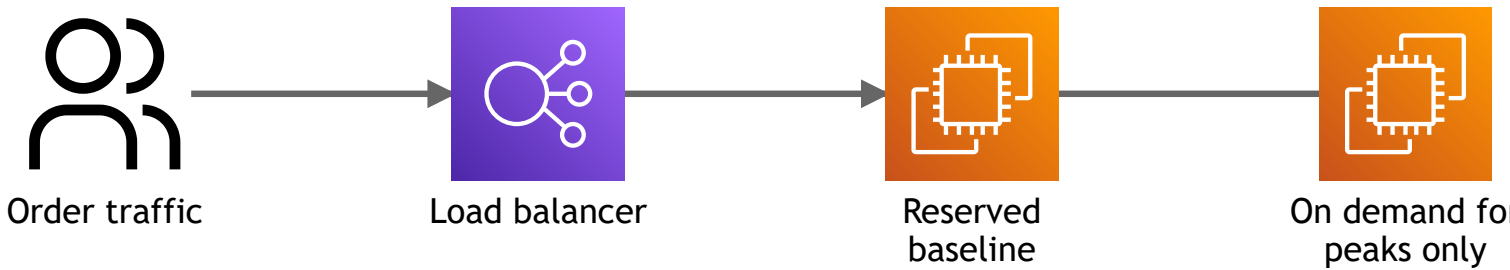
- **Scope:** the steady baseline of the stateless services; instances, routing and SLOs stay as they are
- **Change:** commit to the lowest sustained hourly usage of recent months; peaks stay on demand
- **Safety net:** right-size first, then commit below the measured floor so every committed hour is used
- **Escape route:** nothing in the platform changes; the commitment runs to its term, so it is bought in steps
- **Deferred:** spot workers, until jobs tolerate interruption; autoscaling/serverless, until peaks dominate cost; managed hosting, until operations effort does

```
commitment = lowest sustained hourly usage in recent months   (not the average)
cost per 1000 orders = allocated spend / successful orders x 1000
```

Verification: full cost per successful order, typical day and peak day

Check	Pass condition
Cost per 1000 orders	At least 30% below the on-demand baseline, from allocated spend over successful orders
Typical and peak day	The reduction holds on both, measured separately
Full cost	Compute, storage, data transfer and operations effort are counted, not only instance price
SLO	Latency p95 and availability at or within baseline; peaks served by on-demand capacity with no extra errors
Commitment utilization	Nearly every committed hour is used, for example above 95%
Fitness function	Monthly cost per 1000 orders computed from billing and order data; alert above target or when utilization drops

Solution architecture



Same traffic, load balancer and instances; only the billing of the steady baseline changes, and only peaks pay the on-demand rate.

Glossary

Used in this step

Term	Definition
Cost per 1000 orders	Allocated infrastructure spend divided by successful orders; counts full cost (TCO), not only instance price
On-demand pricing	Pay-as-you-go compute price with no commitment
Reserved capacity	Committed use of resources or spend for discounted pricing
Baseline load	Lowest sustained usage; the part of demand present in every hour
Commitment utilization	Share of committed hours or spend that is actually used
Spot instance	Discounted compute capacity that can be interrupted
Autoscaling group	Automatically adjusts compute capacity based on demand
Serverless function (FaaS)	Executes short-lived functions triggered by events or requests
Cold start	Delay while a new instance or function initializes before its first request
Managed hosting	Provider operates the platform, such as a managed container platform, for a service fee

Related concepts

Term	Definition
Load balancer	Distributes incoming requests across multiple servers or instances
Horizontal scaling	Distribute workload across more instances
Vertical scaling	Add CPU/memory to one host
Batch processing	Process groups of work together
Multi-tenancy	Shares application resources among customers
Fitness function	Measurable or executable architectural quality rule

A51 · Regional resilience and cost

Verify recovery instead of assuming it

Recovery drills take longer than the contracted RTO of 5 minutes.

Recovery is documented but never exercised end to end

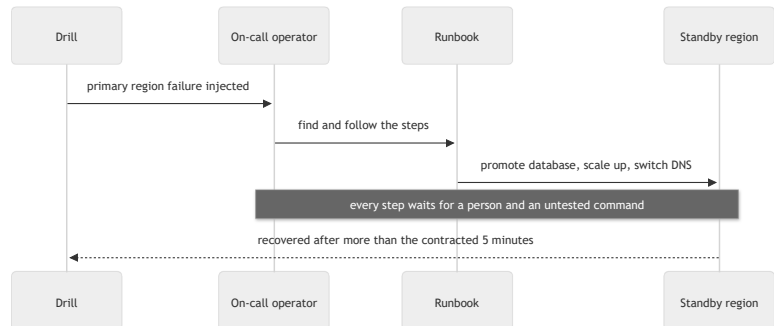
Component	What it is	What it does
Runbook	Written procedure for losing the primary region	Lists the steps an on-call engineer follows, such as promote the database and switch DNS
Standby region	The second region that takes over when the primary fails	Holds a copy of the platform and its data, as built for region failure
Backups	Snapshots and logs of the data	Exist on schedule; the time to restore them has never been measured
Metrics and alerts	Dashboards and paging on service health	Show that the service is down; they do not record recovery time or data loss
Contracted RTO and RPO	Recovery targets promised to the business	Allow 5 minutes of outage for checkout and a bounded data loss

Current architecture



The first drill takes longer than the contract allows

Region loss is rehearsed for the first time. Operators follow the runbook by hand, and slow steps such as detection, database promotion and DNS switching add up. The documentation said recovery would work, and nobody had measured it.



Drill RTO above 5 minutes

symptom: measured recovery is slower than the contract

Never run end to end

condition: runbooks were written but not exercised under failure

RTO and RPO within contract

target: both measured in a repeatable drill

Three ways to learn how recovery really behaves

Solution	What it is	What it does
Chaos Engineering	Controlled experiments under injected failures	Injects a defined failure in a scheduled game day, such as loss of a region, and measures where recovery time goes
Automated Fitness Functions	Measurable or executable architectural quality rules	Run restore tests and failover rehearsals on a schedule or on each change, and fail when RTO or RPO exceeds the contract
Runbooks/observability	Written recovery procedures plus metrics, logs and alerts	Guide operators through recovery and show what is failing; the speed depends on the person

Each option trades test risk, maintenance or human speed

Solution	Gains	Costs
Chaos Engineering	Validates resilience claims with measured RTO and RPO Finds hidden dependencies before a real outage	Production test risk: an experiment can cause a real outage Needs engineering time, a limited blast radius and an abort condition
Automated Fitness Functions	Repeatable on every change, so recovery time cannot regress unnoticed Cheap to run once written	Maintenance and false positives from flaky checks A test environment can differ from production
Runbooks/observability	Quick to write and update Operators see what fails and act on it	Manual: speed and accuracy depend on a person under stress Documents drift from the system and prove nothing

Smallest safe move

One timed drill with a hypothesis and an abort condition

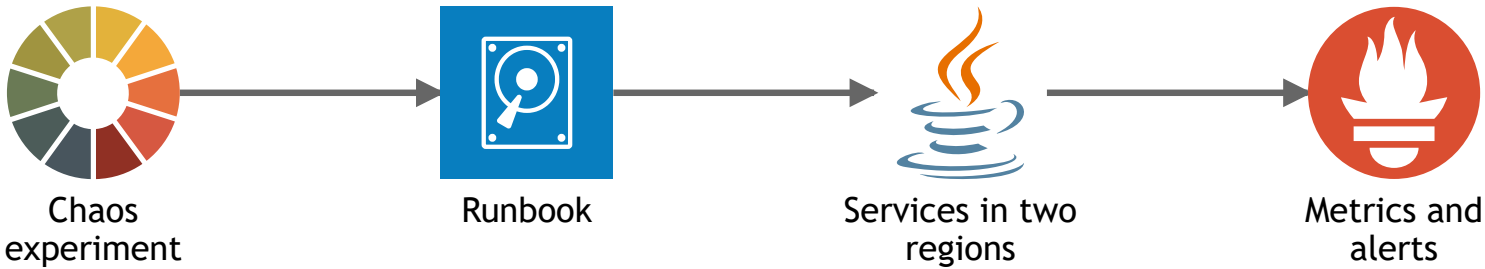
- **Scope:** loss of the primary region for checkout, in a production-like environment with synthetic orders
- **Change:** inject the failure, follow the runbook as written, and time each step to the first successful checkout
- **Safety net:** hypothesis and abort condition written first; failback planned before any promotion
- **Escape route:** abort returns traffic to the primary region; synthetic data on the standby is discarded
- **Deferred:** fitness functions, once the drill repeats; automated failover, for the slowest step that still breaks the RTO

```
# illustrative experiment definition, not tied to a tool
hypothesis: checkout recovers in the standby region within 5 minutes and within the contracted RPO
inject: isolate the primary region from users and from the standby
abort_if: error rate on unaffected traffic above 1 percent
measure: [time_to_first_successful_checkout, acknowledged_orders_missing_on_standby]
```

Verification: measured RTO and RPO under controlled failure

Check	Pass condition
RTO	Timeline records detection, decision, database promotion, traffic switch and first successful checkout; total at most 5 minutes in repeated drills
RPO	Acknowledged orders missing on the standby stay within the contracted RPO
Invariants	No oversold stock and no duplicated payment after failover or failback
Failback	Traffic and data authority return to the primary region and state is reconciled within the agreed time
Experiment safety	Unaffected traffic stays within SLO; the abort condition ends the experiment when it trips
Fitness function	A scheduled restore test or failover rehearsal records RTO and RPO and fails when either exceeds the contract

Solution architecture



Same runbook, platform and metrics; a chaos experiment now injects the failure on purpose, and the metrics record the real RTO and RPO.

Glossary

Used in this step

Term	Definition
Chaos engineering	Controlled experiments under injected failures
Recovery drill	Exercise failover/restoration to measure actual RTO/RPO
Game day	Scheduled exercise in which a team injects a failure and practices the response
RTO	Maximum targeted recovery duration after disruption
RPO	Maximum targeted period of data loss
Runbook	Written step-by-step procedure for operators during an incident
Fitness function	Measurable or executable architectural quality rule
Restore test	Restoring a backup into a separate environment to measure restore time and data integrity
Hypothesis and abort condition	Prediction written before an experiment, and the threshold that stops it
Blast radius	How many users or functions one failure affects
Failback	Return of traffic and data authority to the original region after recovery
Metrics and alerts	Numerical measurements of system behavior, and notifications when they pass thresholds

Related concepts

Term	Definition
Backup and restore	Recovery process using historical snapshots and logs
Disaster recovery site	Secondary infrastructure used to restore operations after major outages
Automated failover	Switch service/data authority on detected failure through orchestration
Warm standby	Partially initialized fallback environment
Infrastructure as code	Version-controlled machine-readable infrastructure configuration
MTTR	Average recovery duration across incidents